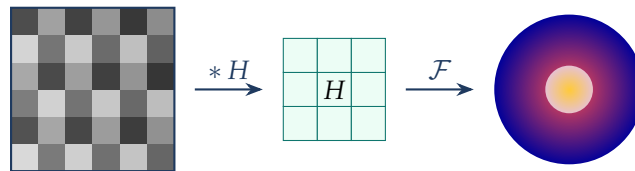

Deep Learning for Computer Vision

Week 1 — Complete Study Notes

Introduction • History • Image Formation • Image Representation
Linear Filtering, Correlation & Convolution • Frequency Domain • Sampling & Interpolation



Based on the IITM / NPTEL course by Prof. Vineeth N. Balasubramanian (IIT Hyderabad)

Lectures §1.1 – §1.7

How to use these notes. Each module mirrors one lecture. Coloured boxes mark intuition, key ideas, derivations, exam tips and pitfalls. Dimensions of every quantity are annotated in teal brackets like $I [H \times W \times C]$. Worked numerical problems and a self-test MCQ bank are at the end, followed by a one-page formula sheet. Where the slides contain a typo or an over-simplification, a *Careful* box flags it.

Contents

1	Course Introduction: What is Computer Vision and Why?	4
1.1	What is Computer Vision?	4
1.2	Why? Applications of Computer Vision	4
1.3	Perspectives of Study	5
1.4	Why is Computer Vision Hard?	5
1.4.1	Evidence from human perception: visual illusions	6
1.4.2	Reason 1: Vision is an inverse problem	7
1.4.3	Reason 2: High-dimensional data	7
1.4.4	Reason 3: Computer vision is “AI-complete”	7
1.4.5	Reason 4: No complete model of the human visual system	7
1.4.6	Reason 5: Verifiability of mathematical/physical models is non-trivial	7
1.5	This Course: Topics, Structure, Objectives	8
1.5.1	The three pillars of computer vision	8
1.5.2	Course segments	8
1.5.3	Eligibility / prerequisites	8
1.6	Resources and References	9
2	History of Computer Vision	10
2.1	Early History: Initial Forays (1959–1982)	10
2.1.1	1959 — Hubel & Wiesel: receptive fields in the cat’s visual cortex	10
2.1.2	1963 — The first digital image; Roberts’ thesis	10
2.1.3	1966 — The MIT Summer Vision Project	11
2.1.4	1971–73 — Line labelling, shape, and pictorial structures	11
2.1.5	1982 — David Marr, <i>Vision</i>	12
2.1.6	1980 — Fukushima’s Neocognitron: the original ConvNet	12
2.2	Towards Algorithms and Practice: Low-level Understanding (1981–1989)	12
2.2.1	1981 — Optical flow (Horn & Schunck)	13
2.2.2	1986 — Canny edge detector	13
2.2.3	1987 — Recognition-by-Components (Biederman)	13
2.2.4	1988 — Snakes (active contour models)	13
2.2.5	1989 — Backprop for CNNs arrives	14
2.2.6	Other low-level milestones of the 1980s	14
2.3	Towards Algorithms and Practice: Next-level Understanding (1991–2009)	14
2.4	The Deep Learning Era (2010–)	15
2.5	History of Applications	16
3	Image Formation	17
3.1	Image Formation: An Overview	17
3.2	Photometry: Models of Reflection	17
3.3	Colour	19
3.3.1	The Bayer grid / filter	19
3.4	Digital Image Sensing	20
3.4.1	The image sensing pipeline	20
3.4.2	CCD vs. CMOS	21

3.4.3	Digital image sensor properties	21
3.4.4	Popular question: with smartphones, do you need DSLRs?	22
3.5	Sampling and Aliasing	22
3.6	Colour Space Representations	22
3.7	Image Compression	23
4	Image Representation	25
4.1	Why RGB if the spectrum is VIBGYOR?	25
4.2	Image as a Matrix	25
4.3	Image as a Function	26
4.4	Image Transformations	27
4.5	Image Processing Operations: Point, Local, Global	27
4.6	Point Operations	28
4.6.1	Example 1: Reversing the contrast (negative)	28
4.6.2	Example 2: Linear contrast stretching	28
4.6.3	Going beyond linear stretching: Histogram Equalisation (homework — derived fully)	28
4.6.4	How useful are point operations?	30
4.7	Local Operations: The Moving Average	30
4.8	Global Operations	31
5	Linear Filtering, Correlation and Convolution	32
5.1	Review: Histogram Equalisation	32
5.2	Image Filters	32
5.3	Correlation and Cross-Correlation	32
5.3.1	Correlation with uniform weights (moving average)	32
5.3.2	Cross-correlation with non-uniform weights	33
5.4	Averaging Filters	33
5.4.1	Moving average = box filter	33
5.4.2	Gaussian averaging filter	33
5.5	Other Filters: The Edge Filter	34
5.6	Beyond Correlation: The Impulse Response	35
5.7	Introducing Convolution	36
5.7.1	Biological motivation: linear summation in the visual cortex	36
5.8	Linear Shift-Invariant (LSI) Operators	36
5.9	Properties of Convolution	37
5.10	Separability	38
5.10.1	How can we tell if K is separable?	38
5.11	Practical Issues	39
5.11.1	Ideal size for the filter?	39
5.11.2	What about the boundaries?	39
5.12	Questions to Think About	40
6	Image in the Frequency Domain	41
6.1	Is Correlation Still Useful? Template Matching	41
6.2	Non-Linear Filters	41
6.2.1	Types of noise	41
6.2.2	Median filter	42
6.2.3	Bilateral filtering	42
6.3	What Do We Lose in a Low-Resolution Image?	43

6.4	Fourier's Idea	43
6.4.1	A sum of sines	44
6.4.2	Building a square wave	44
6.5	Fourier Analysis of Images	45
6.6	Fourier Transform: Magnitude and Phase	46
6.6.1	Which has more information: magnitude or phase?	46
6.7	Continuous vs. Discrete Fourier Transform	47
6.8	The Convolution Theorem	47
6.9	Properties of the Fourier Transform	48
6.10	The Fast Fourier Transform (FFT)	48
6.11	Filtering in the Spatial vs. Frequency Domain	49
6.11.1	Low-pass and high-pass filters	49
6.12	Hybrid Images	50
6.13	Exercise: Match Spatial Image to Fourier Magnitude	50
7	Image Sampling and Interpolation	52
7.1	Cost Improvement Using the Convolution Theorem	52
7.2	What Sense Does a Low-Resolution Image Make to Us?	53
7.3	Sub-sampling	53
7.4	Aliasing	53
7.4.1	Aliasing: problems in practice	54
7.5	Anti-Aliasing	55
7.6	Upsampling and Interpolation	55
7.6.1	Interpolation as reconstruction	55
7.6.2	Interpolation as convolution	56
7.6.3	Types of interpolation (reconstruction kernels)	57
7.7	Interpolation and Decimation	58
A	Numerical Problems with Worked Solutions	59
A.1	Modules 1–3: Images, Image Formation, Sensors, Compression	59
A.2	Module 4: Point Operations	61
A.3	Module 5: Correlation, Convolution, Separability	62
A.4	Module 6: Non-linear Filters and Fourier	64
A.5	Module 7: Cost, Sampling, Aliasing, Interpolation	65
B	Multiple-Choice Self-Test	68
B.1	Module 1: Introduction	68
B.2	Module 2: History	68
B.3	Module 3: Image Formation	69
B.4	Module 4: Image Representation	71
B.5	Module 5: Linear Filtering	72
B.6	Module 6: Frequency Domain and Non-linear Filters	73
B.7	Module 7: Sampling and Interpolation	75
C	Formula Sheet	77

Course Introduction: What is Computer Vision and Why?

1.1 What is Computer Vision?

The lecture opens with three everyday visual puzzles:

1. *Where is the glue stick?* — a cluttered desk scene. You find it in well under a second.
2. *Find the book — what is its full title?* — a bookshelf photo. You must localise the book, then read a rotated spine.
3. *What is wrong with this image?* — a surreal photo-manipulation (Erik Johansson). You need world knowledge (physics, scale, gravity) to spot the “impossibility”.

Can a machine answer these questions? Each requires a different mix of *detection*, *recognition*, *reading* (OCR) and *common-sense reasoning*. That spread is the whole subject.

Key Idea: Definition used in this course

Computer Vision is a field that seeks to automate and endow a computing framework with the ability to *interpret images the way humans do*. It is a sub-topic of Artificial Intelligence.

Other classical definitions.

- “the construction of explicit, meaningful descriptions of physical objects from images” — Ballard & Brown (1982).
- “computing properties of the 3D world from one or more digital images” — Trucco & Verri (1998).
- “to make useful decisions about real physical objects and scenes based on sensed images” — Stockman & Shapiro (2001).

Intuition: Three flavours of the same definition

Read the three definitions as three *outputs* a vision system may produce: **descriptions** (labels, captions — Ballard & Brown), **3D properties** (depth, shape, motion — Trucco & Verri) and **decisions/actions** (brake, grasp, flag a tumour — Stockman & Shapiro). Modern deep vision does all three, and this course is mostly about the first and third.

1.2 Why? Applications of Computer Vision

More applications (homework: follow the links on this slide):

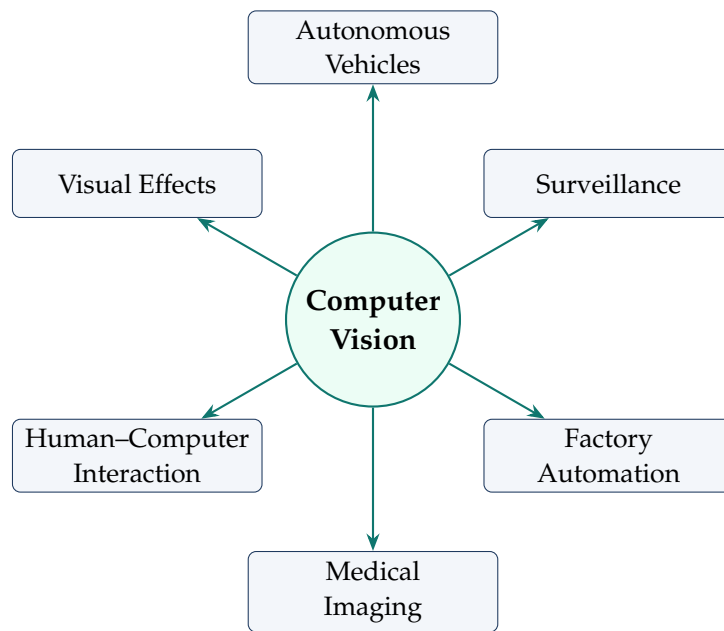


Figure 1.1: The six headline application areas shown on the lecture slide.

Domain	Examples
Retail & retail security	Amazon Go (cashier-less stores), virtual try-on, StopLift (checkout fraud)
Healthcare	Blood-loss detector (surgical sponge imaging), DermLens (skin conditions)
Agriculture	SlantRange (drone crop analytics), Cainthus (livestock facial recognition)
Banking & finance	Mobile cheque deposit, insurance risk profiling
Remote sensing	Land-use understanding, forestry modelling
Structural health monitoring	Oil-well inspection, drone-based bridge inspection & 3D reconstruction
Document understanding	OCR, Robotic Process Automation
Tele- & social media	Image understanding, brand-exposure analytics
Augmented reality	TechSee visual support, warehouse & enterprise management

1.3 Perspectives of Study

Computer vision sits at the intersection of many disciplines. The slide's diagram shows CV, Machine Vision and Image Processing as three overlapping circles, surrounded by the fields that feed into them.

Intuition: CV vs. Image Processing vs. Machine Vision

Image processing: image $\rightarrow$ image (denoise, sharpen, compress). **Computer vision:** image $\rightarrow$ *understanding* (labels, geometry, decisions). **Machine vision:** the industrial/engineering deployment of vision (inspection on a factory line, smart cameras). The overlaps are large: Week 1 of this course is mostly image processing, because it is the foundation that CNNs later *learn* automatically.

1.4 Why is Computer Vision Hard?

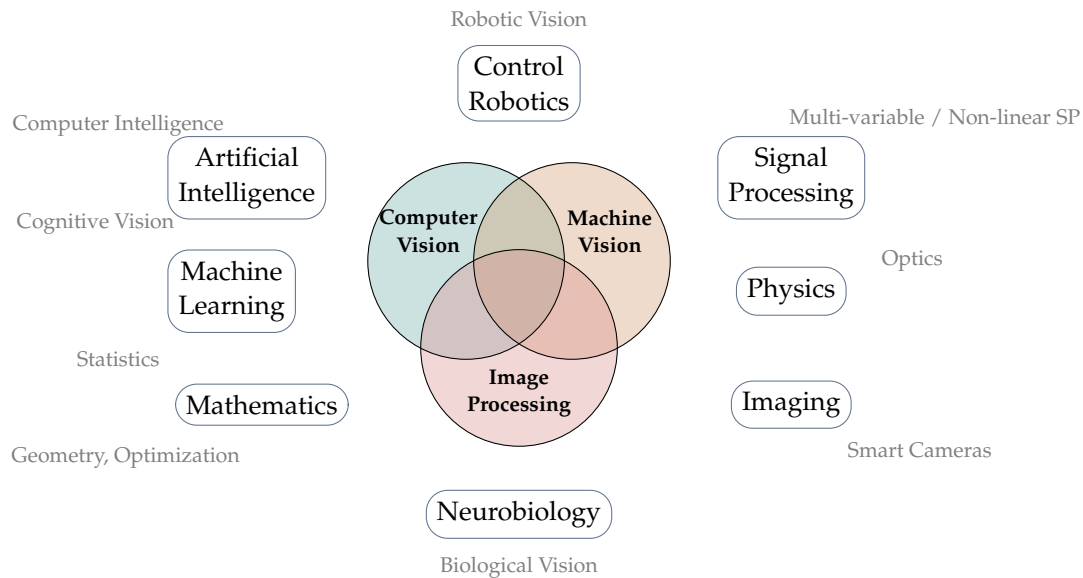


Figure 1.2: Perspectives of study: CV overlaps with machine vision and image processing, and draws on AI/ML, mathematics, robotics, signal processing, physics, imaging and neurobiology.

1.4.1 Evidence from human perception: visual illusions

If even the human visual system — the best vision system we know — can be fooled, the problem is clearly not “just read the pixels”.

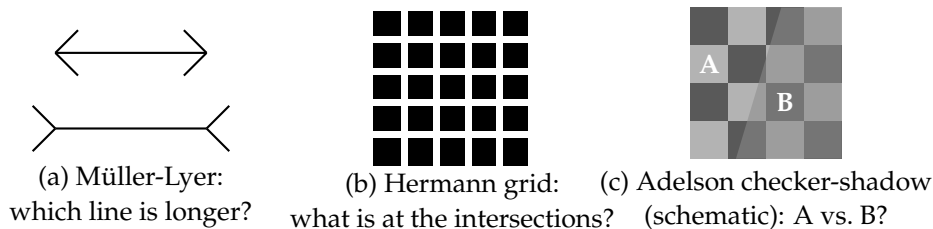


Figure 1.3: Three illusions from the lecture (after Szeliski). (a) Both shafts are *equal*; the arrowheads bias length judgement. (b) Grey “ghost” dots appear at white intersections that are not there (lateral inhibition). (c) In Adelson’s illusion squares A and B have the *same* pixel intensity, but B is “in shadow” so the brain discounts illumination and perceives it as lighter.

(d) Counting red X’s. The slide shows two fields of coloured letters. In one, red X’s are among green X’s/red O’s that differ in *one* feature (colour) — they “pop out”. In the other, each distractor shares one feature with the target (red O’s and green X’s mixed) — a *conjunction search*; you must scan serially, so it is much harder.

Intuition: What the illusions teach us

- The brain does **not** measure raw intensity; it infers *surface properties* (reflectance) by discounting illumination (Adelson). A vision algorithm that only looks at pixel values will “see” A and B as equal — which is physically right but *perceptually* wrong!
- Perception uses **context** and **priors** (Müller-Lyer: arrowheads look like 3D corners).
- Some features are processed **in parallel** (colour pop-out), others **serially** (conjunctions). Attention matters — a theme that returns with attention/transformers.

1.4.2 Reason 1: Vision is an inverse problem

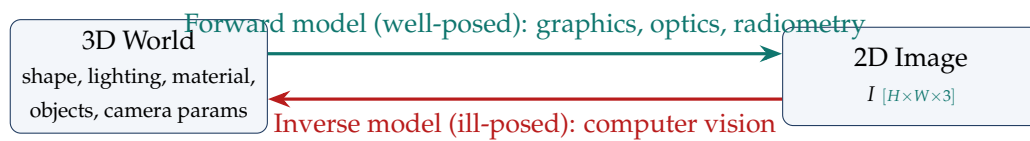


Figure 1.4: Graphics/physics run the forward model; vision must invert it from incomplete, noisy data.

- Most practical CV use-cases are **inverse model** applications.
- We usually have *no knowledge* of how the image was taken or of camera parameters, yet must model the real world in which it was taken (shape, lighting, colour, objects, interactions).
- $\Rightarrow$ we must *almost always model from incomplete/partial, noisy information*.
- **Forward models** are used in physics (radiometry, optics, sensor design) and in computer graphics.

Derivation: Why the inverse is ill-posed — a counting argument

A pinhole projection maps a 3D point (X, Y, Z) to $(x, y) = (fX/Z, fY/Z)$. Every point on the ray $\{(\lambda X, \lambda Y, \lambda Z) : \lambda > 0\}$ projects to the *same* pixel:

$$\left(\frac{f\lambda X}{\lambda Z}, \frac{f\lambda Y}{\lambda Z}\right) = \left(\frac{fX}{Z}, \frac{fY}{Z}\right).$$

So a map $\mathbb{R}^3 \rightarrow \mathbb{R}^2$ loses one dimension (depth) per point: infinitely many scenes explain one image. Additionally, observed brightness $\approx$ *illumination* $\times$ *reflectance*; one number, two unknowns. Hadamard's conditions for a well-posed problem (existence, uniqueness, stability) fail on uniqueness — we need **priors**, which deep learning learns from data.

1.4.3 Reason 2: High-dimensional data

High-dimensional data $\Rightarrow$ heavy computational requirements. A single 1920×1080 RGB frame is $1920 \cdot 1080 \cdot 3 \approx 6.2 \times 10^6$ numbers; a 30 fps video produces $\approx 1.9 \times 10^8$ numbers per second.

1.4.4 Reason 3: Computer vision is “AI-complete”

Truly understanding an image (“what is wrong with this picture?”) can require arbitrary world knowledge, language, physics and common sense — solving vision fully is as hard as solving AI.

1.4.5 Reason 4: No complete model of the human visual system

- Existing models relate largely to *subsystems*, not the whole.
- What is *perceived* vs. what is *cognised*? When is an *object* important for a task, and when is the *context* important?

1.4.6 Reason 5: Verifiability of mathematical/physical models is non-trivial

- How should similarity/dissimilarity between representations be defined? Is it a *distance metric*? Do all images follow such a metric? (Two photos of the same dog can be far apart in pixel ℓ_2 distance, while a dog and a cat on the same lawn can be close.)

- How would a manipulation (*counterfactual*) in a given (noisy) environment behave with respect to the captured image/video? Can a physical model capture this?

Exam Tip

“Why is CV hard?” — list: (1) inverse/ill-posed problem from partial noisy data, (2) high-dimensional data, (3) AI-complete, (4) no complete model of human vision, (5) hard-to-verify models and similarity metrics. Illusions show perception $\neq$ measurement.

1.5 This Course: Topics, Structure, Objectives

1.5.1 The three pillars of computer vision

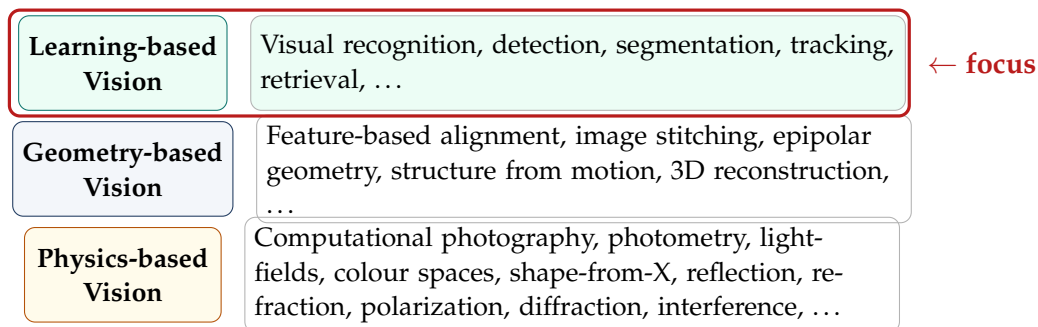


Figure 1.5: The course focuses on **learning-based vision**. For geometry-based vision see Hartley & Zisserman, *Multiple View Geometry*; for physics-based vision see *Physics-Based Vision: Principles and Practice*.

1.5.2 Course segments

Segment 1: Basics & History	Image formation, linear filtering; edges, blobs, features; visual descriptors
Segment 2: Feed-forward & CNNs	Basics of neural networks; CNNs; CNN architectures, variants and visualising CNNs; CNNs for detection & segmentation
Segment 3: RNNs, Attention, Transformers	RNNs; video understanding with CNNs+RNNs; attention in vision models; Transformers & Vision Transformers; ViTs for detection & segmentation
Segment 4: Deep Generative Models	GANs; Variational Autoencoders; Diffusion models
Segment 5: Recent & Advanced	Self-supervised / contrastive learning; vision-language models; foundation models for vision; few-/zero-shot learning; robustness, model compression, neural architecture search; recent VLMs

1.5.3 Eligibility / prerequisites

Theory: a basic ML course; (strongly recommended) a DL course or exposure to neural networks; basics of probability, linear algebra and calculus. **Programming:** Python; a DL framework (PyTorch or TensorFlow) highly recommended.

1.6 Resources and References

- **Traditional CV:** Szeliski, *Computer Vision: Algorithms and Applications*; Forsyth & Ponce, *Computer Vision: A Modern Approach*; and a third classical text shown on the slide.
- **Deep learning:** Goodfellow, Bengio & Courville, *Deep Learning*; Michael Nielsen, *Neural Networks and Deep Learning* (short online book); plus one more DL text shown on the slide.
- **Other topics:** geometry — *Multiple View Geometry in Computer Vision* (+ NPTEL course, weeks 2–5); physics — *Physics-Based Vision: Principles and Practice*.

Homework: go through all links on the “Applications of Computer Vision” slide.

Module Summary: §1.1

CV = making machines interpret images like humans. Applications are everywhere. It is hard because it inverts an information-losing forward process from noisy, high-dimensional data, may need full world knowledge (AI-complete), and lacks complete models of human vision and good similarity metrics. The course focuses on learning-based vision across five segments.

History of Computer Vision

Pitfall / Careful: Scope of this history

The lecture's history is compiled from multiple sources (Szeliski's book and others). It is *slightly biased* towards topics relevant to this course (learning-based recognition), and towards images rather than video. Geometry-based vision, physics-based vision, image/video compression, graphics and computational photography have rich histories that are not covered here.

The lecture splits history into four eras:



Figure 2.1: The four eras of computer-vision history as structured in the lecture.

2.1 Early History: Initial Forays (1959–1982)

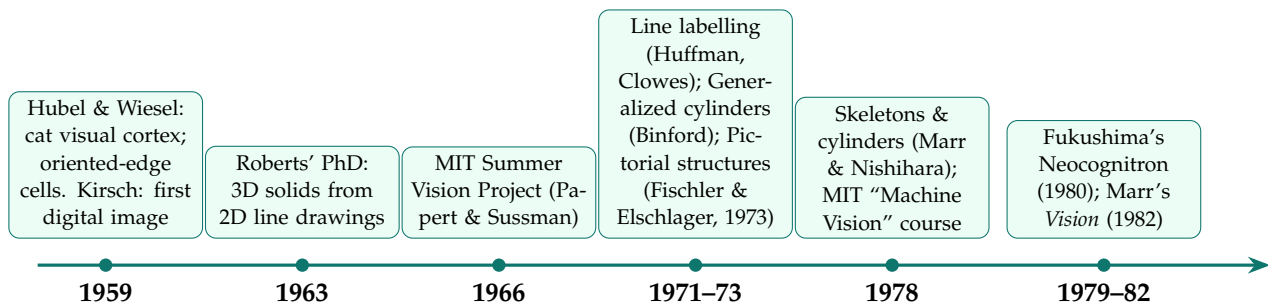


Figure 2.2: Timeline of the early era.

2.1.1 1959 — Hubel & Wiesel: receptive fields in the cat's visual cortex

- Published "Receptive fields of single neurons in the cat's striate cortex".
- Placed electrodes into the primary visual cortex of an anaesthetised cat.
- Showed that **simple** and **complex** neurons exist, and that visual processing **starts with simple structures such as oriented edges**.

(They later received the 1981 Nobel Prize in Physiology or Medicine for this line of work.)

2.1.2 1963 — The first digital image; Roberts' thesis

Russell Kirsch and colleagues built an apparatus to transform images into *number grids*. The first digital image was of Kirsch's infant son: a grainy 5 cm × 5 cm photo of $176 \times 176 = 30,976$ pixels

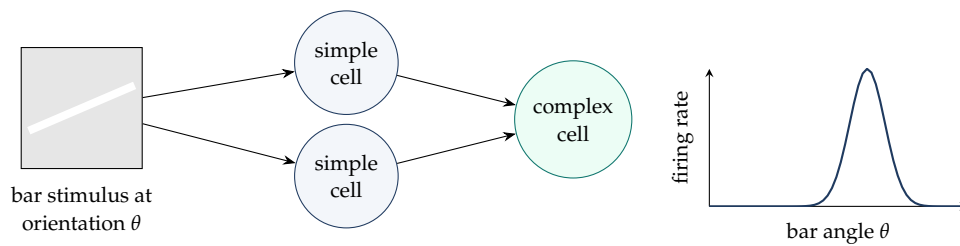


Figure 2.3: Simple cells respond to an oriented edge at a specific position (tuning curve on the right); complex cells pool simple cells and respond to the orientation over a range of positions. This is the biological blueprint for *convolution + pooling*.

(now in the Portland Art Museum). (The scan was made in 1957; the lecture groups it with the early-1960s milestones.)

Lawrence Roberts' PhD thesis (MIT, 1963), *Machine Perception of Three-Dimensional Solids*:

- Extracting 3D information about solid objects from 2D photographs of line drawings ("blocks world").
- Discussed camera transformations, perspective effects, and the rules and assumptions of depth perception.

Roberts is often called a father of computer vision (and later, of the ARPANET).

2.1.3 1966 — The MIT Summer Vision Project

Seymour Papert (with Gerald Sussman) launched the Summer Vision Project: build, in one summer, a platform to automatically **segment background/foreground** and **extract non-overlapping objects** from real-world images. It famously did not succeed — a lesson in how badly the difficulty of vision was underestimated.

2.1.4 1971–73 — Line labelling, shape, and pictorial structures

- **Line labelling:** discern a shape in a line drawing by labelling lines as *convex* (+), *concave* (−) and *occluding* (→) (Huffman, "Impossible objects as nonsense sentences", 1971; Clowes, "On seeing things", 1971).
- **Pictorial Structures** (Fischler & Elschlager, 1973): given a visual object's description, find it in a photograph. Part of the solution is (i) a *descriptive scheme* (parts + spring-like connections) and (ii) a *metric* for the "goodness" of a match.
- **Object recognition via shape:** Binford (1971) *generalised cylinders*; Marr & Nishihara (1978) *skeletons and cylinders*. MIT's AI Lab offered a "Machine Vision" course.

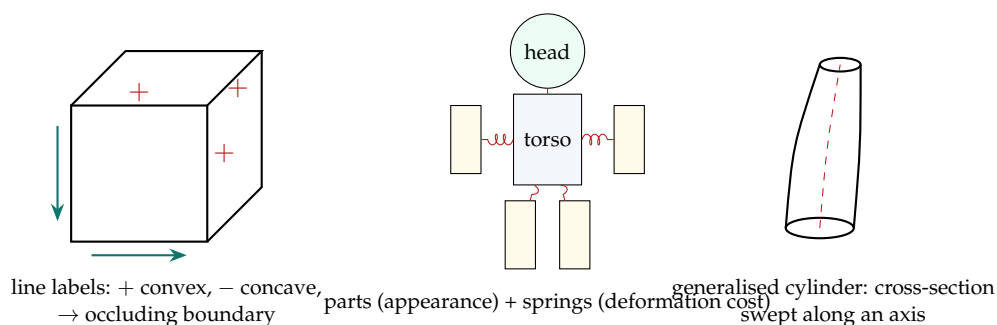


Figure 2.4: Three early "shape" ideas: line labelling, pictorial structures, generalised cylinders.

Key Idea: Pictorial structures as an energy

Although the slides do not write it, the classical formulation (revived by Felzenszwalb & Huttenlocher, 2005) is

$$L^* = \arg \min_{L=(l_1, \dots, l_n)} \left(\sum_{i=1}^n m_i(l_i) + \sum_{(i,j) \in E} d_{ij}(l_i, l_j) \right),$$

where l_i is the location of part i , $m_i(l_i)$ is the *mismatch cost* of placing part i at l_i (appearance), and d_{ij} is the *deformation cost* (a spring) between connected parts. This is exactly “a descriptive scheme + a metric of goodness” from the slide.

2.1.5 1982 — David Marr, Vision

Marr’s book *Vision: A computational investigation into the human representation and processing of visual information*:

- Established that **vision is hierarchical**.
- Introduced a framework where **low-level** algorithms detecting edges, curves, corners etc. feed **high-level** understanding.

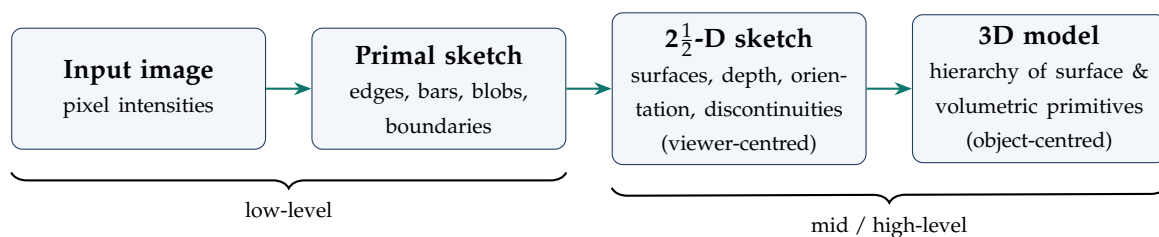


Figure 2.5: Marr’s representational framework.

Intuition: Why Marr still matters for deep learning

A CNN’s early layers learn edge/blob detectors (primal sketch), middle layers learn parts and surface-like features, and deep layers encode object-level abstractions — Marr’s hierarchy, but *learned end-to-end* rather than hand-designed. The “2½-D” name means: depth is represented, but only for surfaces visible from the current viewpoint.

2.1.6 1980 — Fukushima’s Neocognitron: the original ConvNet

- A self-organising artificial network of **simple (S) and complex (C) cells** to recognise patterns, *unaffected by position shifts*.
- Included convolutional layers with weight vectors — i.e. **filters**.

2.2 Towards Algorithms and Practice: Low-level Understanding (1981–1989)

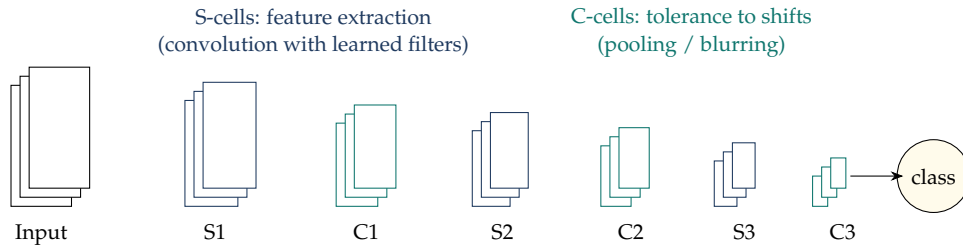


Figure 2.6: Neocognitron: alternating S (convolution-like) and C (pooling-like) layers — the template for LeNet and every modern CNN.

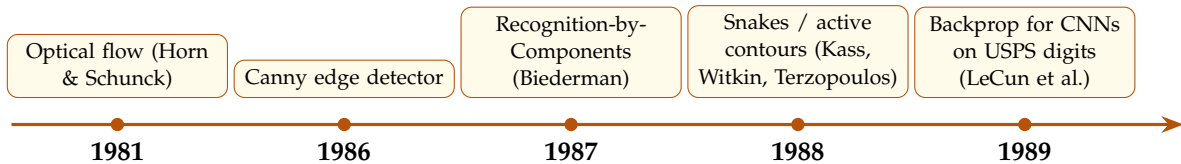


Figure 2.7: The low-level understanding era.

2.2.1 1981 — Optical flow (Horn & Schunck)

Estimate the direction and speed of motion of every pixel between two images. Flow is formulated as a **global energy functional** which is minimised:

$$E(u, v) = \iint \left[\underbrace{(I_x u + I_y v + I_t)^2}_{\text{brightness constancy}} + \alpha^2 \underbrace{(\|\nabla u\|^2 + \|\nabla v\|^2)}_{\text{smoothness}} \right] dx dy,$$

where (u, v) is the flow field and I_x, I_y, I_t are image derivatives.

Derivation: Where the data term comes from

Brightness constancy says a moving point keeps its intensity: $I(x + u \delta t, y + v \delta t, t + \delta t) = I(x, y, t)$. First-order Taylor expansion gives $I + I_x u \delta t + I_y v \delta t + I_t \delta t \approx I$, i.e. $I_x u + I_y v + I_t = 0$. One equation, two unknowns (u, v) per pixel (the *aperture problem*) — so Horn & Schunck add the smoothness term, a *prior*, to make the problem well-posed.

2.2.2 1986 — Canny edge detector

A multi-stage edge detection operator with a **computational theory of edge detection**. Canny used **calculus of variations** to find the function that optimises a given functional (good detection, good localisation, single response per edge). Well-defined and simple to implement, it became the most popular edge detector. Stages: Gaussian smoothing → gradient magnitude/direction → non-maximum suppression → hysteresis thresholding.

2.2.3 1987 — Recognition-by-Components (Biederman)

A **bottom-up** theory of object recognition: objects are assembled from component parts called **geons** — basic 3D shapes (cylinders, cones, blocks, wedges, ...).

2.2.4 1988 — Snakes (active contour models)

Delineate an object outline from a possibly noisy 2D image by evolving a deformable curve $\mathbf{v}(s)$ that minimises $E = \int \left(\underbrace{\alpha |\mathbf{v}'|^2 + \beta |\mathbf{v}''|^2}_{\text{internal (elastic/stiff)}} + \underbrace{E_{\text{image}}(\mathbf{v})}_{\text{attracted to edges}} \right) ds$. Used for tracking, shape recognition,

segmentation, edge detection and stereo matching.

2.2.5 1989 — Backprop for CNNs arrives

LeCun et al., “Backpropagation applied to handwritten zip code recognition” — trained a convolutional network end-to-end with backpropagation on USPS handwritten digits. The Neocognitron’s architecture + gradient-based learning = the modern CNN recipe.

2.2.6 Other low-level milestones of the 1980s

Idea	Reference	One-line meaning
Image pyramids	Burt & Adelson, 1983	multi-resolution stack (Laplacian pyramid as compact code)
Scale-space processing	Witkin, 1984	analyse signal at a continuum of Gaussian blur scales
Wavelets	Mallat, 1989	multiresolution decomposition, localised in space <i>and</i> frequency
Shape-from-X	Pentland 1984; Blake et al. 1985	recover shape from shading, texture, focus, ...
Variational optimisation	Poggio et al., 1985	vision problems as regularised energy minimisation
Markov Random Fields	Geman & Geman, 1984/85	probabilistic priors over neighbouring pixels (Gibbs sampling)

Intuition: The unifying theme of the 1980s

Almost every method above is “*data term + smoothness/prior term, then minimise*” (optical flow, snakes, regularisation, MRFs). This is the same idea as “*loss + regulariser*” in modern deep learning — the difference is that deep nets *learn* the prior.

2.3 Towards Algorithms and Practice: Next-level Understanding (1991–2009)

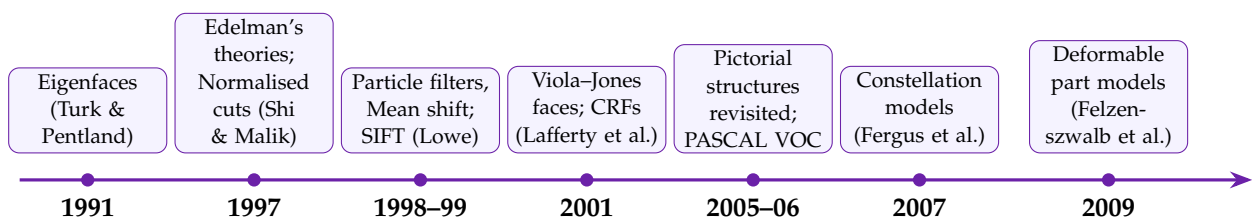


Figure 2.8: The next-level (“feature + classifier”) era.

Eigenfaces (Turk & Pentland, 1991)

Face recognition via PCA. Flatten each face image to a vector $\mathbf{x} \in \mathbb{R}^d$ ($d = H \cdot W$); compute the mean $\boldsymbol{\mu}$ and top- K eigenvectors of the covariance $\boldsymbol{\Sigma} [d \times d]$; represent a face by its projection coefficients $\mathbf{w} = U_K^\top (\mathbf{x} - \boldsymbol{\mu}) [K]$ and recognise by nearest neighbour in this low-dim space. (Builds on Sirovich & Kirby, 1987.)

Computational theories of object recognition (Edelman, 1997)

Survey/theory of how recognition could be computed.

Perceptual grouping, Normalised Cuts (Shi & Malik, 1997/2000)

Segmentation as graph partitioning: minimise $\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$, solved approximately with a generalised eigenvalue problem.

Particle filters (Liu & Chen, 1998) & Mean shift (Cheng, 1995/98) for tracking

Sequential Monte Carlo tracking of multi-modal posteriors; mean shift = mode seeking by iteratively moving to the weighted mean of a window.

SIFT (Lowe, 1999, 2004)

Scale-Invariant Feature Transform: keypoints at scale-space extrema of DoG, with 128-D gradient-orientation histograms — invariant to scale and rotation, robust to illumination.

Viola–Jones face detection (2001)

Real-time detection using Haar-like features, the *integral image*, AdaBoost feature selection and an attentional *cascade*.

Conditional Random Fields (Lafferty et al., 2001)

Discriminative probabilistic models for labelling structured data; widely used for segmentation.

Pictorial structures revisited (Felzenszwalb & Huttenlocher, 2005)

Efficient matching via distance transforms.

PASCAL VOC (2005–)

Benchmark for object detection/classification (20 classes); scene, panorama and location recognition methods grow.

Constellation models (Fergus, Perona & Zisserman, 2007)

Objects as probabilistic constellations of parts; weakly supervised, scale-invariant learning.

Deformable Part Models (Felzenszwalb et al., 2009)

Discriminatively trained part-based models (HOG + latent SVM) — state of the art in detection just before deep learning.

2.4 The Deep Learning Era (2010–)

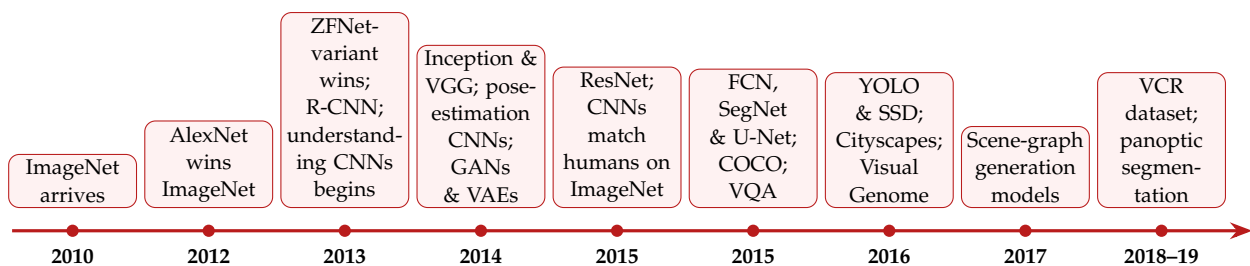


Figure 2.9: Deep learning era (as on the slides; the right-hand column of the slide is the “2015–2019” continuation).

- **2010:** ImageNet arrives (over a million labelled images, 1000 classes in the ILSVRC challenge).
- **2012:** AlexNet (Krizhevsky, Sutskever, Hinton) wins ImageNet by a huge margin — GPUs + ReLU + dropout + big data.
- **2013:** A CNN (variant of ZFNet) wins; R-CNN for object detection; *understanding/visualising CNNs* begins.
- **2014:** Inception (GoogLeNet) and VGG; human pose-estimation CNNs (DeepPose); deep generative models — GANs, VAEs.

- **2015: ResNet** (residual connections, 152 layers); CNNs match human-level performance on ImageNet.
- **2015–16: FCN, SegNet, U-Net** for semantic segmentation; **COCO** and **VQA** datasets; **YOLO** and **SSD** for real-time detection; **Cityscapes**, **Visual Genome** datasets.
- **2017–19:** scene-graph generation models; **VCR** (visual commonsense reasoning) dataset; **panoptic segmentation**, ...

Intuition: What changed in 2012?

The Neocognitron (1980) and LeCun's CNN (1989) already had the architecture. What arrived by 2012 was (1) **data** (ImageNet), (2) **compute** (GPUs), (3) **training tricks** (ReLU, dropout, better init). The *feature engineering* of the previous era (SIFT, HOG, Haar) was replaced by *feature learning*.

2.5 History of Applications

1970s	Optical Character Recognition (OCR)
1980s	Machine vision, smart cameras
1990s	Machine vision in manufacturing, biometrics, medical imaging, recording devices, video surveillance
2000s	More biometrics, better medical imaging, object/face detection, autonomous navigation, Google Goggles, vision on social media
2010s	Everywhere around us

Readings: Szeliski Ch. 1 (§1.1–1.2); Rostyslav Demush, *A Brief History of Computer Vision (and Convolutional Neural Networks)*.

Module Summary: §1.2

Neuroscience (Hubel & Wiesel) → first digital images and blocks-world geometry (Roberts) → Marr's hierarchical framework and the Neocognitron (1980s) → variational low-level vision (flow, Canny, snakes, MRFs) and CNN backprop (1989) → hand-crafted features + classifiers (Eigenfaces, SIFT, Viola-Jones, DPM) and benchmarks (PASCAL) → ImageNet + AlexNet (2012) and the deep era (R-CNN, VGG, Inception, ResNet, GAN/VAE, FCN/U-Net, YOLO/SSD, COCO, VQA, panoptic segmentation).

Image Formation

3.1 Image Formation: An Overview

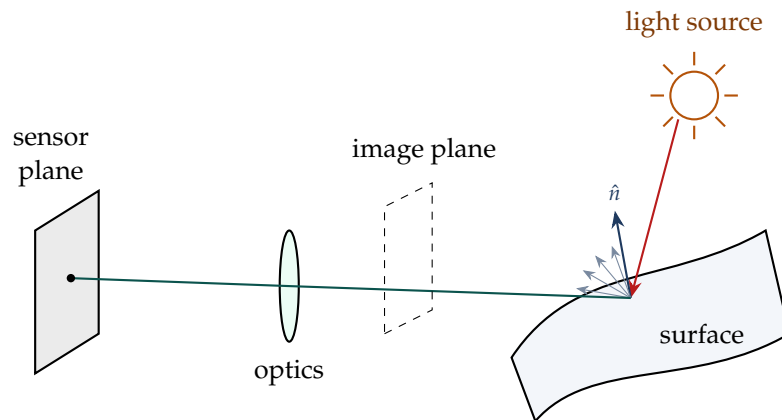


Figure 3.1: Image formation (after Szeliski Fig. 2.1): light leaves a source, reflects off a surface (with normal $\hat{n}$), passes through the optics and is recorded on the sensor plane.

The value recorded at a pixel depends on four groups of factors:

1. **Light source** strength and direction.
2. **Surface** geometry, material and nearby surfaces (inter-reflections).
3. **Sensor** capture properties.
4. **Image representation** and colour.

Related topic	Sub-topics
Geometry	2D transformations, 3D transformations, camera calibration, distortion
Photometry	Lighting, reflectance & shading, optics
Colour	Physics of colour, human colour, colour representation
Sensor	Human perception, camera design, sampling & aliasing, compression

The lecture covers a few relevant topics from these. For a detailed treatment: Forsyth & Ponce, Chapters 1–5.

3.2 Photometry: Models of Reflection

When light hits a surface, three things can happen:

- **Absorption:** a fraction $1 - \rho$ is absorbed, where $\rho \in [0, 1]$ is the **albedo** (fraction reflected). Low albedo $\Rightarrow$ more absorption (dark materials).
- **Diffuse reflection:** light scattered equally in all directions, *independent of viewing direction*. E.g. brick, cloth, rough wood. **Lambert's cosine law:** amount of reflected light $\propto \cos \theta$, θ being the angle between the surface normal and the light direction.
- **Specular reflection:** mirror-like, *depends on viewing direction*. E.g. mirror.

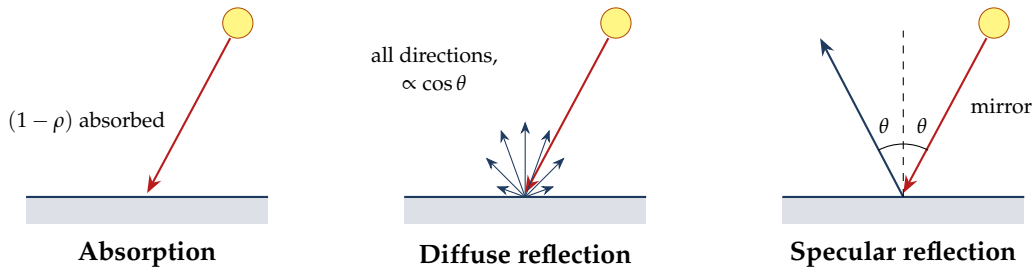


Figure 3.2: Absorption, diffuse (Lambertian) reflection and specular reflection.

Derivation: Lambert's cosine law

A light beam of cross-sectional area A carrying power P hits a surface patch tilted by θ from the beam. The beam spreads over surface area $A / \cos \theta$ (the patch is “stretched”). Irradiance (power per unit surface area) is therefore

$$E = \frac{P}{A / \cos \theta} = \frac{P}{A} \cos \theta = E_0 \cos \theta = E_0 (\hat{\mathbf{n}} \cdot \hat{\mathbf{s}}),$$

with $\hat{\mathbf{n}}$ the unit normal and $\hat{\mathbf{s}}$ the unit direction to the source. A Lambertian surface re-emits a fraction ρ uniformly, so the observed intensity is

$$I = \rho E_0 \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{s}})$$

which does not depend on the viewer direction $\hat{\mathbf{v}}$. (The max handles surfaces facing away.)

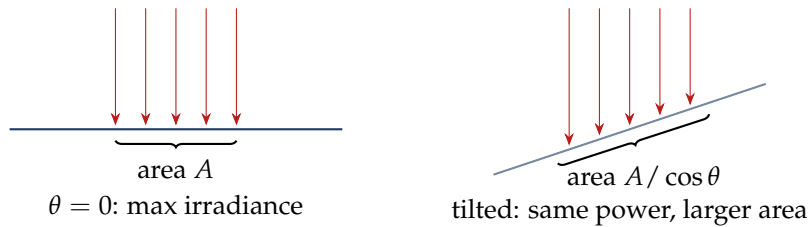


Figure 3.3: Geometric origin of Lambert's cosine law.

More about reflection.

- Most surfaces have *both* specular and diffuse components (e.g. glossy plastic: diffuse body colour + white highlight).
- Intensity depends on illumination angle because *less light comes in at oblique angles* (Lambert).
- Other effects: transparency, refraction, subsurface scattering (skin, marble, milk), fluorescence, phosphorescence.
- **BRDF** — *Bidirectional Reflectance Distribution Function*: a model of local reflection that tells how bright a surface appears when viewed from one direction when light falls on it from another:

$$f_r(\hat{\mathbf{v}}_i, \hat{\mathbf{v}}_r; \lambda) = \frac{dL_r(\hat{\mathbf{v}}_r; \lambda)}{dE_i(\hat{\mathbf{v}}_i; \lambda)} \quad [\text{sr}^{-1}],$$

the ratio of reflected radiance to incident irradiance. For a Lambertian surface $f_r = \rho / \pi$ (constant).

Key Idea: Diffuse + specular: the Phong-style model (beyond the slides)

A common approximation of the total observed intensity is $I = \underbrace{k_a}_{\text{ambient}} + \underbrace{k_d (\hat{\mathbf{n}} \cdot \hat{\mathbf{s}})}_{\text{diffuse}} + \underbrace{k_s (\hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^\alpha}_{\text{specular}}$,
 where $\hat{\mathbf{r}} = 2(\hat{\mathbf{n}} \cdot \hat{\mathbf{s}})\hat{\mathbf{n}} - \hat{\mathbf{s}}$ is the mirror direction and α controls highlight sharpness. Only the specular term depends on the viewer $\hat{\mathbf{v}}$.

3.3 Colour

- Light is composed of a **spectrum of wavelengths**; visible light is roughly 380–750 nm.
- Coloured light arriving at the sensor involves (i) the colour (spectrum) of the light source and (ii) the colour (spectral reflectance) of the surface.

$$C_{\text{sensor}}(\lambda) = L(\lambda) \rho(\lambda), \quad \text{response of channel } k: R_k = \int L(\lambda) \rho(\lambda) S_k(\lambda) d\lambda,$$

where $S_k(\lambda)$ is the spectral sensitivity of sensor channel $k \in \{R, G, B\}$.

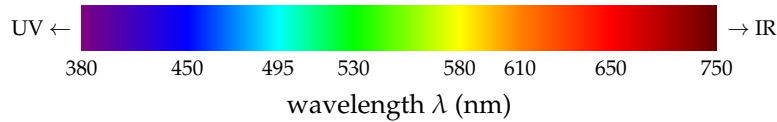


Figure 3.4: The visible band of the electromagnetic spectrum (V-I-B-G-Y-O-R from short to long wavelength).

3.3.1 The Bayer grid / filter

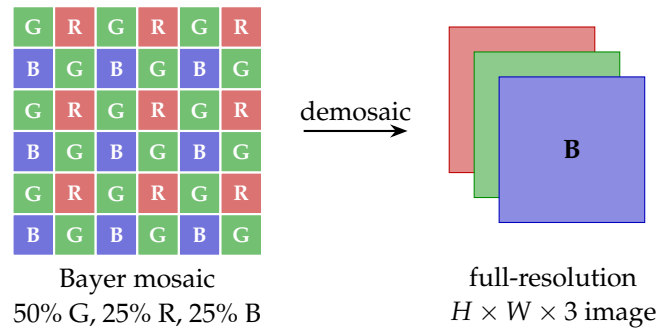


Figure 3.5: Bayer colour filter array. Each photosite measures only one colour; demosaicing interpolates the other two.

- A Bayer arrangement of colour filters on a camera sensor: **50% green, 25% red, 25% blue**.
- Why twice as much green? The human eye is most sensitive in the green (medium wavelength) region, and luminance is dominated by green.
- To obtain a full-colour image, **demosaicing** algorithms use surrounding pixels to estimate missing values.

Derivation: Simplest demosaicing: bilinear

At a red photosite (i, j) , green is missing. Its four edge-neighbours are all green, so $\hat{G}(i, j) = \frac{1}{4} [G(i-1, j) + G(i+1, j) + G(i, j-1) + G(i, j+1)]$. Blue at the same red site: its four *diagonal*

neighbours are blue, so $\hat{B}(i, j) = \frac{1}{4}[B(i-1, j-1) + B(i-1, j+1) + B(i+1, j-1) + B(i+1, j+1)]$. This is a *local linear filter* — a preview of Module 5.

Exam Tip: Question posed on the slide

If the visible spectrum is VIBGYOR, why RGB? — Answered in Module 4: because the human retina has **three** types of cones (L, M, S $\approx$ red, green, blue), so three numbers suffice to reproduce most perceived colours.

3.4 Digital Image Sensing

3.4.1 The image sensing pipeline

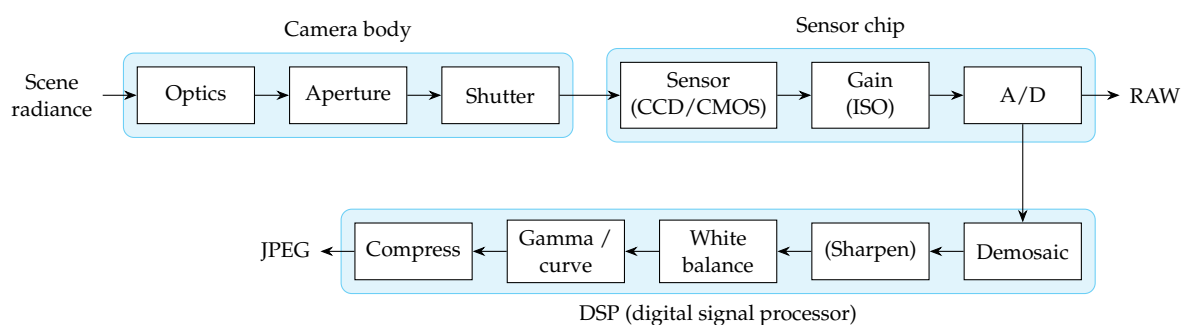


Figure 3.6: Image sensing pipeline (after Szeliski Fig. 2.23). RAW files tap the signal right after A/D conversion; JPEGs go through the full DSP chain.

Intuition: Why deep-learning people should care

Every stage is a *transformation of the data distribution*: white balance shifts colours, gamma is a non-linear point operation, JPEG adds block artefacts. A model trained on phone JPEGs may fail on RAW medical or satellite imagery — this is one source of *domain shift*.

Key Idea: Gamma (from the pipeline)

Sensors respond roughly linearly to light, but displays and human perception are non-linear. Cameras apply $Y' = Y^{1/\gamma}$ with $\gamma \approx 2.2$, which allocates more code values to dark tones (where the eye is more sensitive). It is a *point operation* (Module 4).

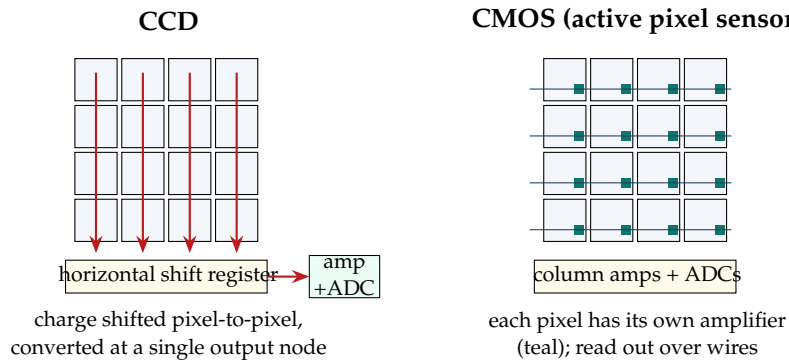


Figure 3.7: CCD moves charge across the chip to one output; CMOS converts charge to voltage inside each pixel.

3.4.2 CCD vs. CMOS

	CCD (Charge-Coupled Device)	CMOS (Complementary Metal-Oxide Semiconductor)
Charge → voltage	At a single output node after moving charge pixel to pixel	Inside each pixel element
Transport	Bucket-brigade charge transfer	Several transistors per pixel amplify and move signal over conventional wires
Digitisation	Separate ADC turns each pixel's value into a digital value	Slide: "CMOS signal is digital, so needs no (external) ADC" — ADCs are on-chip
Typical strengths	Uniformity, low noise (historically)	Low power, speed, cheap, on-chip integration (all phones)

Pitfall / Careful

The slide's phrase "CMOS signal is digital, so it needs no ADC" should be read as "the analog-to-digital conversion is integrated *on the sensor chip*", so the chip outputs digital values and no separate ADC stage is needed. Physically, photodiodes are always analog.

3.4.3 Digital image sensor properties

Shutter speed	Controls the amount of light reaching the sensor (a.k.a. <i>exposure time</i>).
Sampling pitch	Physical spacing between adjacent sensor cells on the imaging chip.
Fill factor	Active sensing area as a fraction of the theoretically available sensing area (product of horizontal and vertical sampling pitches).
Chip size	Size/area of the chip.
Analog gain	Amplification of the sensed signal using automatic gain control (AGC) logic (the <i>ISO</i> setting).
Sensor noise	Noise from various sources in sensing (shot/photon noise, thermal/dark current, read noise, quantisation).
Resolution	How many bits for each pixel, decided by the analog-to-digital conversion module.
Post-processing	Digital enhancement often used before compression and storage.

Derivation: Fill factor

If the sampling pitches are p_x, p_y and the light-sensitive photodiode inside each cell is $a_x \times a_y$, then

$$\text{fill factor} = \frac{a_x a_y}{p_x p_y} \in (0, 1].$$

E.g. pitch $5\ \mu\text{m} \times 5\ \mu\text{m}$ with a $4\ \mu\text{m} \times 3\ \mu\text{m}$ active area gives $12/25 = 48\%$. (Micro-lenses on top of each pixel are used to boost the effective fill factor.)

Derivation: Bit depth and quantisation step

With b bits, there are 2^b levels. Mapping a range $[0, V_{\max}]$ gives step $\Delta = V_{\max}/2^b$. Uniform quantisation error $e \sim \mathcal{U}(-\Delta/2, \Delta/2)$ has variance $\Delta^2/12$ — each extra bit halves Δ and improves SNR by $20 \log_{10} 2 \approx 6.02$ dB.

Intuition: Gain (ISO) is not free light

Raising ISO amplifies *signal and noise alike* (after the photons are collected). It helps use the ADC's range but does not improve the photon-limited SNR. Longer shutter or wider aperture actually collects more photons.

3.4.4 Popular question: with smartphones, do you need DSLRs?

- **DSLR** — Digital Single Lens Reflex: uses a **mirror** to reflect light from the lens to an optical viewfinder, or flips the mirror out of the way to let light fully pass onto the sensor.
- The debate is essentially *mirror vs. mirrorless* cameras.
- **Pros of mirrorless** (incl. phones): accessibility, portability, low cost.
- **Pros of DSLRs**: picture quality (large sensor), versatility and functionality, physical shutter, variable focal length/aperture.

3.5 Sampling and Aliasing

Key Idea: Shannon's sampling theorem

A band-limited signal with maximum frequency $f_{\max}$ can be perfectly reconstructed from samples taken at rate

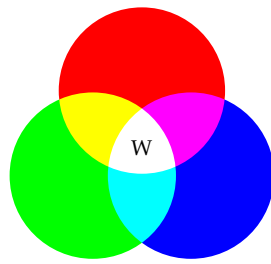
$$f_s \geq 2f_{\max}$$

$2f_{\max}$ is the **Nyquist rate**; the slide calls $f_{\max}$ the Nyquist frequency (the more standard usage is: Nyquist frequency = $f_s/2$, the highest frequency representable at sampling rate f_s). If this is violated — frequencies above the Nyquist frequency are present — **aliasing** happens: high frequencies masquerade as low ones.

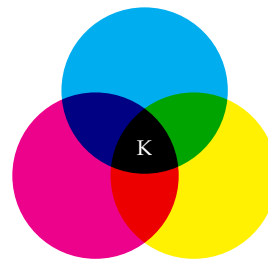
Why is aliasing bad? It creates issues while downsampling and upsampling an image (moiré patterns on a striped shirt, jagged edges). Detailed treatment in Module 7.

3.6 Colour Space Representations

- Popular colour spaces: **RGB**, **CMYK**. **Additive** colours: R, G, B. **Subtractive** colours: C, M, Y.
- Others: XYZ, YUV, L*a*b*, YCbCr, HSV. Standards are set by the CIE (Commission Internationale d'Éclairage).
- Understanding colour spaces is important in the printing industry.



Additive (RGB): light



Subtractive (CMY): ink/pigment

Figure 3.8: Additive primaries R, G, B mix to white; subtractive primaries C, M, Y mix to black (K is added in CMYK printing for true black and ink economy).

Derivation: Key conversions

RGB $\leftrightarrow$ **CMY** (values in $[0, 1]$): $C = 1 - R$, $M = 1 - G$, $Y = 1 - B$. (Cyan ink *subtracts* red light.)

RGB $\rightarrow$ **YCbCr** (ITU-R BT.601, 8-bit full range as used in JPEG):

$$\begin{bmatrix} Y \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}.$$

Y (luma) is a weighted sum dominated by green; $C_b \propto B - Y$ and $C_r \propto R - Y$ are *colour differences*. For a grey pixel $R = G = B = v$: $Y = v$ (rows sum to 1) and $C_b = C_r = 128$ (rows sum to 0) — no chroma, as expected.

HSV: Hue (angle on the colour wheel), Saturation (purity), Value = $\max(R, G, B)$ — designed to match how humans describe colour.

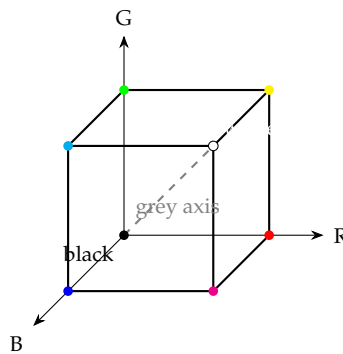


Figure 3.9: The RGB cube. The main diagonal is the grey axis; luma Y roughly measures position along it, chroma measures distance from it.

3.7 Image Compression

- The **last stage** in a camera's processing pipeline.
- Convert the signal into **YCbCr** (or a variant); compress **luminance with higher fidelity than chrominance** (humans are far more sensitive to brightness detail than colour detail $\Rightarrow$ chroma subsampling, e.g. 4:2:0).

- Most common technique: **Discrete Cosine Transform (DCT)**, used in MPEG and JPEG.
- DCT is a variant of the Discrete Fourier Transform — a reasonable approximation of the *eigen-decomposition of image patches* (i.e. close to PCA/KLT for natural images), so energy compacts into few coefficients.
- Videos also use **block-level motion compensation** (predict a block from the previous frame, encode only the residual).
- Quality is measured with **PSNR**.

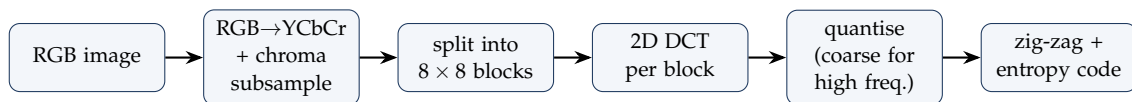


Figure 3.10: JPEG in one line. The lossy step is quantisation.

Derivation: PSNR

For an image I with n pixels and its compressed version $\hat{I}$:

$$\text{MSE} = \frac{1}{n} \sum_{\mathbf{x}} [I(\mathbf{x}) - \hat{I}(\mathbf{x})]^2, \quad \boxed{\text{PSNR} = 10 \log_{10} \frac{I_{\max}^2}{\text{MSE}} \text{ dB}} = 20 \log_{10} \frac{I_{\max}}{\sqrt{\text{MSE}}}.$$

$I_{\max}$ is the maximum possible intensity (255 for 8-bit). It is a *signal-to-noise* ratio where “signal” is the peak power $I_{\max}^2$ and “noise” is the error power (MSE). The log turns ratios into additive decibels.

- $\text{MSE} \rightarrow 0 \Rightarrow \text{PSNR} \rightarrow \infty$ (lossless).
- Halving RMSE increases PSNR by $20 \log_{10} 2 \approx 6.02$ dB.
- Typical lossy 8-bit images: 30–50 dB; below ~ 20 dB looks bad.

Pitfall / Careful

PSNR is pixel-wise; it does not model perception. Two images with the same PSNR can look very different (e.g. a global brightness shift vs. localised blocking artefacts). Perceptual metrics like SSIM/LPIPS exist for this reason.

Readings: Szeliski Ch. 2; (optional, geometric image formation) Forsyth & Ponce Ch. 1–5.

Module Summary: §1.3

Pixel value = $f(\text{light, surface, sensor, representation})$. Surfaces absorb $(1 - \rho)$, reflect diffusely ($\propto \cos \theta$, Lambert) and specularly (view-dependent); BRDF generalises this. Colour = source spectrum $\times$ surface reflectance, sensed through a Bayer mosaic (50/25/25 G/R/B) + demosaicing. Pipeline: optics $\rightarrow$ aperture $\rightarrow$ shutter $\rightarrow$ CCD/CMOS $\rightarrow$ gain $\rightarrow$ ADC $\rightarrow$ demosaic $\rightarrow$ white balance $\rightarrow$ gamma $\rightarrow$ compress. Sampling must obey $f_s \geq 2f_{\max}$. Colour spaces: additive RGB, subtractive CMY(K), YCbCr, HSV, Lab. JPEG: YCbCr + DCT; quality via $\text{PSNR} = 10 \log_{10}(I_{\max}^2 / \text{MSE})$.

Image Representation

4.1 Why RGB if the spectrum is VIBGYOR?

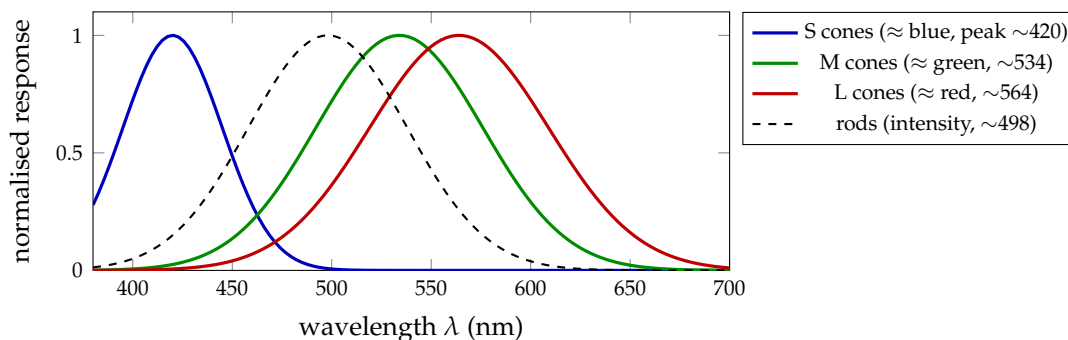


Figure 4.1: Approximate (Gaussian-shaped) spectral sensitivities of the human photoreceptors. Real curves are asymmetric, but the key fact is: **three** cone types.

- The retina has **Long (red), Medium (green) and Short (blue) cones**, plus **rods** for intensity (low-light vision).
- Any spectrum $C(\lambda)$ is perceived only through three numbers L, M, S (each $\int C(\lambda)S_*(\lambda) d\lambda$) — So three primaries (R,G,B) are enough to reproduce most perceived colours — *trichromacy*.
- **Metamers**: physically different spectra producing the same (L, M, S) look identical. RGB displays exploit this.

Fun facts: the genes for the M and L pigments are on the X-chromosome $\Rightarrow$ men (XY) are more likely to be colour blind. Some animals have 1 (night animals), 2 (e.g. dogs), 4 (fish, birds), 5 (pigeons, some reptiles/amphibians), or even 12 (mantis shrimp) cone types.

Derivation: Why three numbers suffice — projection view

Treat a spectrum as a vector $\mathbf{c} \in \mathbb{R}^N$ (sampled at N wavelengths, e.g. $N = 300$). The eye computes

$$\underbrace{\mathbf{r}}_{[3]} = \underbrace{S}_{[3 \times N]} \underbrace{\mathbf{c}}_{[N]}$$

a linear projection onto a 3-D subspace. Everything in the $(N - 3)$ -dimensional null space of S is invisible, which is exactly why metamers exist and why a 3-channel representation is “lossless” for human viewers (but not for, say, hyperspectral analysis).

4.2 Image as a Matrix

- Common to use **one byte per value**: 0 = black, 255 = white (8-bit, $2^8 = 256$ levels).
- One such matrix for every channel in colour images.
- Storage: $H \times W \times C \times \text{bytes}$. A 1024×768 RGB image: $1024 \cdot 768 \cdot 3 = 2,359,296$ bytes ≈ 2.25 MiB uncompressed.

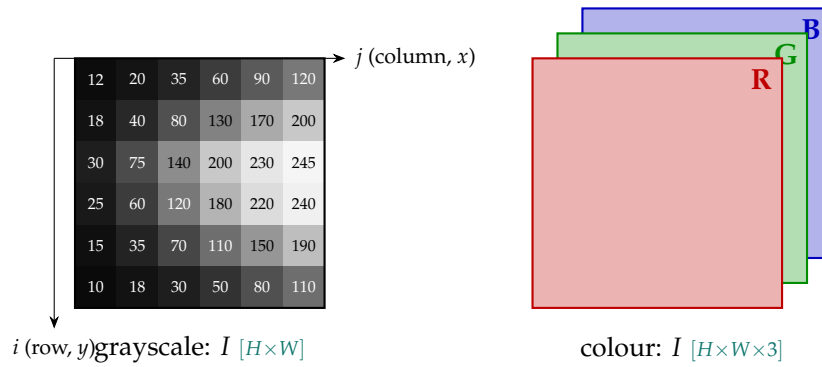


Figure 4.2: A digital image is a matrix of numbers; one matrix per channel for colour images.

Pitfall / Careful: Conventions

Maths writes $I(x, y)$ with x horizontal; arrays index $I[i, j] = [\text{row}, \text{column}] = [y, x]$, with the origin at the *top-left* and y pointing *down*. Libraries differ: PyTorch uses $C \times H \times W$ (channels first); NumPy/OpenCV/PIL use $H \times W \times C$; OpenCV stores **BGR**.

4.3 Image as a Function

We can think of a grayscale image as a function

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}, \quad f(x, y) = \text{intensity at position } (x, y).$$

More realistically $f : [a, b] \times [c, d] \rightarrow [0, I_{\max}]$, and a colour image is a vector-valued function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3, f(x, y) = (r(x, y), g(x, y), b(x, y))$.

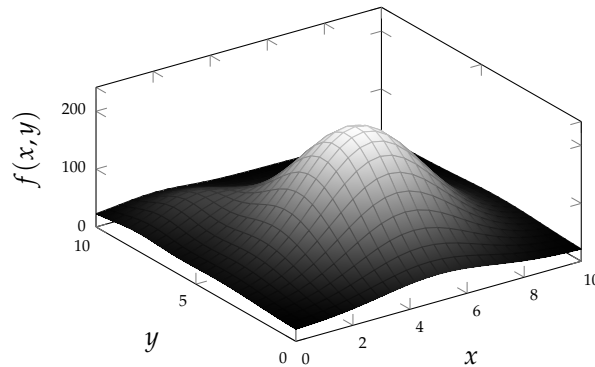


Figure 4.3: A grayscale image viewed as a height map $z = f(x, y)$: bright regions are peaks, edges are steep slopes. Filtering = reshaping this surface.

Key Idea: Digital = sampled + quantised

A digital image is a **discrete** version of f :

- **Sampling** (discretise the domain): $F[i, j] = f(i\Delta, j\Delta)$ on a grid of spacing Δ .
- **Quantisation** (discretise the range): values rounded to one of 2^b levels.

$$F[i, j] = \text{quantize}\{f(i\Delta, j\Delta)\}.$$

Sampling too coarsely causes *aliasing* (Module 7); quantising too coarsely causes *false contours/banding*.

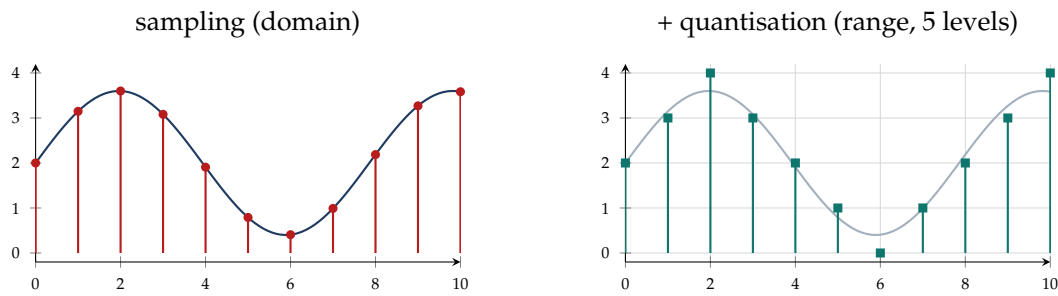


Figure 4.4: From continuous f to digital F : sample the domain, then quantise the values.

4.4 Image Transformations

Because an image is a function, we can transform either its *range* (values) or its *domain* (coordinates):

Transformation	Formula	Effect
Brighten	$\hat{I}(x, y) = I(x, y) + 20$	range change (every pixel +20, clipped at 255)
Horizontal flip	$\hat{I}(x, y) = I(-x, y)$	domain change (mirror about the y -axis)



Figure 4.5: Range vs. domain transformations.

Pitfall / Careful

For discrete images $I(-x, y)$ is implemented as $\hat{I}[i, j] = I[i, W - 1 - j]$ (indices must stay in $[0, W - 1]$).

4.5 Image Processing Operations: Point, Local, Global

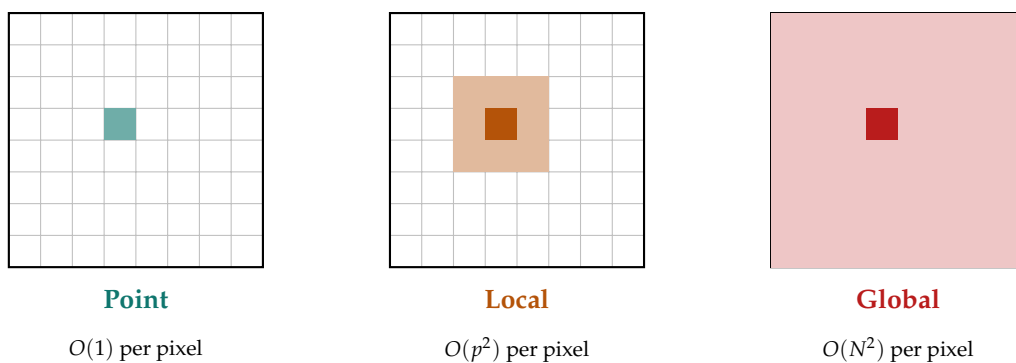


Figure 4.6: Which input pixels (shaded) influence the output at the dark pixel (m_0, n_0) .

Type	Output at (m_0, n_0) depends on	Cost / pixel	Cost / $N \times N$ image
Point	only the input value at the same coordinate	$O(1)$	$O(N^2)$
Local	input values in a $p \times p$ neighbourhood of that coordinate	$O(p^2)$	$O(N^2 p^2)$
Global	all values in the $N \times N$ input image	$O(N^2)$	$O(N^4)$ (naively)

4.6 Point Operations

A point operation is $\hat{I}(m, n) = T(I(m, n))$ for a function $T : [I_{\min}, I_{\max}] \rightarrow [I_{\min}, I_{\max}]$, applied identically at every pixel.

4.6.1 Example 1: Reversing the contrast (negative)

$$\hat{I}(m_0, n_0) = I_{\max} - I(m_0, n_0) + I_{\min}$$

Derivation: Why this formula

We want a *decreasing linear* map $T(v) = av + b$ with $T(I_{\min}) = I_{\max}$ and $T(I_{\max}) = I_{\min}$. Subtracting: $a(I_{\max} - I_{\min}) = I_{\min} - I_{\max} \Rightarrow a = -1$. Then $b = I_{\max} + I_{\min}$. So $T(v) = I_{\max} + I_{\min} - v$. For 8-bit, $I_{\min} = 0$, $I_{\max} = 255$: $T(v) = 255 - v$ (a photographic negative).

4.6.2 Example 2: Linear contrast stretching

$$\hat{I}(m_0, n_0) = (I(m_0, n_0) - \min_{x,y} I(x, y)) \cdot \frac{I_{\max} - I_{\min}}{\max_{x,y} I(x, y) - \min_{x,y} I(x, y)} + I_{\min}$$

Derivation: Stretching as a linear map between two intervals

Let the image occupy $[a, b] = [\min I, \max I]$ and let the display range be $[I_{\min}, I_{\max}]$. The unique increasing affine map sending $a \mapsto I_{\min}$ and $b \mapsto I_{\max}$ is: normalise to $[0, 1]$ via $\frac{v-a}{b-a}$, then scale and shift:

$$T(v) = \frac{v-a}{b-a} (I_{\max} - I_{\min}) + I_{\min}.$$

E.g. an image in $[80, 160]$ stretched to $[0, 255]$: $T(v) = (v - 80) \cdot \frac{255}{80}$, so $T(80) = 0$, $T(120) = 127.5 \rightarrow 128$, $T(160) = 255$. Slope $255/80 \approx 3.19 > 1 \Rightarrow$ differences are amplified (more contrast).

4.6.3 Going beyond linear stretching: Histogram Equalisation (homework — derived fully)

The **histogram** $h(l)$ counts how many pixels have intensity l . Linear stretching only uses the extremes $\min I, \max I$; a single outlier pixel can make it useless. **Histogram equalisation** instead chooses a (non-linear) T so that the output histogram is as *flat (uniform)* as possible — every intensity level used equally, maximising contrast globally.

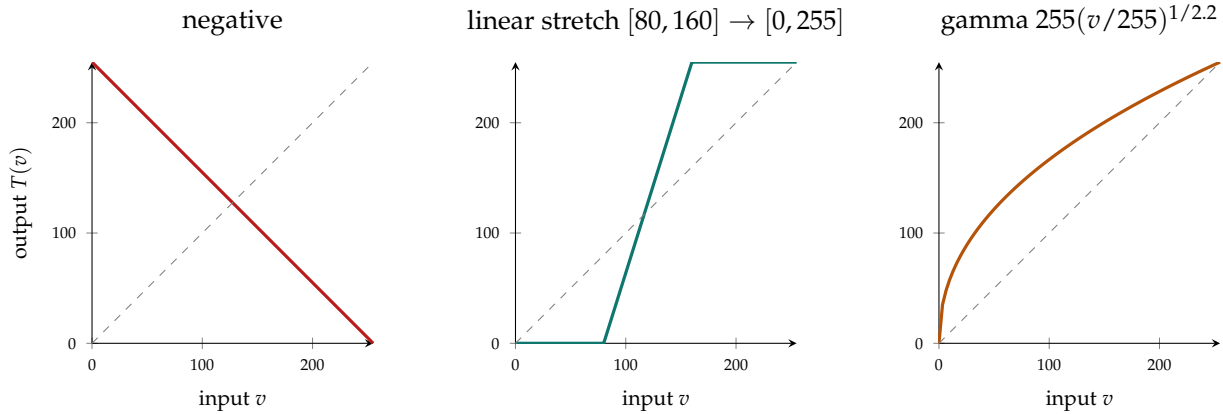


Figure 4.7: Point operations are just transfer curves $T(v)$. Slope > 1 increases contrast in that intensity range; slope < 1 compresses it.

Key Idea: The recipe (from the next lecture's review slide)

Let I have $M \times N$ pixels, $I_{\max}$ the maximum intensity (255), and $h(l)$ the histogram. Integrate (cumulatively sum) h to get the cumulative distribution

$$c_k = \frac{1}{M \times N} \sum_{l=0}^k h(l) \quad (\text{slide indexes from } l = 1),$$

and transform each pixel by

$$\hat{I}(i, j) = I_{\max} \times c_{I(i, j)} \quad (\text{rounded}).$$

E.g. on the slide's example, value 90 has $c_{90} = 0.29$ and is mapped to $\text{round}(255 \times 0.29) = 74$.

Derivation: Why the CDF equalises — continuous proof

Treat intensity as a continuous random variable $r \in [0, 1]$ with density $p_r(r)$ and CDF $F_r(r) = \int_0^r p_r(w) dw$. Choose the monotonically increasing transform $s = T(r) = F_r(r)$. For monotone T , probability mass is conserved: $p_s(s) |ds| = p_r(r) |dr|$, so

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right| = \frac{p_r(r)}{dT/dr} = \frac{p_r(r)}{p_r(r)} = 1, \quad s \in [0, 1].$$

So s is **uniformly distributed** — a perfectly flat histogram. Scaling by $I_{\max}$ gives $s \in [0, I_{\max}]$. In the discrete case, the sum replaces the integral and rounding makes the result only *approximately* flat (levels can merge, never split).

Intuition: Where the CDF is steep, contrast is stretched

$dT/dr = p_r(r)$: the slope of the transfer curve equals the density. Intensity ranges that contain many pixels (tall histogram bars) get a steep slope $\Rightarrow$ they are spread apart; sparse ranges are compressed.

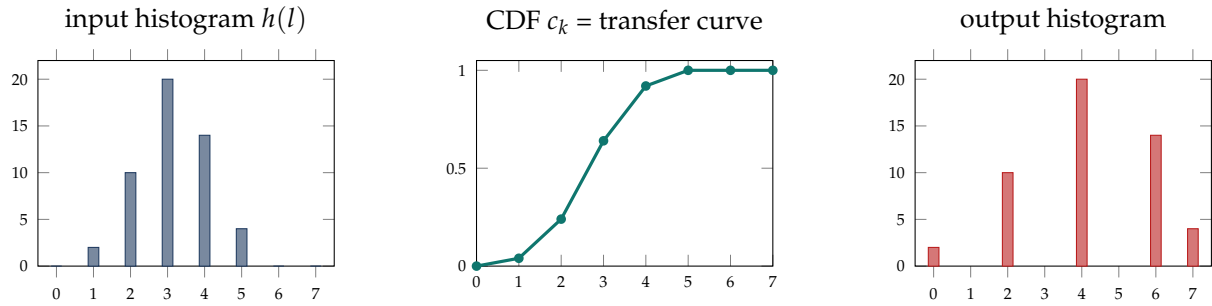


Figure 4.8: Histogram equalisation on a 3-bit (0..7) image of 50 pixels (worked out in Problem A.2). The clustered histogram is spread over the full range.

4.6.4 How useful are point operations?

A single pixel's intensity is influenced by many factors (light source, surface geometry/material/nearby surfaces, sensor properties, representation) — a single value may not tell us everything.

Question: given a camera and a *still scene*, how do you reduce noise using point operations?

Answer: take many images and *average* them (a point operation across time, per pixel).

Derivation: Averaging K frames reduces noise std by $\sqrt{K}$

Model frame k as $I_k(\mathbf{x}) = S(\mathbf{x}) + n_k(\mathbf{x})$ with i.i.d. zero-mean noise, $\text{Var}[n_k] = \sigma^2$. Then

$$\bar{I} = \frac{1}{K} \sum_{k=1}^K I_k = S + \frac{1}{K} \sum_{k=1}^K n_k, \quad \text{Var}[\bar{I}] = \frac{1}{K^2} \sum_{k=1}^K \sigma^2 = \frac{\sigma^2}{K}.$$

Standard deviation drops by $\sqrt{K}$: 16 frames $\Rightarrow 4\times$ less noise. The signal S is unchanged because the scene is still.

With only *one* image, we need **local operations**: average each pixel with its *spatial* neighbours (assuming neighbours share the same true value).

4.7 Local Operations: The Moving Average

Replace each pixel with the **mean of its 3×3 neighbourhood**:

$$G(i, j) = \frac{1}{9} \sum_{u=-1}^1 \sum_{v=-1}^1 I(i+u, j+v).$$

Derivation: Reading off a few output values

Indices (i, j) are 0-based rows/columns of the input.

- $G(1, 2)$: window rows 0–2, cols 1–3 contains a single 90 at $(2, 3) \Rightarrow 90/9 = 10$.
- $G(1, 3)$: rows 0–2, cols 2–4 contain 90's at $(2, 3), (2, 4) \Rightarrow 180/9 = 20$.
- $G(1, 4)$: three 90's $\Rightarrow 270/9 = 30$.
- $G(4, 4)$: rows 3–5, cols 3–5: nine cells, one of which is the "hole" $(5, 4) = 0 \Rightarrow 8 \cdot 90/9 = 80$.
- $G(8, 1)$: rows 7–9, cols 0–2 contain the isolated 90 at $(8, 2) \Rightarrow 10$.

Observe: the **hole** (a 0 inside the 90 block) is *filled in* to 80 and the **isolated bright pixel** is

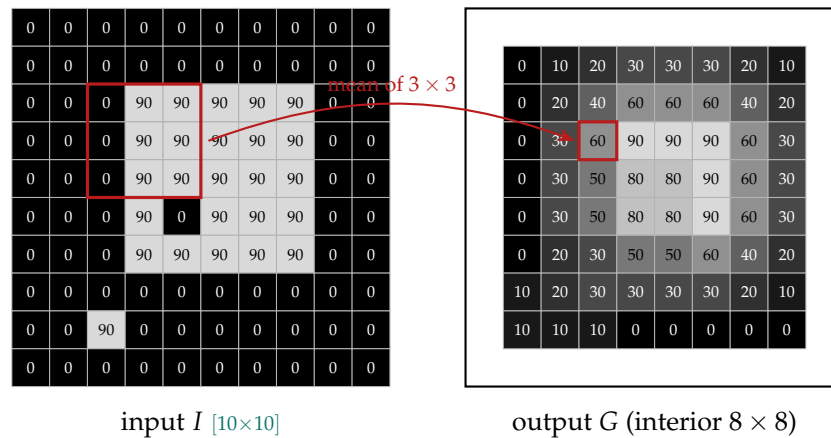


Figure 4.9: The classic moving-average example (Seitz). The red 3×3 window (centred at input row 3, column 3) contains six 90's, so the output there is $540/9 = 60$. Border pixels are left blank because the window would fall off the image.

spread into a 3×3 patch of 10's — noise is attenuated but also smeared; edges become ramps ($0 \rightarrow 30 \rightarrow 60 \rightarrow 90$).

4.8 Global Operations

Output at (m_0, n_0) depends on *all* values of the input. Examples: image coordinate transformations such as the **Fourier transform** (Module 6), histogram equalisation (its T depends on the global histogram, though each pixel is then mapped pointwise).

Exam Tip

Histogram equalisation is a subtle case: the *mapping* is computed globally, but once computed it is applied as a point operation. If asked “point or global?”, explain both aspects.

Readings: Szeliski §3.1. **Question:** what is histogram equalisation and how do you derive its formula? (Done above.)

Module Summary: §1.4

RGB because humans are trichromats (L/M/S cones). An image is a matrix (8-bit, one per channel) or a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, made digital by sampling + quantisation. Operations: point ($O(1)$), local ($O(p^2)$), global ($O(N^2)$) per pixel. Point ops: negative $I_{\max} + I_{\min} - I$, linear stretch, gamma, histogram equalisation ($T = I_{\max} \cdot \text{CDF}$, which makes the output uniform). Averaging K frames reduces noise std by $\sqrt{K}$; with one frame use local averaging (moving average), which leads to linear filtering.

Linear Filtering, Correlation and Convolution

5.1 Review: Histogram Equalisation

(Asked at the start of this lecture; fully derived in §4.6.3.) With $M \times N$ pixels, $I_{\max} = 255$ and histogram h : $c_k = \frac{1}{MN} \sum_{l \leq k} h(l)$ and $\hat{I}(i, j) = I_{\max} \cdot c_{I(i, j)}$. E.g. $90 \mapsto \text{round}(255 \times 0.29) = 74$.

5.2 Image Filters

Key Idea: Definitions

Image filter: modify image pixels based on some function of a *local neighbourhood* of each pixel.

Linear filter: replace each pixel by a *linear combination (weighted sum)* of its neighbours. The weights are called the **kernel**, **mask** or **filter**; its entries $H(u, v)$ are the **filter coefficients**.

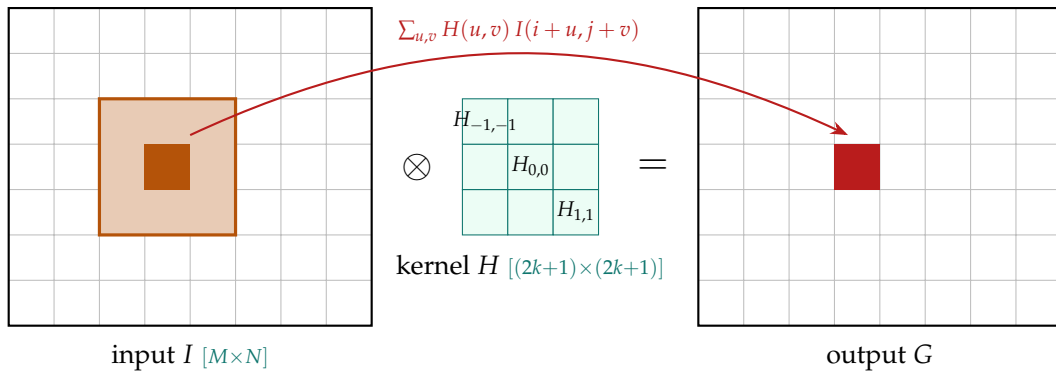


Figure 5.1: Filtering: slide the kernel over the image; at each location take the weighted sum of the covered pixels.

5.3 Correlation and Cross-Correlation

Let the kernel be of size $(2k + 1) \times (2k + 1)$, indexed $u, v \in \{-k, \dots, k\}$ so that $(0, 0)$ is the centre.

5.3.1 Correlation with uniform weights (moving average)

$$G(i, j) = \underbrace{\frac{1}{(2k+1)^2}}_{\text{uniform weight to each pixel}} \underbrace{\sum_{u=-k}^k \sum_{v=-k}^k I(i+u, j+v)}_{\text{loop over the neighbourhood of } I(i, j)} .$$

5.3.2 Cross-correlation with non-uniform weights

$$G(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k H(u, v) I(i+u, j+v) \quad \text{denoted } G = H \otimes I.$$

- Can be viewed as a **dot product** between the local neighbourhood and the kernel, for each pixel: $G(i, j) = \langle \text{vec}(H), \text{vec}(I_{\mathcal{N}(i,j)}) \rangle$, both vectors of length $(2k+1)^2$.
- Dimensions: I $[M \times N]$, H $[(2k+1) \times (2k+1)]$, G $[(M-2k) \times (N-2k)]$ without padding ("valid"), or $[M \times N]$ with k pixels of padding ("same").

Intuition: Correlation = similarity score

A dot product is large when two vectors point the same way. So $G(i, j)$ is large where the image patch *looks like* the kernel. A kernel is therefore a tiny “template”, and filtering asks “how much does each patch resemble this template?” — exactly what a CNN’s first layer does with learned kernels.

5.4 Averaging Filters

5.4.1 Moving average = box filter

The moving-average example of Module 4 is cross-correlation with

$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad [3 \times 3].$$

Effect: smooths/blurs the image; noise is reduced but edges get soft, and because the kernel has sharp edges it produces slightly “blocky” artefacts (see Module 6: its Fourier transform is a sinc with ripples).

5.4.2 Gaussian averaging filter

What if we want nearest neighbouring pixels to have the most influence? Use weights that decay with distance, e.g.

$$H = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix},$$

an approximation of the 2D Gaussian

$$h(u, v) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{u^2 + v^2}{2\sigma^2}\right)$$

Derivation: Normalisation and properties of the Gaussian kernel

- **Sums to 1:** $\iint h \, du \, dv = \frac{1}{2\pi\sigma^2} \left(\int e^{-u^2/2\sigma^2} du \right) \left(\int e^{-v^2/2\sigma^2} dv \right) = \frac{1}{2\pi\sigma^2} \sqrt{2\pi}\sigma \sqrt{2\pi}\sigma = 1$. So filtering preserves average brightness (a flat region stays unchanged). For a discrete kernel, divide by the sum of the sampled weights.
- **Separable:** $h(u, v) = g(u)g(v)$ with $g(t) = \frac{1}{\sqrt{2\pi}\sigma} e^{-t^2/2\sigma^2}$ (because $e^{a+b} = e^a e^b$) — see §5.10.

- **Rotationally symmetric:** depends only on $r^2 = u^2 + v^2$ — no preferred direction.
- **Size rule of thumb:** 99.7% of the mass lies within 3σ , so use half-width $k = \lceil 3\sigma \rceil$, i.e. kernel width $2\lceil 3\sigma \rceil + 1$.
- σ controls blur: larger $\sigma \Rightarrow$ wider averaging $\Rightarrow$ more smoothing.

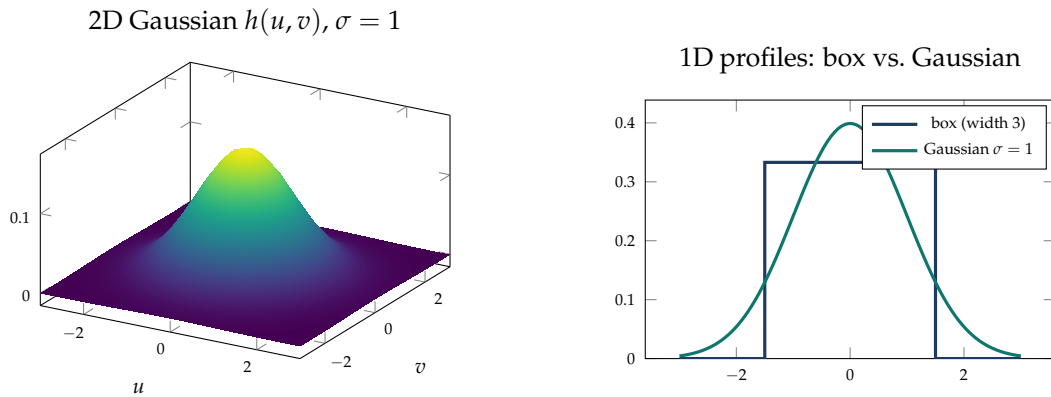


Figure 5.2: Box vs. Gaussian weights. The box gives equal weight up to a hard cut-off; the Gaussian falls off smoothly.

Key Idea: Box vs. Gaussian (comparison slide)

Both blur. The **box filter** weights all neighbours equally and has a hard boundary $\Rightarrow$ “blocky” smoothing and ringing in frequency. The **Gaussian filter** weights near neighbours more $\Rightarrow$ smoother, more natural blur, no ringing. The Gaussian is the default smoothing kernel in vision.

5.5 Other Filters: The Edge Filter

What should H look like to find edges? An edge is a place where intensity changes quickly, i.e. a large derivative. Approximate the derivative with a finite difference:

$$\frac{\partial I}{\partial x} \approx \frac{I(i, j+1) - I(i, j-1)}{2} \iff H = \frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad (\text{cross-correlation}).$$

Smoothing perpendicular to the derivative gives the Prewitt/Sobel kernels:

$$H_x^{\text{Sobel}} = \frac{1}{8} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (\text{vertical edges}), \quad H_y^{\text{Sobel}} = \frac{1}{8} \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (\text{horizontal edges}).$$

The output G is signed (positive for dark $\rightarrow$ bright, negative for bright $\rightarrow$ dark); the slide displays $|G(i, j)|$ to show edge strength.

Intuition: Smoothing vs. edge kernels

Smoothing kernels are all positive and **sum to 1** (preserve flat regions). Derivative/edge kernels have positive and negative weights and **sum to 0** (a flat region gives zero response). This single check classifies most kernels on sight.

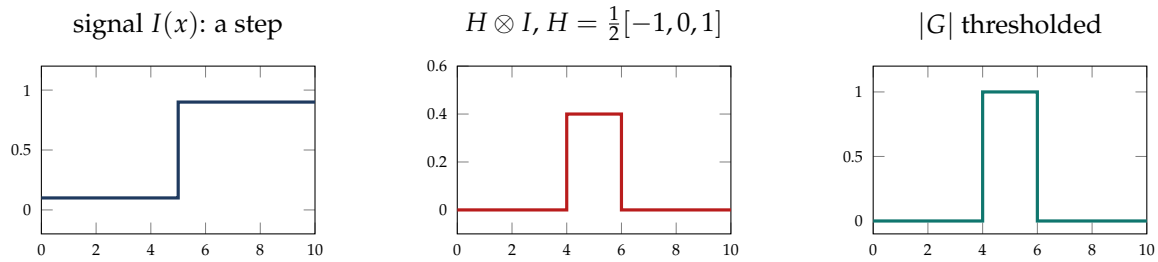


Figure 5.3: A derivative kernel turns a step edge into a peak — edge detection = filtering + thresholding.

5.6 Beyond Correlation: The Impulse Response

What is the result of filtering an impulse image with an arbitrary kernel H ? Let $I = \delta$ (a single 1 at the centre, zeros elsewhere) and $H = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$.

Derivation: Correlation flips the kernel

$G(i, j) = \sum_{u, v} H(u, v) \delta(i + u, j + v)$. The only non-zero term has $i + u = 0, j + v = 0$, i.e. $u = -i, v = -j$:

$$G(i, j) = H(-i, -j).$$

So the output around the impulse is H **rotated by 180°**:

$$\delta \otimes H \text{ gives } \begin{bmatrix} i & h & g \\ f & e & d \\ c & b & a \end{bmatrix}.$$

E.g. at position $(-1, -1)$ (top-left) we get $H(1, 1) = i$.

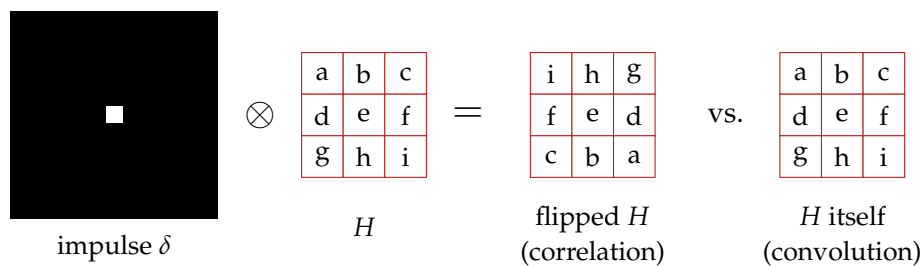


Figure 5.4: Impulse response: correlation returns the kernel flipped in both directions; convolution returns the kernel itself.

5.7 Introducing Convolution

Key Idea: Convolution

For a $(2k + 1) \times (2k + 1)$ kernel,

$$G(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k H(u, v) I(i - u, j - v) \quad \text{denoted} \quad G = H * I.$$

Equivalent to **flipping the filter in both directions** (bottom-to-top, right-to-left) and then applying cross-correlation.

Derivation: Convolution = correlation with the flipped kernel

Define $\tilde{H}(u, v) = H(-u, -v)$. Substitute $u' = -u$, $v' = -v$ in the convolution sum (the range $-k..k$ is symmetric, so it is unchanged):

$$\begin{aligned} \sum_{u,v} H(u, v) I(i - u, j - v) &= \sum_{u',v'} H(-u', -v') I(i + u', j + v') \\ &= \sum_{u',v'} \tilde{H}(u', v') I(i + u', j + v') = (\tilde{H} \otimes I)(i, j). \end{aligned}$$

Impulse response: $(H * \delta)(i, j) = \sum_{u,v} H(u, v) \delta(i - u, j - v) = H(i, j)$ — convolution with an impulse *reproduces* the kernel, which is why convolution (not correlation) is the natural operation for LSI systems: a system is fully described by its impulse response.

Pitfall / Careful: Symmetric kernels and deep learning

For kernels symmetric under 180° rotation (box, Gaussian) correlation = convolution. For asymmetric kernels (Sobel) they differ by a sign/flip. “Convolution” layers in PyTorch/TensorFlow actually compute **cross-correlation** — it does not matter because the kernel is learned (it would just learn the flipped version).

5.7.1 Biological motivation: linear summation in the visual cortex

Recall Hubel & Wiesel (1959). Movshon, Thompson & Tolhurst (1978) showed that **simple cells perform linear spatial summation over their receptive fields** — i.e. a simple cell’s response is (approximately) a linear filter applied to the image patch in its receptive field.

5.8 Linear Shift-Invariant (LSI) Operators

Both correlation and convolution are **linear shift-invariant** operators. Writing $\circ$ for the operator:

- **Linearity (superposition):** $I \circ (h_0 + h_1) = I \circ h_0 + I \circ h_1$, and more generally $(\alpha I_1 + \beta I_2) \circ h = \alpha(I_1 \circ h) + \beta(I_2 \circ h)$.
- **Shift invariance:** shifting the signal commutes with applying the operator. If $I'(i, j) = I(i - a, j - b)$ then $(I' \circ h)(i, j) = (I \circ h)(i - a, j - b)$.

Pitfall / Careful: Slide notation

The slide writes shift invariance as $g(i, j) = h(i + k, j + l) \iff (f \circ g)(i, j) = (f \circ h)(i + k, j + l)$: “if one input is a shifted copy of another, the outputs are equally shifted copies”. The version above is the same statement in the more common form.

Derivation: Proof that convolution is shift-invariant

Let $I'(i, j) = I(i - a, j - b)$. Then $(H * I')(i, j) = \sum_{u, v} H(u, v) I'(i - u, j - v) = \sum_{u, v} H(u, v) I(i - a - u, j - b - v) = (H * I)(i - a, j - b)$. $\square$

Linearity follows because the sum is linear in I .

Intuition: Why do we want shift invariance in vision?

“The effect of the operator is the same everywhere.” A cat in the top-left corner should be detected by the same feature detector as a cat in the bottom-right. Shift invariance (strictly, *equivariance*: shift in $\Rightarrow$ shift out) lets one small kernel be reused at every location — this **weight sharing** is the core inductive bias of CNNs and why they need far fewer parameters than fully-connected nets.

5.9 Properties of Convolution

Property	Statement	Meaning / use
Commutative	$a * b = b * a$	Conceptually no difference between filter and signal.
Associative	$a * (b * c) = (a * b) * c$	Applying filters one after another, $((a * b_1) * b_2) * b_3$, equals applying one cumulative filter $a * (b_1 * b_2 * b_3)$.
Distributive	$a * (b + c) = a * b + a * c$	Responses to two filters can be combined by combining the filters.
Scalars factor out	$ka * b = a * kb = k(a * b)$	Scale before or after — same result.
Identity	$a * e = a, e = [\dots, 0, 0, 1, 0, 0, \dots]$	The unit impulse is the identity element.

Derivation: Commutativity and associativity (1D, infinite support)

Commutativity: $(a * b)[n] = \sum_m a[m] b[n - m]$. Substitute $m' = n - m$: $= \sum_{m'} a[n - m'] b[m'] = (b * a)[n]$.

Associativity:

$$\begin{aligned}
 ((a * b) * c)[n] &= \sum_p \left(\sum_m a[m] b[p - m] \right) c[n - p] = \sum_m a[m] \sum_q b[q] c[n - m - q] \quad (q = p - m) \\
 &= \sum_m a[m] (b * c)[n - m] = (a * (b * c))[n].
 \end{aligned}$$

Pitfall / Careful

Cross-correlation is **not** commutative or associative in general (because of the missing flip): $a \otimes b \neq b \otimes a$ for asymmetric kernels. This is one reason convolution is the mathematically preferred operation.

Exam Tip: Useful consequence of associativity

Smoothing then differentiating, $\partial * (G_\sigma * I)$, equals $(\partial * G_\sigma) * I$: pre-compute the “derivative of Gaussian” kernel once. Also, two Gaussians compose to a Gaussian: $G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}$. Size rule: convolving a k_1 -wide kernel with a k_2 -wide kernel yields a $(k_1 + k_2 - 1)$ -wide kernel (e.g. two $3 \times 3 \rightarrow$ one 5×5 — the VGG insight).

5.10 Separability

A $k \times k$ convolution needs k^2 multiply-adds per pixel. For some kernels we can instead do a 1D *vertical* convolution followed by a 1D *horizontal* convolution: $2k$ operations. Such a kernel is **separable**:

$$K = \mathbf{v} \mathbf{h}^\top \quad \mathbf{v} [k \times 1], \mathbf{h} [k \times 1], K [k \times k] \text{ (rank 1).}$$

Derivation: Why $K = \mathbf{v} \mathbf{h}^\top$ allows two 1D passes

$K(u, w) = v(u)h(w)$. Then

$$\begin{aligned} (K * I)(i, j) &= \sum_u \sum_w v(u)h(w) I(i - u, j - w) \\ &= \sum_u v(u) \underbrace{\left[\sum_w h(w) I(i - u, j - w) \right]}_{J(i - u, j) = (\mathbf{h} *_{\text{rows}} I)} = (\mathbf{v} *_{\text{cols}} J)(i, j). \end{aligned}$$

Cost per pixel: k (horizontal) + k (vertical) = $2k$ instead of k^2 . Speed-up = $k/2$: for $k = 15$, $225 \rightarrow 30$ ops ($7.5\times$).

Example 1 (Gaussian/binomial).

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \frac{1}{4} \underbrace{\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}}_{\mathbf{v}} \underbrace{\frac{1}{4} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}}_{\mathbf{h}^\top}, \quad \mathbf{v} = \mathbf{h} = \frac{1}{4} [1, 2, 1]^\top.$$

Example 2 (Sobel).

$$\frac{1}{8} \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \frac{1}{4} \underbrace{\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}}_{\mathbf{v}: \text{smooth}} \underbrace{\frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}}_{\mathbf{h}^\top: \text{differentiate}}.$$

(The factorisation is unique only up to scale and sign: $(\alpha \mathbf{v})(\mathbf{h}/\alpha)^\top$ gives the same K , so $\mathbf{v} = -\frac{1}{4} [1, 2, 1]^\top$, $\mathbf{h} = \frac{1}{2} [1, 0, -1]^\top$ is equally valid — check any answer by multiplying out.) Sobel = “smooth in y , differentiate in x ”.

5.10.1 How can we tell if K is separable?

- **Visual inspection:** are all rows (or columns) scalar multiples of each other?
- **Analytically — SVD:** write

$$K = U \Sigma V^\top = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\top, \quad \Sigma = \text{diag}(\sigma_i).$$

$$K \text{ [3x3]: 9 ops} = \mathbf{v} \mathbf{h}^T: 3 + 3 = 6 \text{ ops}$$

Figure 5.5: A separable kernel is an outer product of a column and a row.

If only **one** singular value is non-zero, $K = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T$ is separable, with vertical kernel $\sqrt{\sigma_1} \mathbf{u}_1$ and horizontal kernel $\sqrt{\sigma_1} \mathbf{v}_1$.

Key Idea: Beyond the slide: low-rank approximation

If K is not separable but $\sigma_1 \gg \sigma_2$, keep the first r terms: $K \approx \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ — cost $2kr$ instead of k^2 . (Eckart–Young: this is the best rank- r approximation in Frobenius norm.) The same idea compresses CNN layers.

5.11 Practical Issues

5.11.1 Ideal size for the filter?

The bigger the mask:

- more neighbours contribute;
- **smaller noise variance** of the output (for i.i.d. noise and an n -pixel box: $\sigma^2 \rightarrow \sigma^2/n$);
- bigger noise **spread** (a noisy pixel affects a larger area);
- **more blurring**;
- more expensive to compute (k^2 per pixel).

Derivation: Output noise variance of a linear filter

If $I = S + n$ with i.i.d. noise of variance σ^2 , the noise in the output is $\sum_{u,v} H(u,v) n(i-u, j-v)$ with variance $\sigma^2 \sum_{u,v} H(u,v)^2$. For a $k \times k$ box, $H = 1/k^2$ everywhere: $\sigma^2 \cdot k^2 \cdot \frac{1}{k^4} = \sigma^2/k^2$. For the $\frac{1}{16}[1, 2, 1; 2, 4, 2; 1, 2, 1]$ Gaussian: $\sum H^2 = \frac{1+4+1+4+16+4+1+4+1}{256} = \frac{36}{256} \approx 0.141$ (vs. $1/9 \approx 0.111$ for the 3×3 box).

5.11.2 What about the boundaries?

Without padding, we lose information at the boundaries (output shrinks by $2k$ in each dimension). Strategies:

Zero padding	Pad with 0 (constant). Simple; darkens borders after blurring (“blurred zero”). Standard in CNNs.
Wrap around	Treat the image as periodic (torus). Matches the DFT’s assumption; creates seams from opposite edges.
Copy edge (clamp)	Repeat the border pixel. Avoids dark borders.
Mirror (reflect)	Reflect across the edge. Usually the most natural for smoothing.
Normalised zero	Zero pad, then divide by the sum of weights actually inside the image.

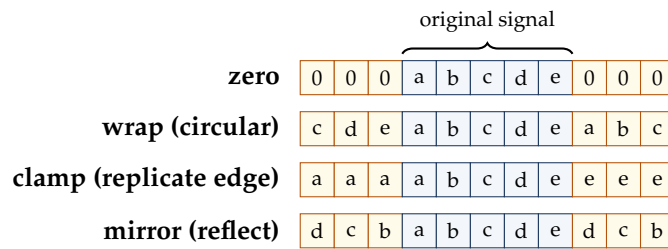


Figure 5.6: Padding strategies illustrated on a 1D row $[a, b, c, d, e]$ padded by 3 on each side (padded values in amber). 2D images apply the same rule along rows and columns.

Derivation: Output size formula (used throughout CNNs)

For input width N , kernel width k , padding p on each side and stride s :

$$N_{\text{out}} = \left\lfloor \frac{N + 2p - k}{s} \right\rfloor + 1$$

Valid ($p = 0, s = 1$): $N - k + 1$. *Same* ($s = 1$): need $N + 2p - k + 1 = N \Rightarrow p = (k - 1)/2$ (for odd $k = 2m + 1, p = m$). E.g. $N = 32, k = 5, p = 2, s = 1 \Rightarrow 32$; $N = 32, k = 5, p = 0, s = 1 \Rightarrow 28$.

5.12 Questions to Think About

1. Do we need (cross-)correlation at all? Yes — for **template matching** (next module).
2. Are all filters linear? No — median and bilateral filters are non-linear (next module).

Readings: Szeliski Ch. 3 (§3.1–3.3); Forsyth & Ponce Ch. 7 (§7.1–7.2).

Module Summary: §1.5

Linear filter = weighted sum over a neighbourhood. Cross-correlation $G = \sum H(u, v)I(i + u, j + v)$ (dot-product/template view); convolution $G = \sum H(u, v)I(i - u, j - v)$ = correlation with a 180°-flipped kernel; impulse response of correlation is the flipped kernel, of convolution the kernel itself. Box and Gaussian blur (sum to 1); derivative/Sobel kernels detect edges (sum to 0). Both operations are LSI; convolution is commutative, associative, distributive, has identity δ . Separable kernels ($K = \mathbf{v}\mathbf{h}^\top$, rank 1 by SVD) cost $2k$ instead of k^2 . Bigger kernels: less noise, more blur, more cost. Pad with zero/wrap/clamp/mirror; $N_{\text{out}} = \lfloor (N + 2p - k)/s \rfloor + 1$.

Image in the Frequency Domain

This lecture first answers the two questions left open in Module 5 — *do we still need correlation?* and *are all filters linear?* — and then introduces the Fourier view of images.

6.1 Is Correlation Still Useful? Template Matching

- Correlation can be used for **template matching**: filters *look like the objects they are intended to find*.
- Use **Normalised Cross-Correlation (NCC)** — to control for relative brightness — as the score for finding a given pattern.
- Even if the template is not identical to a sub-image in the scene, the match can be meaningful if scale, orientation and general appearance are right.

Derivation: From raw correlation to NCC

Raw correlation $\sum_{u,v} T(u,v) I(i+u, j+v)$ is biased: a *bright* region scores high whether or not it matches T . Fix it by (1) subtracting means, (2) dividing by norms. With $\bar{T}$ the template mean and $\bar{I}_{ij}$ the mean of the patch under the template:

$$\text{NCC}(i, j) = \frac{\sum_{u,v} (T(u,v) - \bar{T})(I(i+u, j+v) - \bar{I}_{ij})}{\sqrt{\sum_{u,v} (T(u,v) - \bar{T})^2} \sqrt{\sum_{u,v} (I(i+u, j+v) - \bar{I}_{ij})^2}} \in [-1, 1].$$

This is the **cosine similarity** (Pearson correlation) between the mean-centred template and patch vectors. By Cauchy–Schwarz $|\text{NCC}| \leq 1$, with equality iff patch = $aT + b$ for some $a > 0$ (NCC = +1). Hence it is **invariant to affine brightness/contrast changes** $I \rightarrow aI + b$ ($a > 0$).

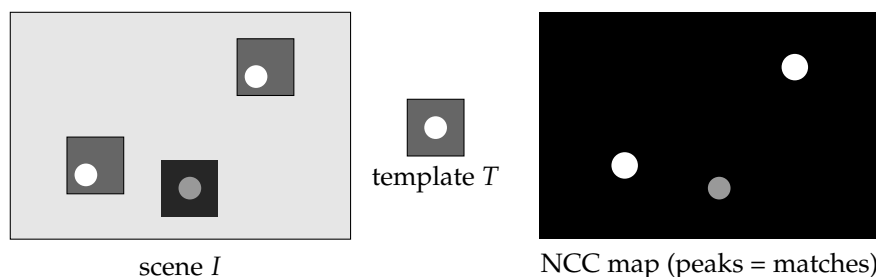


Figure 6.1: Template matching: NCC gives peaks where the scene matches the template, including the darker (contrast-changed) copy.

6.2 Non-Linear Filters

6.2.1 Types of noise

- **Salt-and-pepper noise**: random pixels set to black (0) or white (255).

- **Impulse noise:** random occurrences of white pixels.
- **Gaussian noise:** each pixel perturbed by $n \sim \mathcal{N}(0, \sigma^2)$ — small changes everywhere.

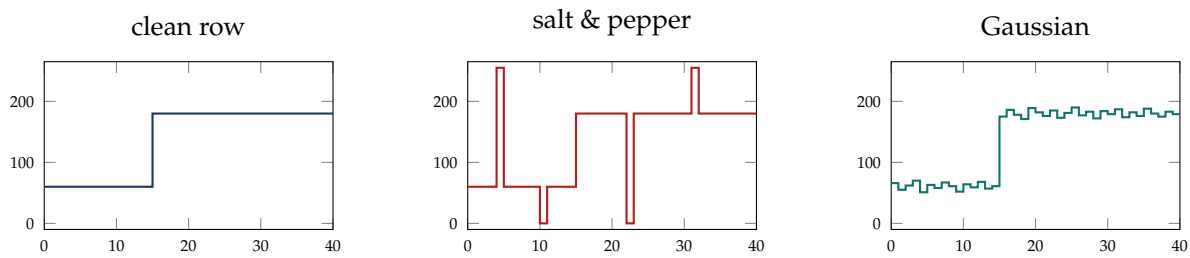


Figure 6.2: One image row under different noise models.

Reducing salt-and-pepper noise with Gaussian filters — the problem: a Gaussian (any linear average) *spreads* each outlier into its neighbourhood instead of removing it; to suppress it you need a large σ , which blurs the whole image. The outliers are *not* small zero-mean perturbations, so averaging is the wrong model. **What do we do?** Use a non-linear, robust statistic.

6.2.2 Median filter

Replace each pixel with the **median** of all pixels in its neighbourhood.

- **Non-linear:** $\text{med}(A + B) \neq \text{med}(A) + \text{med}(B)$ in general.
- **Does not spread noise;** can **remove spike noise** completely.
- **Robust to outliers** (breakdown point 50%), and **preserves step edges**.
- **Not good for Gaussian noise** (the mean is the better estimator there).

Derivation: Example: the outlier disappears

3×3 neighbourhood (centre = 99, an outlier):

$$\begin{bmatrix} 10 & 11 & 10 \\ 9 & \mathbf{99} & 11 \\ 10 & 11 & 12 \end{bmatrix} \xrightarrow{\text{sort}} 9, 10, 10, 10, \underline{11}, 11, 11, 12, 99 \Rightarrow \text{median} = 11.$$

The mean would be $183/9 \approx 20.3$ — the outlier drags it far from the true ≈ 10.5 . The median ignores it.

Why non-linear (counter-example, 1D, window 3): $A = [0, 0, 9]$, $B = [9, 0, 0]$: $\text{med } A = 0$, $\text{med } B = 0$, but $\text{med}(A + B) = \text{med}[9, 0, 9] = 9 \neq 0$.

Why edges survive (1D, window 3): $[0, 0, 0, 10, 10, 10] \rightarrow [\cdot, 0, 0, 10, 10, \cdot]$ — the step is untouched, whereas a box filter gives $[\cdot, 0, 3.3, 6.7, 10, \cdot]$.

Key Idea: Mean vs. median as optimisation

The mean minimises $\sum_k (x_k - m)^2$ (ℓ_2); the median minimises $\sum_k |x_k - m|$ (ℓ_1). ℓ_1 penalises large deviations only linearly, so outliers have bounded influence — that is robustness.

6.2.3 Bilateral filtering

Noise removal usually comes at the expense of blurring edges. **Bilateral filtering** is a simple, non-linear, **edge-preserving smoothing**: *reject (softly) pixels whose values differ too much from the central*

pixel value.

$$g(i, j) = \frac{\sum_{k,l} I(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$

with a data-dependent weight composed of a **domain kernel** (spatial closeness) and a **range kernel** (intensity similarity):

$$w(i, j, k, l) = \exp \left(-\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2} - \frac{\|I(i, j) - I(k, l)\|^2}{2\sigma_r^2} \right)$$

$$= \underbrace{e^{-\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2}}}_{\text{domain (Gaussian in space)}} \cdot \underbrace{e^{-\frac{\|I(i, j) - I(k, l)\|^2}{2\sigma_r^2}}}_{\text{range (Gaussian in intensity)}}.$$

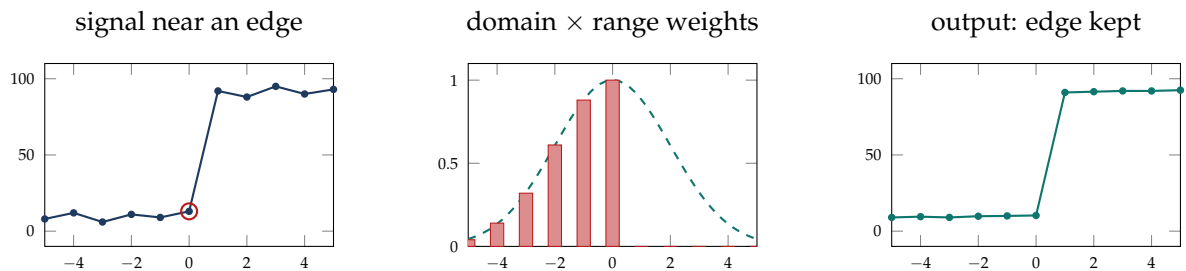


Figure 6.3: Bilateral filter at the circled pixel (value 13): the domain kernel (dashed) would average across the edge, but the range kernel gives ≈ 0 weight to the pixels near 90, so only same-side pixels are averaged.

Intuition: Two limits

$\sigma_r \rightarrow \infty$: range term $\rightarrow 1$, the bilateral filter becomes an ordinary Gaussian blur. $\sigma_r \rightarrow 0$: only pixels with (nearly) identical values get weight — almost no smoothing. Across a strong edge, $\|I(i, j) - I(k, l)\| \gg \sigma_r$, so the weight vanishes and the edge is preserved. The filter is non-linear because w depends on the image itself, and it is not shift-invariant in the LSI sense (the kernel changes at every pixel).

6.3 What Do We Lose in a Low-Resolution Image?

Compare a high-resolution photo with a downscaled version: the overall layout and colours remain, but **fine detail** (leaves, texture, sharp edges) is gone. Fine detail = rapid change = **high spatial frequency**. To reason about this precisely we need a representation in terms of frequencies.

6.4 Fourier's Idea

Jean Baptiste Joseph Fourier (1768–1830) had an idea in 1807: **any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies**.

- Many (Lagrange, Laplace, Poisson and other “big wigs”) did not believe him; not translated into English until 1878.
- (Mostly) true! Called the **Fourier series**; there are some subtle restrictions (Dirichlet conditions: e.g. finitely many discontinuities, absolutely integrable).

6.4.1 A sum of sines

Building block: $A \sin(\omega x + \phi)$ — **amplitude** A , **frequency** ω , **phase** ϕ . Add enough of them to get any signal $f(x)$ you want.

Example. $g(t) = \sin(2\pi f t) + \frac{1}{3} \sin(2\pi(3f)t)$.

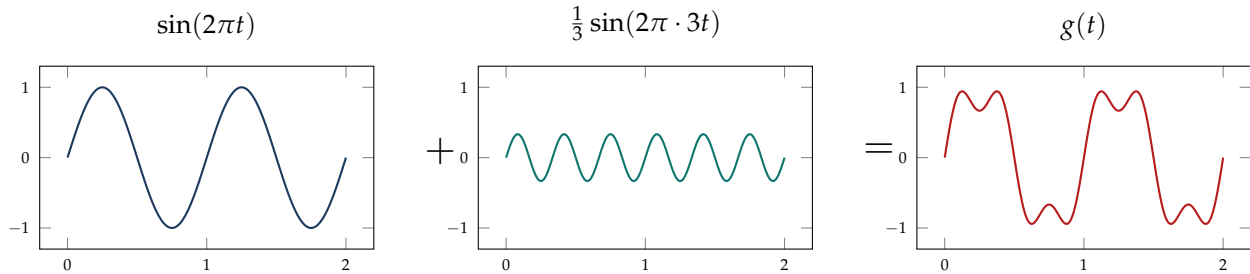


Figure 6.4: Two sinusoids add up to a signal that already looks “squarer”. Its spectrum has two spikes: height 1 at f and $\frac{1}{3}$ at $3f$.

6.4.2 Building a square wave

Keep adding odd harmonics with amplitude $1/k$:

$$\text{square}(t) = A \sum_{k \text{ odd}} \frac{1}{k} \sin(2\pi k t)$$

(the slide writes $A \sum_{k=1}^{\infty} \frac{1}{k} \sin(2\pi k t)$ with the understanding that only odd k appear; with $A = \frac{4}{\pi}$ the square wave has amplitude ± 1).

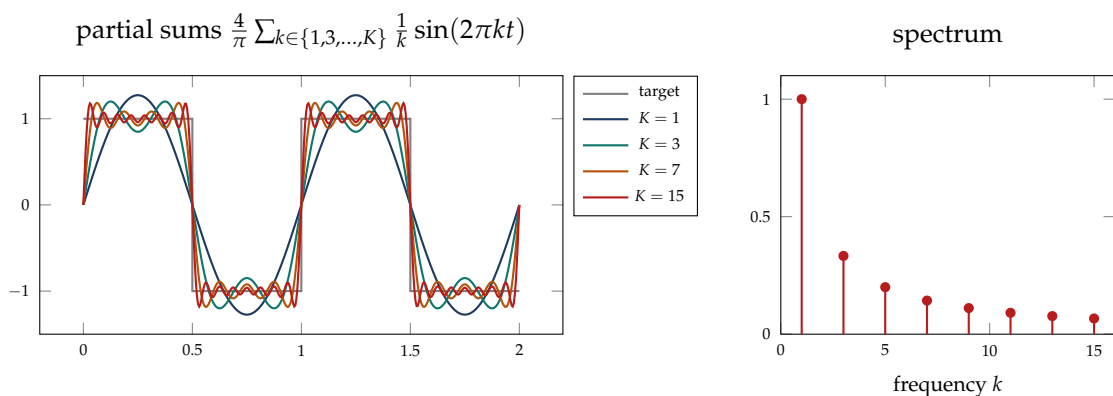


Figure 6.5: Adding odd harmonics converges to a square wave. Sharp edges require *many high frequencies*; the overshoot near jumps ($\approx 9\%$) is the Gibbs phenomenon.

Derivation: Fourier coefficients of the square wave

Let $s(t) = +1$ on $(0, \frac{1}{2})$, -1 on $(\frac{1}{2}, 1)$, period 1. It is odd, so only sine terms: $s(t) = \sum_k b_k \sin(2\pi k t)$ with

$$\begin{aligned} b_k &= 2 \int_0^1 s(t) \sin(2\pi k t) dt = 2 \left[\int_0^{1/2} \sin(2\pi k t) dt - \int_{1/2}^1 \sin(2\pi k t) dt \right] \\ &= 2 \left[\frac{1 - \cos \pi k}{2\pi k} - \frac{\cos \pi k - \cos 2\pi k}{2\pi k} \right] = \frac{2(1 - \cos \pi k)}{\pi k}, \end{aligned}$$

which simplifies to $b_k = \frac{2}{\pi k} (1 - (-1)^k)$, i.e. $b_k = \frac{4}{\pi k}$ for odd k and 0 for even k . Hence $A = \frac{4}{\pi}$.

Intuition: Frequency $\leftrightarrow$ detail

Low frequencies describe **slow, large-scale variation** (overall shading); high frequencies describe **rapid change**: edges, texture, noise. This is why a low-resolution image (which cannot hold high frequencies) loses detail and why blurring = removing high frequencies.

6.5 Fourier Analysis of Images

- The Fourier image encodes a whole series of sinusoids through a range of spatial frequencies from zero (the DC/mean, at the centre) up to the **Nyquist frequency**.
- A signal containing a single spatial frequency f is plotted as a **single peak** at point f along the frequency axis; the **height** is the amplitude (contrast) of that sinusoid.
- In 2D a sinusoidal grating $\cos(2\pi(u_0x + v_0y))$ appears as **two dots** at $\pm(u_0, v_0)$ — the spectrum has **mirror-image reflections** along the axes.

Derivation: Why the spectrum of a real image is symmetric

For real $h(x)$, $H(\omega) = \int h(x)e^{-j\omega x}dx$, so $H(-\omega) = \int h(x)e^{+j\omega x}dx = \overline{\int h(x)e^{-j\omega x}dx} = \overline{H(\omega)}$ (conjugate symmetry). Therefore $|H(-\omega)| = |H(\omega)|$: the magnitude is symmetric about the origin, and the negative-frequency half carries no new information. A real cosine is $\cos \omega_0 x = \frac{1}{2}(e^{j\omega_0 x} + e^{-j\omega_0 x})$ — two complex exponentials $\Rightarrow$ two peaks at $\pm\omega_0$.

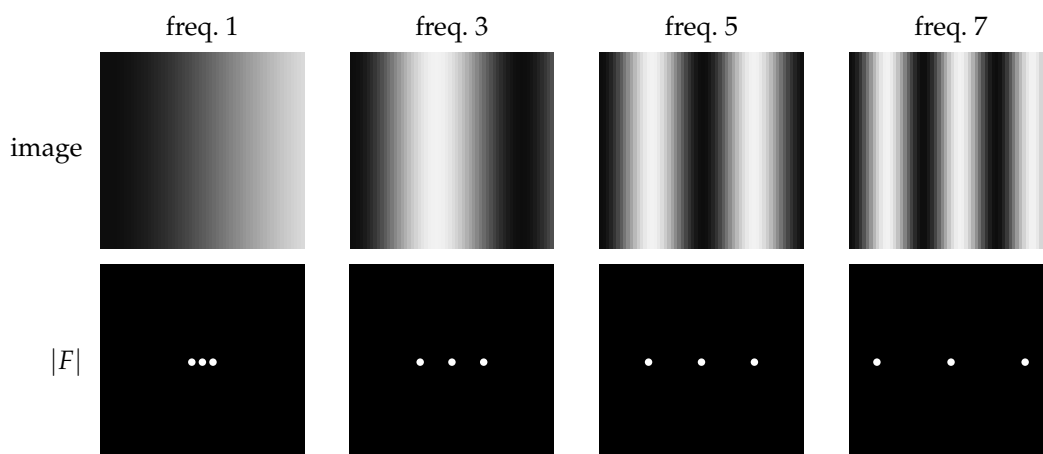


Figure 6.6: Vertical stripes at increasing frequency: the magnitude spectrum shows the DC dot at the centre and a symmetric pair of dots whose distance from the centre grows with frequency (after Forsyth & Ponce). Summing frequencies (1, 1 + 3, 1 + 3 + 5, ...) adds more dot pairs.

Natural images (zebra, cheetah example). Their magnitude spectra look broadly similar: most energy at low frequencies, decaying roughly as $1/f$, with a bright “cross” along the axes (caused by the image borders under the implicit periodic extension) and oriented streaks for strongly oriented textures (the zebra’s stripes).

6.6 Fourier Transform: Magnitude and Phase

The Fourier transform stores, at each frequency, a **complex number** $\varphi = \text{Re}(\varphi) + j\text{Im}(\varphi)$, i.e. a magnitude and a phase:

$$A = \pm \sqrt{\text{Re}(\varphi)^2 + \text{Im}(\varphi)^2}, \quad \phi = \tan^{-1} \frac{\text{Im}(\varphi)}{\text{Re}(\varphi)}.$$

- **Magnitude** encodes *how much* signal there is at a particular frequency.
- **Phase** encodes spatial information (indirectly) — *where* the structures are.

Pitfall / Careful

Magnitude is conventionally taken as the non-negative root, $A = |\varphi| \geq 0$ (the slide's $\pm$ is unnecessary). Use $\text{atan2}(\text{Im}, \text{Re})$ to get the phase in the correct quadrant: $\tan^{-1}$ alone cannot distinguish φ from $-\varphi$.

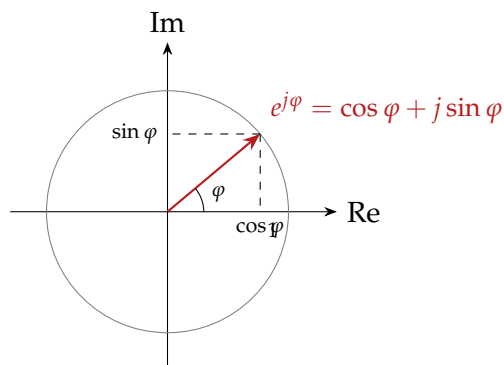


Figure 6.7: Euler's formula: a complex exponential is a point on the unit circle. Scaling by A and rotating by ϕ describes a sinusoid of amplitude A and phase ϕ .

6.6.1 Which has more information: magnitude or phase?

Experiment (Forsyth & Ponce): take the cheetah and zebra images, compute their FTs, then **swap** — combine the magnitude of one with the phase of the other and invert. The reconstruction **looks like the image whose phase was used**. So **phase carries most of the structural ("where") information**; magnitude spectra of natural images are all rather similar.

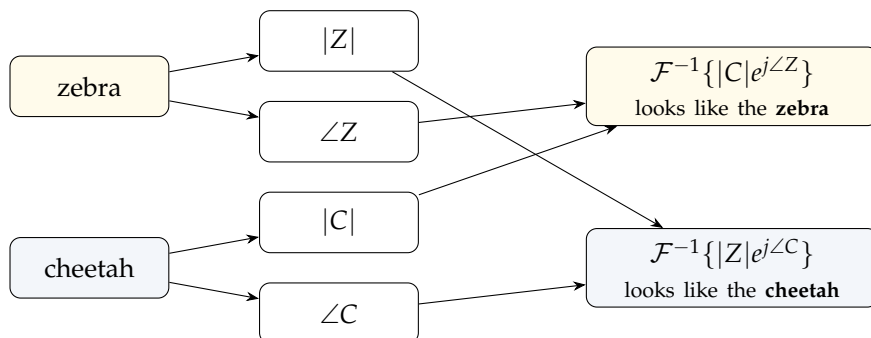


Figure 6.8: The phase-swap experiment: the image follows the phase.

6.7 Continuous vs. Discrete Fourier Transform

Key Idea: Definitions

Continuous FT: $H(\omega) = \int_{-\infty}^{\infty} h(x) e^{-j\omega x} dx$, inverse $h(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} H(\omega) e^{j\omega x} d\omega$.

Discrete FT (DFT) of N samples $h(0), \dots, h(N-1)$:

$$H(k) = \sum_{x=0}^{N-1} h(x) e^{-j\frac{2\pi kx}{N}}, \quad k = 0, \dots, N-1 \quad h(x) = \frac{1}{N} \sum_{k=0}^{N-1} H(k) e^{j\frac{2\pi kx}{N}}.$$

N is the length of the sampled signal. (The slide writes “ dx ” after the DFT sum — a typo; the DFT is a plain sum.)

2D DFT of an $M \times N$ image: $F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$ [$M \times N$ complex].

Derivation: DFT of a 4-point signal by hand

$h = [1, 2, 3, 4]$, $N = 4$, $W = e^{-j2\pi/4} = -j$, so $H(k) = \sum_x h(x)(-j)^{kx}$.

$$H(0) = 1 + 2 + 3 + 4 = 10 \quad (\text{DC} = N \times \text{mean})$$

$$H(1) = 1 + 2(-j) + 3(-1) + 4(j) = -2 + 2j$$

$$H(2) = 1 - 2 + 3 - 4 = -2$$

$$H(3) = 1 + 2(j) + 3(-1) + 4(-j) = -2 - 2j = \overline{H(1)} \quad (\text{conjugate symmetry})$$

Magnitudes: 10, $2\sqrt{2}$, 2, $2\sqrt{2}$. Phase of $H(1)$: $\text{atan2}(2, -2) = 135^\circ$.

6.8 The Convolution Theorem

Key Idea: Convolution theorem

The Fourier transform of the convolution of two functions is the product of their Fourier transforms:

$$\mathcal{F}[g * h] = \mathcal{F}[g] \mathcal{F}[h] \implies g * h = \mathcal{F}^{-1}[\mathcal{F}[g] \mathcal{F}[h]].$$

Convolution in the spatial domain = multiplication in the frequency domain.

Derivation: Proof (continuous, 1D)

$$\begin{aligned} \mathcal{F}[g * h](\omega) &= \int \left(\int g(\tau) h(x - \tau) d\tau \right) e^{-j\omega x} dx = \int g(\tau) \left(\int h(x - \tau) e^{-j\omega x} dx \right) d\tau \\ &\stackrel{s=x-\tau}{=} \int g(\tau) \left(\int h(s) e^{-j\omega(s+\tau)} ds \right) d\tau \\ &= \underbrace{\int g(\tau) e^{-j\omega\tau} d\tau}_{G(\omega)} \underbrace{\int h(s) e^{-j\omega s} ds}_{H(\omega)} = G(\omega) H(\omega). \quad \square \end{aligned}$$

For the DFT the same holds for **circular** convolution (indices mod N) — which is why “wrap-around” padding appears naturally, and why you zero-pad to length $\geq N + k - 1$ to get ordi-

nary (linear) convolution via FFT.

6.9 Properties of the Fourier Transform

Property	Signal	Transform
Superposition	$f_1(x) + f_2(x)$	$F_1(\omega) + F_2(\omega)$
Shift	$f(x - x_0)$	$F(\omega) e^{-j\omega x_0}$
Reversal	$f(-x)$	$F^*(\omega)$ (for real f)
Convolution	$f(x) * h(x)$	$F(\omega)H(\omega)$
Correlation	$f(x) \otimes h(x)$	$F(\omega)H^*(\omega)$
Multiplication	$f(x)h(x)$	$F(\omega) * H(\omega)$ (up to $1/2\pi$)
Differentiation	$f'(x)$	$j\omega F(\omega)$
Domain scaling	$f(ax)$	$\frac{1}{ a } F(\omega/a)$
Real images	$f(x) = f^*(x)$	$\Leftrightarrow F(\omega) = F^*(-\omega)$
Parseval	$\sum_x [f(x)]^2$	$= \sum_\omega [F(\omega)]^2$ (up to normalisation)

Intuition: Reading the table

- **Shift** changes only the *phase* ($|e^{-j\omega x_0}| = 1$) — magnitude is shift-invariant, consistent with “phase = where”.
- **Differentiation** multiplies by $j\omega$: it boosts high frequencies $\Rightarrow$ derivative filters are *high-pass* and amplify noise.
- **Scaling**: squeeze in space $\Leftrightarrow$ stretch in frequency. A narrow kernel has a wide spectrum and vice versa (uncertainty principle).
- **Parseval**: energy is the same in both domains.

Derivation: Shift property

$$\int f(x - x_0) e^{-j\omega x} dx \stackrel{s=x-x_0}{=} \int f(s) e^{-j\omega(s+x_0)} ds = e^{-j\omega x_0} F(\omega).$$

Derivation: The Gaussian is its own Fourier transform (up to scale)

For $g(x) = e^{-x^2/2\sigma^2}$, completing the square in the exponent,

$$G(\omega) = \int e^{-\frac{x^2}{2\sigma^2} - j\omega x} dx = \int e^{-\frac{(x+j\sigma^2\omega)^2}{2\sigma^2}} dx \cdot e^{-\frac{\sigma^2\omega^2}{2}} = \sqrt{2\pi}\sigma e^{-\frac{\omega^2}{2\sigma^2}}.$$

So a Gaussian of width σ in space is a Gaussian of width $1/\sigma$ in frequency: a **smooth low-pass filter with no ripples**. By contrast, the box of width w transforms to sinc: $\int_{-w/2}^{w/2} e^{-j\omega x} dx = \frac{2 \sin(\omega w/2)}{\omega}$ — it has negative lobes and lets some high frequencies through (ringing). This is the frequency-domain reason the Gaussian beats the box.

6.10 The Fast Fourier Transform (FFT)

- Computing the DFT of N numbers directly: each of N outputs is a sum of N terms $\Rightarrow \propto N^2$ operations.

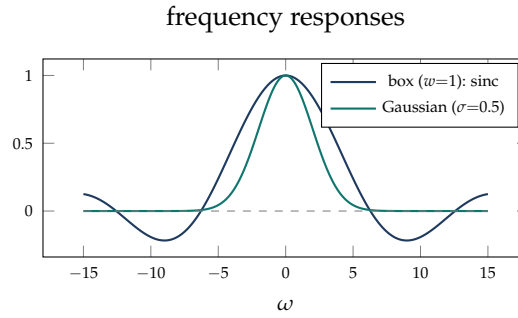


Figure 6.9: The box filter's sinc response has side lobes (it passes some high frequencies, with sign flips); the Gaussian decays smoothly.

- The **FFT** reduces this to $\propto N \log N$. It is a **recursive divide-and-conquer** algorithm for computing the DFT.
- Applications: fast convolution, correlation (and so template matching), filtering, compression.

Derivation: Radix-2 Cooley–Tukey in one line

Split into even and odd samples (N a power of 2, $W_N = e^{-j2\pi/N}$):

$$H(k) = \underbrace{\sum_{m=0}^{N/2-1} h(2m) W_{N/2}^{mk}}_{E(k): \text{DFT of evens}} + W_N^k \underbrace{\sum_{m=0}^{N/2-1} h(2m+1) W_{N/2}^{mk}}_{O(k): \text{DFT of odds}}, \quad H(k + \frac{N}{2}) = E(k) - W_N^k O(k).$$

Two half-size DFTs plus $O(N)$ “butterfly” combinations: $T(N) = 2T(N/2) + O(N) \Rightarrow T(N) = O(N \log N)$. For $N = 1024$: $N^2 \approx 10^6$ vs. $N \log_2 N \approx 10^4$ — about $100\times$ faster.

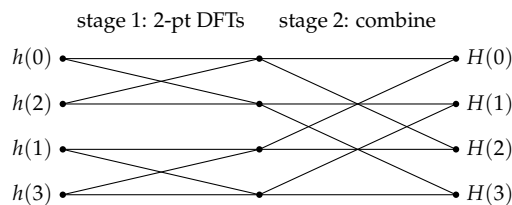


Figure 6.10: 4-point FFT butterfly: $\log_2 4 = 2$ stages, each with $O(N)$ work. Inputs are in bit-reversed order.

6.11 Filtering in the Spatial vs. Frequency Domain

6.11.1 Low-pass and high-pass filters

- **Low-pass filters:** allow low frequencies through, block high frequencies. Example: **Gaussian filter** (blur).
- **High-pass filters:** allow high frequencies through, block low frequencies. Example: **edge filter**.
- **Band-pass:** keep a range (e.g. difference of Gaussians).

A high-pass can be built from a low-pass: $f_{\text{high}} = f - f * G_\sigma = f * (\delta - G_\sigma)$; *unsharp masking* is $f + \alpha (f - f * G_\sigma)$.

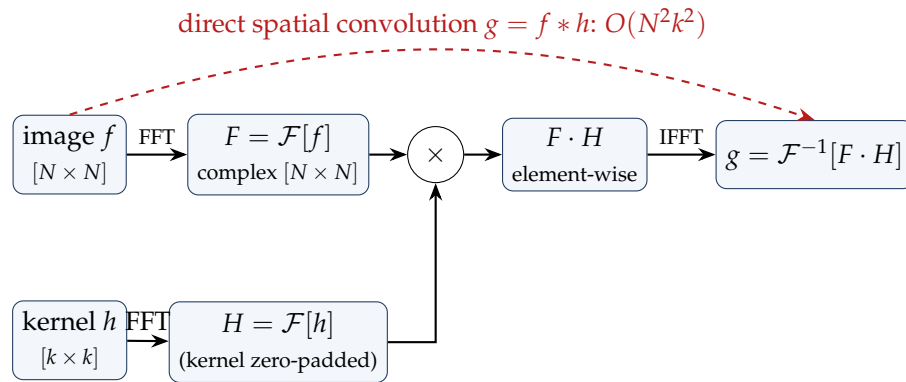


Figure 6.11: Two routes to the same result. The frequency route costs $O(N^2 \log N)$ (see Module 7 for the full cost analysis).

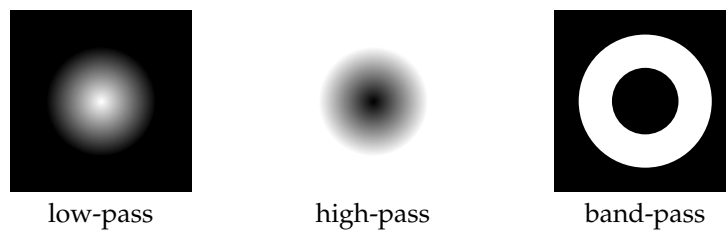


Figure 6.12: Frequency-domain masks (white = pass). Centre = low frequencies.

6.12 Hybrid Images

Oliva, Torralba & Schyns (SIGGRAPH 2006): combine the **low frequencies of one image** with the **high frequencies of another**,

$$\text{hybrid} = G_{\sigma_1} * I_1 + (I_2 - G_{\sigma_2} * I_2).$$

Up close (high frequencies visible) you see I_2 ; from far away (your visual system effectively low-passes/sub-samples) you see I_1 . A vivid demonstration that perception is multi-scale.

6.13 Exercise: Match Spatial Image to Fourier Magnitude

The slide shows five spectra (1–5) and five images (A–E). The answer (given at the start of the next lecture):

Spectrum	Image	Reasoning
1	D	A single blurry white blob is smooth everywhere $\Rightarrow$ only low frequencies (a small central blob; FT of a Gaussian-ish blob is a Gaussian-ish blob).
2	B	Flowers : rich texture in all directions $\Rightarrow$ broad, isotropic spread of energy.
3	A	Vertical bars (a few dark/light rectangles): variation only along $x \Rightarrow$ energy along the horizontal frequency axis; the two lobes are the sinc-like spectrum of rectangular bars.
4	C	Night scene dominated by long straight (mostly horizontal) structures and bright lights $\Rightarrow$ a strong streak along one frequency axis (perpendicular to those structures).
5	E	Seascape with a strong horizon and some vertical structure $\Rightarrow$ a cross-shaped spectrum.

Exam Tip: Rules for this kind of question

(1) Edges/lines in the image produce streaks **perpendicular** to them in the spectrum. (2) Smooth images $\Rightarrow$ compact central blob; textured/noisy $\Rightarrow$ spread-out spectrum. (3) Periodic patterns $\Rightarrow$ isolated dots. (4) The spectrum is always point-symmetric about the centre for real images.

Readings: Szeliski §3.4; thefouriertransform.com; BetterExplained's interactive guide to the FT.

Questions: what cost improvement does the convolution theorem give? (Answered in §7.1.) Complete the matching exercise (above).

Module Summary: §1.6

Correlation lives on as template matching via NCC (brightness-invariant). Non-linear filters: median (removes salt-and-pepper, preserves edges, poor for Gaussian noise), bilateral (domain $\times$ range Gaussian weights; edge-preserving smoothing). Fourier: any signal = weighted sum of sinusoids $A \sin(\omega x + \phi)$; square wave = odd harmonics $1/k$. Image spectra: centre = DC/low frequencies, symmetric for real images; magnitude = how much, phase = where (phase dominates perception). FT/DFT formulas; convolution theorem $\mathcal{F}[g * h] = \mathcal{F}[g]\mathcal{F}[h]$; FFT $O(N \log N)$; Gaussian = low-pass (its FT is Gaussian), edge = high-pass; hybrid images mix low of one image with high of another.

Image Sampling and Interpolation

7.1 Cost Improvement Using the Convolution Theorem

Recall $\mathcal{F}[g * h] = \mathcal{F}[g]\mathcal{F}[h]$, so $g * h = \mathcal{F}^{-1}[\mathcal{F}[g]\mathcal{F}[h]]$.

Key Idea: Complexity comparison

Image $N \times N$, kernel $k \times k$.

- **Spatial convolution:** k^2 multiply-adds at each of N^2 pixels $\Rightarrow O(N^2 k^2)$.
- **Via Fourier:** FFT of image $O(N^2 \log N^2)$ + FFT of kernel $O(k^2 \log k^2)$ + element-wise product $O(N^2)$ + inverse FFT $O(N^2 \log N^2)$ $\Rightarrow \approx O(N^2 \log N^2 + k^2 \log k^2)$ in total (other terms additive).

Derivation: Where the $N^2 \log N^2$ comes from

A 2D DFT is separable: do a 1D FFT on each of the N rows ($N \cdot N \log N$) then on each of the N columns ($N \cdot N \log N$): $2N^2 \log N = N^2 \log N^2$. (In practice the kernel is zero-padded to $N \times N$ before the FFT so that the spectra can be multiplied element-wise; the slide's $k^2 \log k^2$ term counts the kernel's intrinsic size.)

Derivation: When is the FFT route worth it?

Ignoring constants, FFT wins when $k^2 > \log_2 N^2 = 2 \log_2 N$. For $N = 1024$: $2 \log_2 N = 20$, so kernels larger than about 5×5 benefit. Concretely, $N = 1024$, $k = 31$: spatial $\approx N^2 k^2 = 1.05 \times 10^6 \times 961 \approx 1.0 \times 10^9$; FFT $\approx 3 \times N^2 \log_2 N^2 \approx 3 \times 1.05 \times 10^6 \times 20 \approx 6.3 \times 10^7$ — roughly $16\times$ cheaper. For tiny 3×3 kernels (CNNs!) direct convolution is usually faster; real FFT constants are also larger than 1.

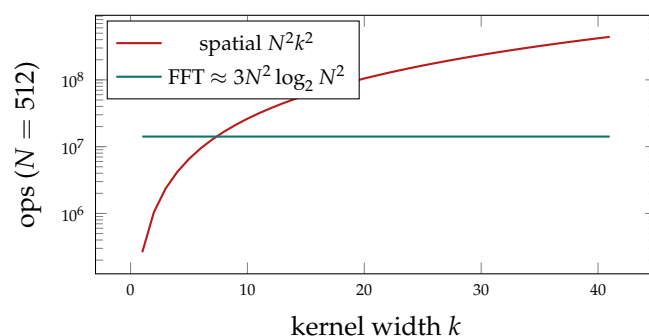


Figure 7.1: Spatial cost grows with k^2 ; the FFT route is (almost) independent of k . The crossover is at a modest kernel size.

Matching exercise answer (from Module 6): $1 \rightarrow D$, $2 \rightarrow B$, $3 \rightarrow A$, $4 \rightarrow C$, $5 \rightarrow E$.

7.2 What Sense Does a Low-Resolution Image Make to Us?

A photo of the Charminar sub-sampled and zoomed is still recognisable. Clues from human perception:

- Early processing in humans filters for various **orientations and scales of frequency**.
- Perceptual cues in the **mid-high frequencies** dominate perception.
- When we see an image from far away, we are effectively **sub-sampling** it.

7.3 Sub-sampling

Naive $\frac{1}{2}$ sub-sampling: throw away every other row and column. Repeating gives $\frac{1}{4}, \frac{1}{8}, \dots$ size images. Displayed at the same size, the $\frac{1}{4}$ and $\frac{1}{8}$ images look “crufty” — jagged, with false patterns. *What’s happening? Aliasing.*

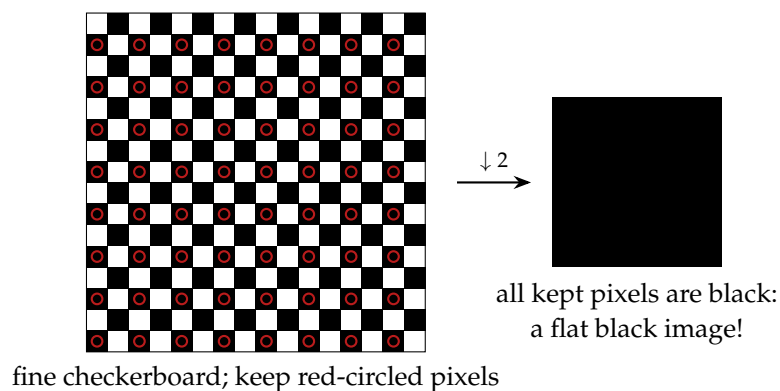


Figure 7.2: Extreme aliasing: sub-sampling a checkerboard at the pixel frequency by 2 keeps only one colour. The finest detail is not just lost — it turns into a completely wrong (here constant) signal.

7.4 Aliasing

Key Idea: Definition

Aliasing occurs when the sampling rate is not high enough to capture the amount of detail in the image. To do sampling right, we need to understand the structure (frequency content) of the signal. The minimum sampling rate is the **Nyquist rate**. **Shannon’s sampling theorem:**

$$f_s \geq 2f_{\max}.$$

Derivation: Where the alias frequency comes from

Sample $x(t) = \cos(2\pi ft)$ at $t = n/f_s$: $x[n] = \cos(2\pi fn/f_s)$. Since $\cos$ is 2π -periodic, for any integer m ,

$$\cos\left(2\pi \frac{f}{f_s} n\right) = \cos\left(2\pi \frac{f - mf_s}{f_s} n\right).$$

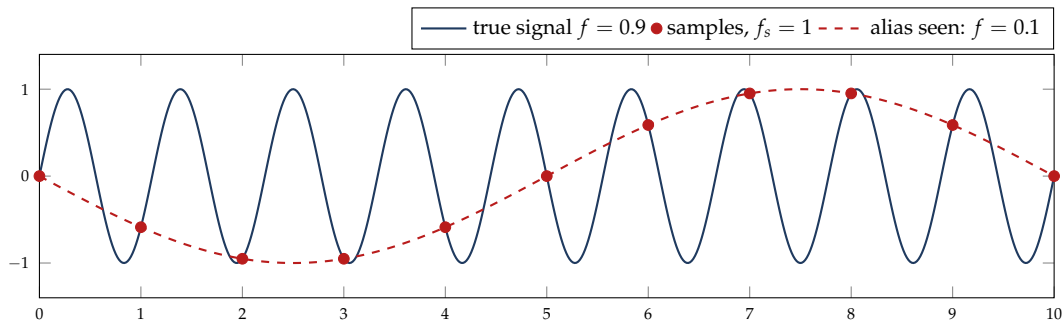


Figure 7.3: Sampling a 0.9 Hz sinusoid at 1 sample/s (far below $2 \times 0.9 = 1.8$) produces samples that lie exactly on a 0.1 Hz sinusoid. The reconstruction “sees” the slow alias.

So frequencies f and $f - mf_s$ produce *identical samples*. The reconstruction picks the one in $[-f_s/2, f_s/2]$:

$$f_{\text{alias}} = |f - f_s \cdot \text{round}(f/f_s)|$$

Example above: $f = 0.9$, $f_s = 1 \Rightarrow |0.9 - 1| = 0.1$. Only if $f < f_s/2$ (i.e. $f_s > 2f$) is $f_{\text{alias}} = f$ — that is Shannon’s condition.

Derivation: Frequency-domain view of sampling (why $2f_{\text{max}}$)

Sampling multiplies $f(x)$ by an impulse train of spacing $T = 1/f_s$. By the (multiplication $\leftrightarrow$ convolution) property, the spectrum becomes the original spectrum **replicated** every f_s : $F_s(\omega) \propto \sum_m F(\omega - mf_s)$. If F is band-limited to $[-f_{\text{max}}, f_{\text{max}}]$, the copies do not overlap iff $f_s - f_{\text{max}} \geq f_{\text{max}}$, i.e. $f_s \geq 2f_{\text{max}}$. Overlapping copies (aliasing) cannot be separated afterwards.

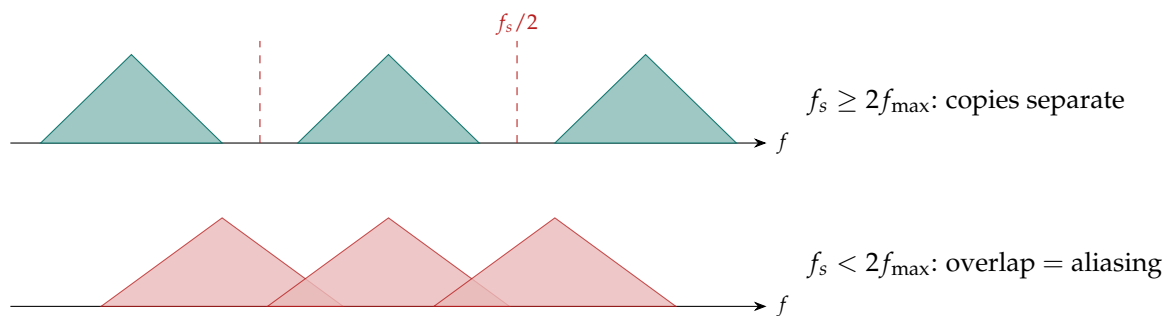


Figure 7.4: Sampling replicates the spectrum at multiples of f_s .

7.4.1 Aliasing: problems in practice

- **Images:** a striped shirt’s pattern looks weird on screen (moiré) — the stripe frequency exceeds the display/camera Nyquist limit.
- **Video — wagon-wheel effect:** wheels appear to spin slowly or even *backwards* at high speed. Temporal sampling (frame rate) is too low for the rotation frequency.
- **Graphics:** checkerboards disintegrate in ray tracing (distant squares are smaller than a pixel).
- **Nyquist limit 2D example:** a zone-plate/concentric-ring pattern sub-sampled shows false rings where the local frequency exceeds half the sampling rate.

Derivation: Wagon-wheel effect, quantitatively

A wheel with 8 identical spokes (symmetry period 45°) is filmed at 24 fps. Suppose it turns 40° per frame. Because spokes are indistinguishable, the eye matches each spoke to the *nearest* spoke position in the next frame: $40^\circ \equiv 40^\circ - 45^\circ = -5^\circ$. The wheel appears to rotate **backwards** at $5^\circ \times 24 = 120^\circ/\text{s}$. Same formula as above with angle instead of time.

7.5 Anti-Aliasing

Two remedies: (1) **sample more often** (not always possible), or (2) **remove the high frequencies before sampling** — blur, then sub-sample.

Key Idea: Gaussian pre-filtering

Filter with a Gaussian (a low-pass filter) *then* sub-sample. Pre-filtering removes frequencies above the new Nyquist limit, so the result is blurry but free of false patterns. “Blurry is better than wrong.” Repeating “blur + $\downarrow 2$ ” builds a **Gaussian pyramid**.

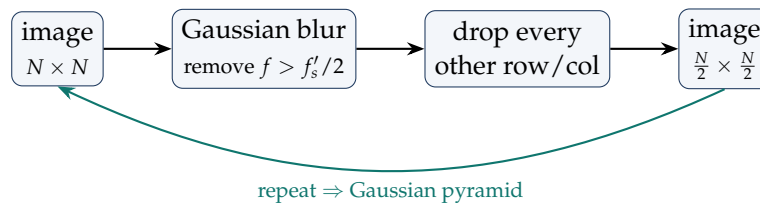


Figure 7.5: Subsampling with Gaussian pre-filtering.

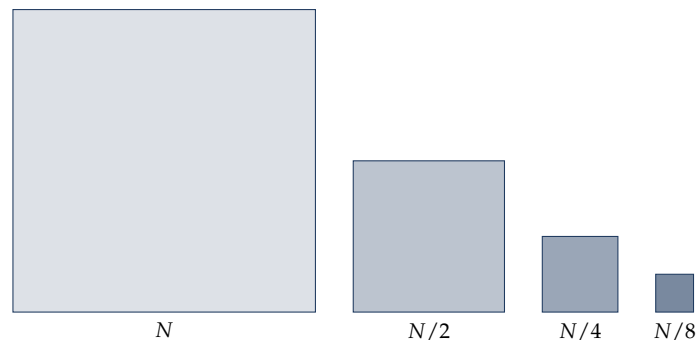


Figure 7.6: A Gaussian pyramid: each level is a blurred, half-size copy of the previous one. Total storage $\leq \frac{4}{3}$ of the original ($1 + \frac{1}{4} + \frac{1}{16} + \dots$).

7.6 Upsampling and Interpolation

Upsampling: how to go from a small image to a $10\times$ larger one? **Interpolation.** Simplest method: repeat each row and column 10 times — **nearest-neighbour interpolation** (blocky result).

7.6.1 Interpolation as reconstruction

Recall how a digital image is formed:

$$F[x, y] = \text{quantize}\{f(xd, yd)\}.$$

It is a discrete point-sampling of a continuous function. If we could somehow reconstruct the original f , any new image could be generated at any resolution and scale.

What if we don't know f ? Guess an approximation — done in a principled way via **filtering**:

1. Convert F to a continuous function (impulses at the sample positions):

$$f_F(x) = \begin{cases} F(\frac{x}{d}) & \text{if } \frac{x}{d} \text{ is an integer} \\ 0 & \text{otherwise.} \end{cases}$$

2. Reconstruct by convolving with a reconstruction kernel h :

$$\hat{f} = h * f_F.$$

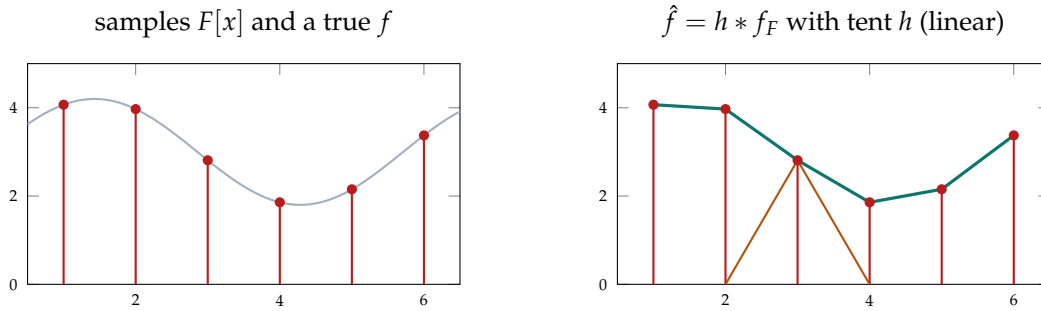


Figure 7.7: Reconstruction: place a copy of the kernel h (amber, one shown) at each sample, scaled by the sample value; their sum is the interpolant (teal).

7.6.2 Interpolation as convolution

To interpolate (upsample) an image by rate r , convolve with an interpolation kernel:

$$g(i, j) = \sum_{k, l} f(k, l) h(i - rk, j - rl)$$

Similar to discrete convolution $g = f * h \Rightarrow g(i, j) = \sum_{k, l} f(k, l) h(i - k, j - l)$, except that k, l in $h(\cdot)$ are replaced by rk, rl : input sample (k, l) sits at output position (rk, rl) .

Derivation: Checking it with nearest-neighbour, $r = 2$, 1D

$f = [a, b, c]$ at positions 0, 2, 4 of the output. Box kernel $h(x) = 1$ for $x \in \{0, 1\}$ (a 2-wide box), 0 otherwise. $g(i) = \sum_k f(k) h(i - 2k)$: $g(0) = a, g(1) = a, g(2) = b, g(3) = b, g(4) = c, g(5) = c \Rightarrow [a, a, b, b, c, c]$ — each value repeated r times. With the tent $h(x) = \max(0, 1 - |x|/2)$: $g(1) = \frac{1}{2}a + \frac{1}{2}b, g(3) = \frac{1}{2}b + \frac{1}{2}c \Rightarrow [a, \frac{a+b}{2}, b, \frac{b+c}{2}, c]$ — linear interpolation.

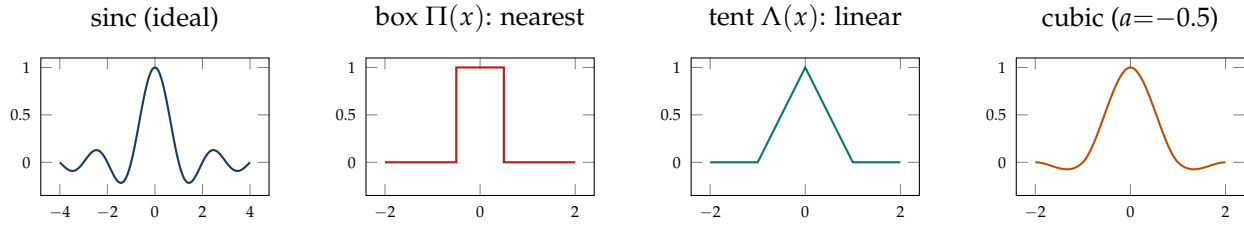


Figure 7.8: Reconstruction kernels (Curless). Sinc is ideal (perfect for band-limited signals) but infinitely wide; box gives nearest-neighbour (blocky); tent gives linear (piecewise-linear, continuous); cubic/B-spline/Gaussian give smoother results.

7.6.3 Types of interpolation (reconstruction kernels)

Method	Kernel	Result
Ideal	$\text{sinc}(x) = \frac{\sin \pi x}{\pi x}$	Perfect for band-limited signals (it is the ideal low-pass filter in frequency); infinite support, ringing in practice.
Nearest neighbour	box, width 1	Piecewise constant, blocky, discontinuous; fastest. Neighbours used: 1 (1×1 in 2D).
Linear / bilinear	tent $\Lambda(x) = \max(0, 1 - x)$	Piecewise linear, continuous; slight blur. 2×2 neighbours in 2D.
Cubic / bicubic	piecewise cubic (Keys, $a = -0.5$)	Smooth, sharper than bilinear; slight overshoot. 4×4 neighbours.
Gaussian / B-spline	Gaussian, cubic B-spline	Very smooth, but blurs (not interpolating for Gaussian: does not pass exactly through samples).

Derivation: Bilinear interpolation formula

To estimate $f(x, y)$ with $x = i + \alpha$, $y = j + \beta$, $\alpha, \beta \in [0, 1]$, from the four surrounding pixels, interpolate linearly along x twice then along y :

$$\begin{aligned}
 f(x, j) &\approx (1 - \alpha)f(i, j) + \alpha f(i + 1, j), & f(x, j + 1) &\approx (1 - \alpha)f(i, j + 1) + \alpha f(i + 1, j + 1), \\
 f(x, y) &\approx (1 - \beta)f(x, j) + \beta f(x, j + 1) \\
 &= (1 - \alpha)(1 - \beta)f_{00} + \alpha(1 - \beta)f_{10} + (1 - \alpha)\beta f_{01} + \alpha\beta f_{11}.
 \end{aligned}$$

The four weights sum to 1 and equal the *area of the opposite sub-rectangle*. This is exactly convolution with the separable 2D tent $h(x, y) = \Lambda(x)\Lambda(y)$.

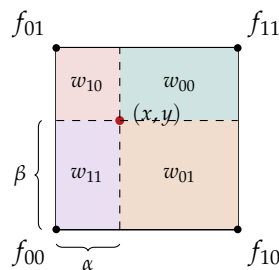


Figure 7.9: Bilinear weights: each corner's weight is the area of the sub-rectangle *opposite* it, e.g. $w_{00} = (1 - \alpha)(1 - \beta)$. (Here the y axis points up; f_{01} is the corner at $(i, j + 1)$.)

Key Idea: Cubic convolution kernel (Keys)

$$h(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1, & |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a, & 1 < |x| < 2 \\ 0, & \text{otherwise,} \end{cases} \quad a = -0.5.$$

It satisfies $h(0) = 1$, $h(\pm 1) = h(\pm 2) = 0$ (so it *interpolates*: passes through the samples) and is C^1 . The negative lobes give a mild sharpening (and overshoot).

Examples (slide). Upsampling the same image with nearest-neighbour (blocky), bilinear (smooth but soft) and bicubic (smooth and sharper) — bicubic is the default in most editors.

7.7 Interpolation and Decimation

Interpolation (upsample by r) $g(i, j) = \sum_{k, l} f(k, l) h(i - rk, j - rl)$

Decimation (downsample by r) $g(i, j) = \sum_{k, l} f(k, l) h\left(i - \frac{k}{r}, j - \frac{l}{r}\right)$ (as written on the slide)

Pitfall / Careful: A cleaner way to write decimation

The standard form (Szeliski eq. 3.65) is $g(i, j) = \sum_{k, l} f(k, l) h(ri - k, rj - l)$: output pixel i sits at input position ri ; blur with h around it (the low-pass anti-aliasing filter), keep only every r -th position. Both forms express “convolve with a smoothing kernel, then keep every r -th sample”; just make sure the kernel h is a low-pass filter with cut-off at the new Nyquist frequency.

Intuition: Tie-in to deep learning

Strided convolution and pooling are decimation; transposed convolution (“deconvolution”) and bilinear upsampling in U-Nets/FCNs are interpolation. Naive strided sub-sampling in CNNs aliases, which is one reason CNNs are not perfectly shift-invariant — “anti-aliased CNNs” insert a blur before striding, exactly the Gaussian pre-filtering of this lecture.

Readings: Szeliski Ch. 3 (§3.5.1–3.5.2); Forsyth & Ponce Ch. 7 (§7.4).

Module Summary: §1.7

FFT convolution costs $O(N^2 \log N^2 + k^2 \log k^2)$ vs. $O(N^2 k^2)$ spatially. Naive sub-sampling aliases when $f_s < 2f_{\max}$: high frequencies appear as false low ones ($f_{\text{alias}} = |f - f_s \text{round}(f/f_s)|$) — moiré, wagon wheels, broken checkerboards. Anti-alias by Gaussian pre-filtering before sub-sampling (Gaussian pyramid). Upsampling = reconstruction $\hat{f} = h * f_F$; kernels: sinc (ideal), box (nearest), tent (bilinear), cubic (bicubic). Interpolation: $g = \sum f(k, l) h(i - rk, j - rl)$; decimation: blur then keep every r -th sample.

PRACTICE A

Numerical Problems with Worked Solutions

Problems are grouped by module. Try each before reading the solution. All arithmetic has been checked numerically.

A.1 Modules 1–3: Images, Image Formation, Sensors, Compression

Problem 1 (Data size)

(a) How many numbers are in one 1920×1080 RGB frame? (b) How many gigabytes does one minute of uncompressed 8-bit 30 fps video take? (c) Russell Kirsch's first digital image was 176×176 . How many pixels, and how many bits if stored as 1 bit/pixel (black/white)?

Solution

- (a) $1920 \times 1080 \times 3 = 6,220,800 \approx 6.2 \times 10^6$ values.
(b) 1 byte each: $6,220,800 \times 30 \text{ fps} \times 60 \text{ s} = 1.12 \times 10^{10}$ bytes ≈ 11.2 GB. This is why compression and efficient architectures matter ("high-dimensional data").
(c) $176^2 = 30,976$ pixels $\Rightarrow 30,976$ bits = 3,872 bytes.

Problem 2 (Lambertian shading)

A Lambertian surface has albedo $\rho = 0.6$ and is lit by a distant source of irradiance $E_0 = 200$. Find the observed intensity when the angle between the surface normal and the light direction is $\theta = 0^\circ, 60^\circ, 90^\circ, 120^\circ$. Does the answer change if the camera moves?

Solution

$I = \rho E_0 \max(0, \cos \theta) = 120 \max(0, \cos \theta)$.
 $\theta = 0^\circ$: 120. 60° : $120 \times 0.5 = 60$. 90° : 0 (grazing light). 120° : $\cos < 0 \Rightarrow 0$ (surface faces away; self-shadow).
Moving the camera does *not* change I : diffuse reflection is independent of viewing direction. (Only a specular component would change.)

Problem 3 (Two light sources — linearity of light)

A Lambertian patch ($\rho = 0.5$) with normal $\hat{\mathbf{n}} = (0, 0, 1)$ is lit by two sources of irradiance 100 each: one along $(0, 0, 1)$, one at 45° to the normal. Find I .

Solution

Light adds linearly: $I = \rho \sum_s E_s \cos \theta_s = 0.5 (100 \cdot 1 + 100 \cos 45^\circ) = 0.5 (100 + 70.71) = 85.36$.

Problem 4 (Bayer sensor and demosaicing)

(a) A 4000×3000 sensor uses a Bayer filter. How many photosites are green, red and blue? (b) At a red photosite the four edge-neighbours have green values 120, 130, 110, 140 and the four diagonal neighbours have blue values 40, 50, 60, 50. Estimate (R, G, B) with bilinear demosaicing if the red reading is 200.

Solution

(a) Total 12×10^6 . Green 50% = 6,000,000; red 25% = 3,000,000; blue 25% = 3,000,000.
 (b) $G = \frac{120+130+110+140}{4} = 125$, $B = \frac{40+50+60+50}{4} = 50$, $R = 200$ (measured). Pixel $\approx (200, 125, 50)$.

Problem 5 (Fill factor, bit depth)

A sensor has pitch $4 \mu\text{m} \times 4 \mu\text{m}$ and a $3 \mu\text{m} \times 2.4 \mu\text{m}$ photodiode per cell. (a) Fill factor? (b) A 12-bit ADC maps 0–1 V. Quantisation step and quantisation-noise std? (c) How much does SNR improve going from 10-bit to 12-bit?

Solution

(a) $\frac{3 \times 2.4}{4 \times 4} = \frac{7.2}{16} = 45\%$.
 (b) $\Delta = 1/2^{12} = 244 \mu\text{V}$. Noise std = $\Delta/\sqrt{12} \approx 70.5 \mu\text{V}$.
 (c) 2 extra bits $\times 6.02 \text{ dB} \approx 12 \text{ dB}$.

Problem 6 (PSNR)

Original $I = \begin{bmatrix} 100 & 120 \\ 140 & 160 \end{bmatrix}$, compressed $\hat{I} = \begin{bmatrix} 102 & 118 \\ 141 & 165 \end{bmatrix}$ (8-bit). Compute MSE and PSNR. If the error at every pixel were halved, what is the new PSNR?

Solution

Errors: $-2, 2, -1, -5$. Squares: 4, 4, 1, 25. MSE = $34/4 = 8.5$.
 PSNR = $10 \log_{10} \frac{255^2}{8.5} = 10 \log_{10}(7650) = 38.84 \text{ dB}$.
 Halving errors $\Rightarrow \text{MSE}/4 \Rightarrow \text{PSNR} + 10 \log_{10} 4 = +6.02 \text{ dB} \Rightarrow 44.86 \text{ dB}$.

Problem 7 (Colour conversion)

Convert the pixel $(R, G, B) = (200, 100, 50)$ to (a) CMY in $[0, 1]$ and (b) YCbCr (BT.601 full range).

Solution

(a) Normalise: $(0.784, 0.392, 0.196)$. $C = 1 - R = 0.216$, $M = 0.608$, $Y = 0.804$.
 (b) $Y = 0.299(200) + 0.587(100) + 0.114(50) = 59.8 + 58.7 + 5.7 = 124.2$.
 $C_b = 128 - 0.1687(200) - 0.3313(100) + 0.5(50) = 128 - 33.75 - 33.13 + 25 = 86.1$.
 $C_r = 128 + 0.5(200) - 0.4187(100) - 0.0813(50) = 128 + 100 - 41.87 - 4.07 = 182.1$.
 $C_r > 128$ (reddish), $C_b < 128$ (not bluish) — consistent with an orange pixel.

A.2 Module 4: Point Operations

Problem 8 (Negative and linear stretching)

$I = \begin{bmatrix} 50 & 60 & 70 \\ 80 & 90 & 100 \\ 110 & 120 & 130 \end{bmatrix}$, 8-bit ($I_{\min} = 0$, $I_{\max} = 255$). (a) Contrast-reverse it. (b) Linearly stretch it to $[0, 255]$.

Solution

$$(a) \hat{I} = 255 - I = \begin{bmatrix} 205 & 195 & 185 \\ 175 & 165 & 155 \\ 145 & 135 & 125 \end{bmatrix}.$$

(b) $\min = 50$, $\max = 130$, $\text{scale} = 255/80 = 3.1875$: $\hat{I} = (I - 50) \times 3.1875 = \begin{bmatrix} 0 & 31.9 & 63.8 \\ 95.6 & 127.5 & 159.4 \\ 191.3 & 223.1 & 255 \end{bmatrix} \approx \begin{bmatrix} 0 & 32 & 64 \\ 96 & 128 & 159 \\ 191 & 223 & 255 \end{bmatrix}$. Neighbouring grey levels, originally 10 apart, are now ≈ 32 apart.

Problem 9 (Histogram equalisation)

A 3-bit image (levels 0..7) has 50 pixels with histogram $h = [0, 2, 10, 20, 14, 4, 0, 0]$. Equalise it.

Solution

l	0	1	2	3	4	5	6	7
$h(l)$	0	2	10	20	14	4	0	0
cumulative	0	2	12	32	46	50	50	50
c_l	0	0.04	0.24	0.64	0.92	1	1	1
$7c_l$	0	0.28	1.68	4.48	6.44	7	7	7
new level	0	0	2	4	6	7	7	7

Output histogram: level 0: 2, level 2: 10, level 4: 20, level 6: 14, level 7: 4. The data, which occupied levels 1–5, now spans 0–7 and the big bins are spread apart (not perfectly flat, because discrete bins cannot be split).

Problem 10 (Temporal averaging)

A still scene is photographed with sensor noise std $\sigma = 12$ grey levels. How many frames must be averaged to bring the noise std down to 3?

Solution

$$\sigma/\sqrt{K} \leq 3 \Rightarrow \sqrt{K} \geq 4 \Rightarrow K \geq 16 \text{ frames.}$$

Problem 11 (Moving average on the slide's grid)

Using the 10×10 image of §4.7 (a block of 90's at rows 2–6, columns 3–7 with a 0 at (5,4), and an isolated 90 at (8,2)), compute the 3×3 moving average at (a) (3,3), (b) (5,5), (c) (7,2).

Solution

(a) Rows 2–4, cols 2–4: col 2 all 0, cols 3–4 all 90 $\Rightarrow 6 \times 90/9 = 60$.

(b) Rows 4–6, cols 4–6: nine cells, one is the 0 at (5,4) $\Rightarrow 8 \times 90/9 = 80$.

(c) Rows 6–8, cols 1–3: (6,3) = 90, (8,2) = 90, rest 0 $\Rightarrow 180/9 = 20$. These match the slide's output grid (values 60, 80 and 20 at those positions).

A.3 Module 5: Correlation, Convolution, Separability**Problem 12 (Correlation vs. convolution by hand)**

$$I = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 0 & 1 & 2 & 3 & 1 \\ 1 & 0 & 1 & 2 & 2 \\ 2 & 1 & 0 & 1 & 3 \\ 1 & 2 & 1 & 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}.$$

Compute the “valid” cross-correlation $H \otimes I$ and convolution $H * I$.

Solution

Output size: $(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$.

Top-left correlation: patch $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix} \cdot H = 1(1) + 3(-1) + 0(2) + 2(-2) + 1(1) + 1(-1) =$

$1 - 3 - 4 + 1 - 1 = -6$. Doing all nine positions:

$$H \otimes I = \begin{bmatrix} -6 & -4 & 3 \\ 0 & -6 & -4 \\ 4 & 0 & -7 \end{bmatrix}, \quad H * I = \begin{bmatrix} 6 & 4 & -3 \\ 0 & 6 & 4 \\ -4 & 0 & 7 \end{bmatrix}.$$

Convolution flips H by 180° , giving $\tilde{H} = -H$ here (this kernel is anti-symmetric), so $H * I = -(H \otimes I)$. For a symmetric kernel the two results would be identical.

Problem 13 (Impulse response)

Filter the impulse image δ (1 at the centre of a 5×5 zero image) with $H = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ using (a) correlation, (b) convolution. Show the central 3×3 .

Solution

(a) $G(i, j) = H(-i, -j)$: $\begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$ (180° rotated). (b) $G(i, j) = H(i, j)$: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ (kernel reproduced).

Problem 14 (Output size)

Input 224×224 . Find output size for (a) $k = 3, p = 0, s = 1$; (b) $k = 3, p = 1, s = 1$; (c) $k = 7, p = 3, s = 2$; (d) $k = 5, p = 0, s = 3$. (e) What padding keeps size with $k = 9, s = 1$?

Solution

$N_{\text{out}} = \lfloor (N + 2p - k)/s \rfloor + 1$. (a) $224 - 3 + 1 = 222$. (b) 224. (c) $\lfloor (224 + 6 - 7)/2 \rfloor + 1 = \lfloor 111.5 \rfloor + 1 = 112$. (d) $\lfloor 219/3 \rfloor + 1 = 73 + 1 = 74$. (e) $p = (9 - 1)/2 = 4$.

Problem 15 (Separability via rank/SVD)

Which kernels are separable? Factor those that are. (a) $K_1 = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$, (b) $K_2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ (Laplacian).

Solution

(a) Every row is a multiple of $[1, 2, 3]$ (rows $\times 1, \times 2, \times 3$) $\Rightarrow$ rank 1: $K_1 = [1, 2, 3]^\top [1, 2, 3]$. Singular values: 14, 0, 0. Separable.
 (b) Rows $[0, 1, 0]$ and $[1, -4, 1]$ are not proportional $\Rightarrow$ rank ≥ 2 . Singular values $\approx 4.449, 0.449, 0 \Rightarrow$ rank 2: **not separable**. (It is, however, a *sum* of two separable kernels: $[0, 1, 0]^\top [1, -2, 1] + [1, -2, 1]^\top [0, 1, 0]$ — i.e. $\partial_{xx} + \partial_{yy}$.)

Problem 16 (Separable speed-up)

A 7×7 Gaussian is applied to a 1000×1000 image. Count multiplications for direct 2D vs. separable filtering.

Solution

Direct: $10^6 \times 49 = 4.9 \times 10^7$. Separable: $10^6 \times (7 + 7) = 1.4 \times 10^7$. Speed-up $49/14 = 3.5 \times (= k/2)$.

Problem 17 (Sampled Gaussian kernel)

Build a 3×3 Gaussian kernel with $\sigma = 1$ and normalise it.

Solution

Unnormalised $e^{-(u^2+v^2)/2}$: centre $e^0 = 1$; edge neighbours $e^{-1/2} = 0.6065$; corners $e^{-1} = 0.3679$. Sum = $1 + 4(0.6065) + 4(0.3679) = 4.8977$.

$$H = \begin{bmatrix} 0.0751 & 0.1238 & 0.0751 \\ 0.1238 & 0.2042 & 0.1238 \\ 0.0751 & 0.1238 & 0.0751 \end{bmatrix}.$$

(Compare the binomial $\frac{1}{16}[1, 2, 1; 2, 4, 2; 1, 2, 1] = [0.0625, 0.125, 0.0625; \dots; 0.25 \text{ centre}]$ — a close integer approximation.) Note 3×3 truncates at 1σ ; the 3σ rule would ask for 7×7 .

Problem 18 (Noise reduction by filters)

Image noise is i.i.d. with std 10. What is the output noise std after (a) a 5×5 box, (b) the 3×3 binomial kernel?

Solution

Output variance = $\sigma^2 \sum H^2$. (a) $\sum H^2 = 25 \times \frac{1}{625} = \frac{1}{25} \Rightarrow \text{var} = 4$, std = 2. (b) $\sum H^2 = \frac{36}{256} = 0.1406 \Rightarrow \text{var} = 14.06$, std ≈ 3.75 . The larger box reduces noise more — at the cost of more blur.

A.4 Module 6: Non-linear Filters and Fourier**Problem 19 (Median vs. mean)**

Row $x = [10, 12, 11, 200, 13, 12, 0, 11, 12]$. Apply a width-3 median and a width-3 mean at interior positions 1–7.

Solution

Median: $[11, 12, 13, 13, 12, 11, 11]$ — both the spike (200) and the dropout (0) vanish.

Mean: $[11.0, 74.3, 74.7, 75.0, 8.3, 7.7, 7.7]$ — the spike is smeared over three pixels and the dropout pulls values down.

Problem 20 (Bilateral weights)

Centre pixel value 50. Neighbours: one at squared distance 1 with value 52, one at squared distance 1 with value 150 (across an edge), one at squared distance 2 with value 48. With $\sigma_d = 1$, $\sigma_r = 10$, compute the bilateral output (including the centre itself), and compare with a plain Gaussian (domain-only) average.

Solution

$$w = \exp\left(-\frac{d^2}{2} - \frac{(v-50)^2}{200}\right).$$

Centre: $w = 1$. Value 52: $e^{-0.5-0.02} = 0.5945$. Value 150: $e^{-0.5-50} \approx 0$. Value 48: $e^{-1-0.02} = 0.3606$.

Bilateral: $\frac{1(50)+0.5945(52)+0(150)+0.3606(48)}{1+0.5945+0+0.3606} = \frac{98.23}{1.955} = 50.24$ — the edge pixel is ignored.

Domain-only weights 1, 0.6065, 0.6065, 0.3679: $\frac{50+31.54+90.98+17.66}{2.581} = 73.7$ — dragged towards the other side of the edge (blurring).

Problem 21 (NCC vs. raw correlation)

Template $T = [1, 2, 3, 4]$. Patches $P = [12, 14, 16, 18]$ and $Q = [4, 3, 2, 1]$. Compute raw correlation and NCC for each.

Solution

Raw: $T \cdot P = 12 + 28 + 48 + 72 = 160$; $T \cdot Q = 4 + 6 + 6 + 4 = 20$.

NCC: mean-centre $T \rightarrow [-1.5, -0.5, 0.5, 1.5]$, $P \rightarrow [-3, -1, 1, 3] = 2 \times$ that $\Rightarrow$ NCC = +1 (perfect match up to brightness/contrast). $Q \rightarrow [1.5, 0.5, -0.5, -1.5] \Rightarrow$ NCC = -1 (perfect anti-match). Raw correlation mostly reflects brightness; NCC reflects *shape*.

Problem 22 (DFT by hand)

Compute the DFT of $h = [1, 0, -1, 0]$ and interpret it.

Solution

$W = e^{-j\pi/2} = -j$. $H(k) = \sum_x h(x)(-j)^{kx} = 1 - (-j)^{2k} = 1 - (-1)^k$.

$H = [0, 2, 0, 2]$. DC = 0 (zero mean). Energy only at $k = 1$ and $k = 3 (= -1)$: $h(x) = \cos(\pi x/2)$ is a single cosine at frequency 1 cycle per 4 samples, which appears as a symmetric pair. Check with inverse: $h(0) = \frac{1}{4}(0 + 2 + 0 + 2) = 1$. ✓

Problem 23 (Convolution theorem check)

$a = [1, 2, 0, 0]$, $b = [1, 1, 1, 0]$. Compute the circular convolution directly and via DFTs.

Solution

Direct: $(a * b)[n] = \sum_m a[m]b[(n - m) \bmod 4]$: $[1, 1 + 2, 1 + 2, 2] = [1, 3, 3, 2]$.

DFTs: $A = [3, 1 - 2j, -1, 1 + 2j]$, $B = [3, -j, 1, j]$. Product $A \cdot B = [9, -2 - j, -1, -2 + j]$.

Inverse: $x[0] = \frac{1}{4}(9 - 2 - j - 1 - 2 + j) = 1$; similarly $x = [1, 3, 3, 2]$. ✓

A.5 Module 7: Cost, Sampling, Aliasing, Interpolation

Problem 24 (FFT vs. direct convolution cost)

$N = 512$, $k = 15$. Estimate the operation counts of spatial convolution and FFT-based convolution (count forward FFT of image, inverse FFT, ignore constants and the kernel FFT).

Solution

Spatial: $N^2 k^2 = 262,144 \times 225 = 5.9 \times 10^7$.

FFT: one forward + one inverse $\approx 2 N^2 \log_2 N^2 = 2 \times 262,144 \times 18 = 9.4 \times 10^6$ (add N^2 for the product). Roughly 6× cheaper; for $k = 3$ the spatial route (2.4×10^6) would win.

Problem 25 (Alias frequencies)

A signal is sampled at $f_s = 100$ Hz. What frequency is observed for true frequencies 45, 70, 130, 250 Hz? What is the minimum f_s to capture 130 Hz?

Solution

$f_{\text{alias}} = |f - f_s \text{round}(f/f_s)|$. 45 $\rightarrow$ 45 (below Nyquist 50, fine). 70 $\rightarrow |70 - 100| = 30$. 130 $\rightarrow |130 - 100| = 30$. 250 $\rightarrow |250 - 200|$ or $|250 - 300| = 50$. To capture 130 Hz: $f_s > 260$ Hz.

Problem 26 (Wagon wheel)

A wheel with 6 identical spokes is filmed at 30 fps and turns x° per frame. For which per-frame rotations does it appear to go backwards? What is the apparent speed at $50^\circ/\text{frame}$?

Solution

Symmetry period = $360/6 = 60^\circ$. Apparent step = $x - 60 \text{round}(x/60)$. It appears to rotate backwards when $x \bmod 60 \in (30^\circ, 60^\circ)$. At $50^\circ/\text{frame}$: $50 - 60 = -10^\circ/\text{frame} \Rightarrow$ backwards at $10 \times 30 = 300^\circ/\text{s}$. (At exactly $30^\circ/\text{frame}$ the direction is ambiguous; at $60^\circ/\text{frame}$ it appears stationary.)

Problem 27 (Sub-sampling a grating)

An image contains vertical stripes with period 3 pixels. It is sub-sampled by keeping every 2nd column. Is there aliasing? What should the Gaussian pre-filter do?

Solution

Frequency $f = 1/3$ cycles/pixel. After keeping every 2nd column, the new sampling rate is $f_s = 0.5$ samples/original pixel, Nyquist = $0.25 < 1/3 \Rightarrow$ **aliasing**. Apparent frequency $|1/3 - 0.5| = 1/6$ cycles per original pixel, i.e. period 6 original pixels = 3 new pixels, but it is a false pattern. The pre-filter should attenuate everything above 0.25 cycles/pixel (a Gaussian with $\sigma \approx 1$ pixel is typical for $\downarrow 2$), so the stripes blur to near-uniform grey instead of a fake pattern.

Problem 28 (Bilinear interpolation)

Pixels $f(0,0) = 10$, $f(1,0) = 20$, $f(0,1) = 30$, $f(1,1) = 40$ (first index x , second y). Estimate $f(0.25, 0.5)$.

Solution

$\alpha = 0.25$, $\beta = 0.5$. Along x at $y = 0$: $0.75(10) + 0.25(20) = 12.5$; at $y = 1$: $0.75(30) + 0.25(40) = 32.5$. Along y : $0.5(12.5) + 0.5(32.5) = 22.5$.
Weights form: $(0.75)(0.5)10 + (0.25)(0.5)20 + (0.75)(0.5)30 + (0.25)(0.5)40 = 3.75 + 2.5 + 11.25 + 5 = 22.5$. ✓

Problem 29 (Upsampling a 1D signal)

Upsample $f = [2, 6, 4]$ by $r = 2$ using (a) nearest neighbour, (b) linear interpolation (tent kernel), producing values at positions $0, 1, \dots, 4$ (and 5 for NN).

Solution

(a) $[2, 2, 6, 6, 4, 4]$. (b) Samples at $0, 2, 4$; midpoints are averages: $[2, 4, 6, 5, 4]$. Using $g(i) = \sum_k f(k)h(i-2k)$ with $h(x) = \max(0, 1 - |x|/2)$: $g(1) = 2 \cdot \frac{1}{2} + 6 \cdot \frac{1}{2} = 4$, $g(3) = 6 \cdot \frac{1}{2} + 4 \cdot \frac{1}{2} = 5$.
✓

Problem 30 (Gaussian pyramid storage)

An image is 1024×1024 . How many levels until 1×1 , and how many pixels in the whole pyramid?

Solution

Sizes $1024, 512, \dots, 1$: $\log_2 1024 + 1 = 11$ levels. Total = $\sum_{i=0}^{10} 4^i = \frac{4^{11}-1}{3} = 1,398,101 \approx \frac{4}{3} \times 1,048,576$.

PRACTICE B

Multiple-Choice Self-Test

Answer each question before reading the explanation underneath. Questions follow module order; a few mix modules.

B.1 Module 1: Introduction

Q1. Which statement best explains why computer vision is called an *inverse problem*?

- (a) Vision algorithms run backwards in time
- (b) Images are stored upside-down on the sensor
- (c) We must infer 3D scene properties from 2D images produced by an information-losing forward process
- (d) Graphics is harder than vision

Answer: (c). Graphics/physics compute image from scene (forward). Vision must recover scene from image, which is ill-posed (e.g. depth is lost in projection).

Q2. In Adelson's checker-shadow illusion, squares A and B:

- (a) Have the same pixel value, but B is perceived lighter because the brain discounts the shadow
- (b) Differ only in hue
- (c) Are identical only on colour screens
- (d) Have different pixel values; the brain perceives them correctly

Answer: (a). Same intensity; perception estimates reflectance by compensating for illumination. Shows perception $\neq$ measurement.

Q3. Which is *not* listed as a reason vision is hard?

- (a) Computer vision is AI-complete
- (b) Images are always noise-free, so models overfit
- (c) No complete model of the human visual system exists
- (d) High-dimensional data

Answer: (b). Images are noisy and incomplete — that is part of why it is hard.

Q4. The course focuses on which pillar of vision?

- (a) Geometry-based vision
- (b) Computational photography
- (c) Physics-based vision
- (d) Learning-based vision

Answer: (d). Recognition, detection, segmentation, tracking, retrieval — learning-based vision.

Q5. Counting red X's is harder among red O's and green X's than among green X's alone because:

- (a) Green masks red
- (b) There are more letters
- (c) Red is hard to see
- (d) The target is a conjunction of features and requires serial search

Answer: (d). Single-feature targets pop out (parallel); conjunctions need serial attention.

B.2 Module 2: History

Q6. Hubel and Wiesel (1959) showed that visual processing starts with:

- (a) Depth maps (b) Simple structures such as oriented edges
- (c) Colour histograms (d) Whole objects

Answer: (b). Simple and complex cells respond to oriented edges/bars — the inspiration for convolutional filters.

Q7. The first digital image (Kirsch) had resolution:

- (a) 176×176 (b) 512×512
- (c) 256×256 (d) 64×64

Answer: (a). $176 \times 176 = 30,976$ pixels.

Q8. Marr's $2\frac{1}{2}$ -D sketch represents:

- (a) An object-centred volumetric 3D model (b) Edges and bars only
- (c) Visible surfaces, depth and discontinuities in a viewer-centred frame (d) A compressed image

Answer: (c). Primal sketch $\rightarrow 2\frac{1}{2}$ -D sketch (viewer-centred surfaces/depth) $\rightarrow$ 3D model (object-centred).

Q9. Which is called “the original ConvNet”?

- (a) LeNet-5 (b) AlexNet
- (c) Neocognitron (d) Perceptron

Answer: (c). Fukushima's Neocognitron (1980): S-cells (filters) and C-cells (shift tolerance).

Q10. Horn & Schunck's optical flow is formulated as:

- (a) A single convolution (b) A lookup table
- (c) A global energy functional that is minimised (d) A classifier

Answer: (c). Data term (brightness constancy) + smoothness term, minimised globally.

Q11. Canny's edge detector used which mathematical tool to derive the optimal detector?

- (a) Calculus of variations (b) Fourier series
- (c) Graph cuts (d) Backpropagation

Answer: (a). As stated on the slide: calculus of variations to optimise a functional.

Q12. Viola–Jones (2001) is best known for:

- (a) Optical flow (b) Semantic segmentation
- (c) Real-time face detection (d) Image compression

Answer: (c). Haar features + integral image + AdaBoost + cascade.

Q13. Which chronological order is correct?

- (a) VGG $\rightarrow$ AlexNet $\rightarrow$ ResNet (b) AlexNet $\rightarrow$ ImageNet $\rightarrow$ ResNet
- (c) ImageNet $\rightarrow$ AlexNet $\rightarrow$ ResNet (d) ResNet $\rightarrow$ VGG $\rightarrow$ AlexNet

Answer: (c). ImageNet (2010), AlexNet (2012), VGG/Inception (2014), ResNet (2015).

Q14. Geons are part of which theory?

- (a) Constellation models (b) Recognition-by-Components (Biederman)
- (c) Normalised cuts (d) Pictorial structures

Answer: (b). Objects assembled from basic 3D shapes called geons.

B.3 Module 3: Image Formation

Q15. For a Lambertian surface, the reflected intensity depends on:

- (a) Only the viewing direction
- (b) Neither light nor view direction
- (c) The cosine of the angle between normal and viewer
- (d) The cosine of the angle between normal and light direction, not the viewing direction

Answer: (d). $I = \rho E_0 \cos \theta$, θ between $\hat{n}$ and light; independent of viewer.

Q16. Albedo ρ is:

- (a) The refractive index
- (b) The specular exponent
- (c) The fraction of light reflected
- (d) The fraction of light absorbed

Answer: (c). A fraction $1 - \rho$ is absorbed.

Q17. The BRDF describes:

- (a) The noise model of a CCD
- (b) How bright a surface appears from one direction when lit from another
- (c) The spectral sensitivity of a sensor
- (d) The shape of a lens

Answer: (b). Bidirectional Reflectance Distribution Function.

Q18. A Bayer filter has the colour proportions:

- (a) 50% R, 25% G, 25% B
- (b) 25% R, 25% G, 50% B
- (c) 33% R, 33% G, 33% B
- (d) 25% R, 50% G, 25% B

Answer: (d). Twice as many green sites, matching human sensitivity.

Q19. Demosaicing is:

- (a) Removing mosaic artefacts from JPEGs
- (b) Compressing RAW files
- (c) Estimating the two missing colour values at each photosite from neighbours
- (d) Converting RGB to YCbCr

Answer: (c). Each photosite measures one colour; demosaicing interpolates the rest.

Q20. In a CCD sensor:

- (a) No charge is ever generated
- (b) Each pixel has its own amplifier and ADC
- (c) Colour is measured without filters
- (d) Charge is shifted pixel to pixel and converted to voltage at an output node

Answer: (d). That is the CCD bucket-brigade; CMOS converts inside each pixel.

Q21. Increasing analog gain (ISO):

- (a) Collects more photons
- (b) Reduces exposure time automatically
- (c) Amplifies both signal and noise
- (d) Increases bit depth

Answer: (c). Gain is applied after photon collection.

Q22. Fill factor is:

- (a) Active sensing area as a fraction of the available pixel area
- (b) Bits per pixel
- (c) Shutter open fraction
- (d) Number of pixels per mm

Answer: (a). Definition from the slide.

Q23. JPEG compresses chrominance more than luminance because:

- (a) Chrominance has more bits
- (b) Luminance is not needed
- (c) Humans are more sensitive to brightness detail than colour detail
- (d) DCT only works on colour

Answer: (c). Hence YCbCr and chroma subsampling.

Q24. If the MSE between an 8-bit image and its reconstruction is 65.025, PSNR is:

- (a) 30 dB
- (b) 40 dB
- (c) 20 dB
- (d) 65 dB

Answer: (a). $255^2 / 65.025 = 1000$; $10 \log_{10} 1000 = 30$ dB.

Q25. Mixing cyan, magenta and yellow inks ideally gives:

- (a) Black
- (b) Grey 50%
- (c) White
- (d) Red

Answer: (a). Subtractive mixing; CMY absorb R, G, B respectively.

Q26. The DCT is used in JPEG because:

- (a) It removes aliasing
- (b) It is exactly the PCA of every image
- (c) It approximates the eigendecomposition of image patches, compacting energy into few coefficients
- (d) It is non-linear

Answer: (c). A good, image-independent approximation to the KLT/PCA for natural images.

B.4 Module 4: Image Representation

Q27. We use RGB rather than 7 VIBGYOR channels because:

- (a) Sensors only detect three wavelengths
- (b) Screens cannot show violet
- (c) VIBGYOR is not a spectrum
- (d) Humans have three cone types (L, M, S)

Answer: (d). Trichromacy: three numbers suffice to match most perceived colours.

Q28. Why are men more likely to be colour blind?

- (a) Their rods are weaker
- (b) The M and L pigment genes lie on the X chromosome
- (c) They have fewer cones
- (d) Testosterone affects cones

Answer: (b). Men have a single X, so a defective gene is not compensated.

Q29. A digital image is a version of $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ that is:

- (a) Sampled only
- (b) Sampled and quantised
- (c) Quantised only
- (d) Neither

Answer: (b). Domain discretised (sampling) and range discretised (quantisation).

Q30. Per-pixel cost of a local operation with a $p \times p$ neighbourhood is:

- (a) $O(1)$
- (b) $O(p)$
- (c) $O(N^2)$
- (d) $O(p^2)$

Answer: (d). Point $O(1)$, local $O(p^2)$, global $O(N^2)$.

Q31. For 8-bit images with $I_{\min} = 0$, contrast reversal maps 40 to:

- (a) -40
- (b) 215
- (c) 255
- (d) 40

Answer: (b). $255 - 40 + 0 = 215$.

Q32. An image spans $[60, 180]$. After linear stretching to $[0, 255]$, the value 120 becomes:

- (a) 255
- (b) 127.5
- (c) 120
- (d) 160

Answer: (b). $(120 - 60) \times 255 / 120 = 127.5$ — the midpoint maps to the midpoint.

Q33. Histogram equalisation uses as its transfer function:

- (a) The (scaled) cumulative distribution function
- (b) The histogram itself
- (c) A logarithm
- (d) A Gaussian

Answer: (a). $T(r) = I_{\max} F_r(r)$, which makes the output approximately uniform.

Q34. Averaging 25 frames of a static scene with i.i.d. noise reduces the noise standard deviation by a factor of:

- (a) $\sqrt{5}$
- (b) 25
- (c) 5
- (d) 12.5

Answer: (c). Variance σ^2/K , std $\sigma/\sqrt{K}$, $\sqrt{25} = 5$.

Q35. The Fourier transform of an image is an example of a:

- (a) Point operation
- (b) Local operation
- (c) Non-linear operation
- (d) Global operation

Answer: (d). Each coefficient depends on all pixels.

B.5 Module 5: Linear Filtering

Q36. The kernel for a 3×3 moving average is:

- (a) $[-1, 0, 1]$
- (b) All $1/9$
- (c) $1/16[1, 2, 1; 2, 4, 2; 1, 2, 1]$
- (d) All ones

Answer: (b). Uniform weights summing to 1.

Q37. Convolution differs from cross-correlation in that convolution:

- (a) Uses only symmetric kernels
- (b) Uses absolute values
- (c) Normalises by the patch norm
- (d) Flips the kernel in both directions before sliding

Answer: (d). $G = \sum H(u, v)I(i - u, j - v)$ = correlation with the 180° -rotated kernel.

Q38. Correlating an impulse image with kernel H yields:

- (a) $H^\top$
- (b) H
- (c) A constant
- (d) H rotated by 180°

Answer: (d). $G(i, j) = H(-i, -j)$; convolution would return H itself.

Q39. For which kernel do correlation and convolution give identical results?

- (a) Gaussian
- (b) Sobel x
- (c) $[1, 2, 3]$
- (d) $[-1, 0, 1]$

Answer: (a). Kernels symmetric under 180° rotation.

Q40. Which property holds for convolution but not in general for cross-correlation?

- (a) Commutativity/associativity
- (b) Shift invariance
- (c) Linearity
- (d) Output size

Answer: (a). Both are LSI; only convolution is commutative and associative.

Q41. Shift invariance is desirable in vision because:

- (a) The same feature should be detected the same way anywhere in the image
- (b) It removes noise
- (c) It makes filters non-linear
- (d) It reduces memory

Answer: (a). "The effect of the operator is the same everywhere" — basis of CNN weight sharing.

Q42. Applying filters b_1, b_2, b_3 in sequence equals applying $b_1 * b_2 * b_3$ once, because of:

- (a) Associativity
- (b) Commutativity
- (c) Identity
- (d) Distributivity

Answer: (a). $((a * b_1) * b_2) * b_3 = a * (b_1 * b_2 * b_3)$.

Q43. A $k \times k$ separable kernel reduces per-pixel cost from k^2 to:

- (a) $2k$
- (b) k
- (c) $k^2/2$
- (d) $\log k$

Answer: (a). One 1D pass of length k in each direction.

Q44. Using SVD, a kernel K is separable iff:

- (a) All singular values are equal
- (b) Exactly one singular value is non-zero
- (c) Its determinant is 1
- (d) It is square

Answer: (b). Rank 1: $K = \sigma_1 u_1 v_1^\top$; kernels $\sqrt{\sigma_1} u_1$ and $\sqrt{\sigma_1} v_1$.

Q45. Which of these is *not* separable?

- (a) Sobel
- (b) Gaussian
- (c) Box filter
- (d) Laplacian $[0, 1, 0; 1, -4, 1; 0, 1, 0]$

Answer: (d). Rank 2 (singular values $\approx 4.45, 0.45, 0$).

Q46. Increasing the filter size (for smoothing) does NOT:

- (a) Increase blurring
- (b) Increase computation
- (c) Reduce output noise variance
- (d) Reduce the spread of noise

Answer: (d). A bigger mask spreads each noisy pixel over a *bigger* area.

Q47. With input 32×32 , kernel 5×5 , padding 0, stride 1, the output is:

- (a) 28×28
- (b) 27×27
- (c) 30×30
- (d) 32×32

Answer: (a). $32 - 5 + 1 = 28$.

Q48. Which padding treats the image as periodic?

- (a) Clamp
- (b) Zero
- (c) Wrap
- (d) Mirror

Answer: (c). Wrap-around, matching the DFT's circular convolution.

Q49. An edge (derivative) kernel's coefficients typically:

- (a) Sum to 0
- (b) Sum to 1
- (c) Are all equal
- (d) Are all positive

Answer: (a). Flat regions give zero response; smoothing kernels sum to 1.

B.6 Module 6: Frequency Domain and Non-linear Filters

Q50. Normalised cross-correlation is preferred for template matching because it:

- (a) Ignores the template
- (b) Is invariant to brightness/contrast changes of the patch
- (c) Is non-linear in the template
- (d) Is faster

Answer: (b). Mean-centring and dividing by norms makes it invariant to $I \rightarrow aI + b, a > 0$.

Q51. The median filter is best suited for:

- (a) Salt-and-pepper noise
- (b) Gaussian noise
- (c) Aliasing
- (d) Motion blur

Answer: (a). Robust to outliers, does not spread spikes; not ideal for Gaussian noise.

Q52. The median of $\{3, 90, 4, 5, 2, 4, 3, 4, 5\}$ is:

- (a) 3
- (b) 4
- (c) 5
- (d) 13.3

Answer: (b). Sorted: 2,3,3,4,4,4,5,5,90.

Q53. In the bilateral filter, the range kernel:

- (a) Is always uniform
- (b) Down-weights pixels whose intensity differs a lot from the centre
- (c) Makes the filter linear
- (d) Weights pixels by spatial distance

Answer: (b). Domain = spatial closeness; range = intensity similarity. The latter preserves edges.

Q54. As $\sigma_r \rightarrow \infty$, the bilateral filter becomes:

- (a) Median filter
- (b) Gaussian blur
- (c) Edge detector
- (d) Identity

Answer: (b). Range weights all $\rightarrow 1$, leaving only the spatial Gaussian.

Q55. A square wave's Fourier series contains:

- (a) All harmonics equally (b) A single frequency
 (c) Only odd harmonics with amplitudes $\propto 1/k$ (d) Only even harmonics

Answer: (c). $\frac{4}{\pi} \sum_{k \text{ odd}} \frac{1}{k} \sin(2\pi kt)$.

Q56. In an image's centred Fourier magnitude, the centre represents:

- (a) The edges (b) The highest frequency
 (c) The phase (d) The DC (average) component

Answer: (d). Zero frequency is at the centre; radius increases with frequency.

Q57. The magnitude spectrum of a real image is symmetric because:

- (a) Of the window used (b) Images are square
 (c) The FFT forces it (d) $F(-\omega) = \overline{F(\omega)}$ for real signals

Answer: (d). Conjugate symmetry $\Rightarrow |F(-\omega)| = |F(\omega)|$.

Q58. Swapping phase between two images and reconstructing, the results look like:

- (a) Pure noise (b) The image that provided the phase
 (c) An average of both (d) The image that provided the magnitude

Answer: (b). Phase carries the spatial structure.

Q59. Translating an image changes its Fourier transform's:

- (a) Neither (b) Phase only
 (c) Magnitude only (d) Both

Answer: (b). $f(x - x_0) \leftrightarrow F(\omega)e^{-j\omega x_0}$; $|e^{-j\omega x_0}| = 1$.

Q60. The convolution theorem states:

- (a) $\mathcal{F}[g * h] = \mathcal{F}[g] \mathcal{F}[h]$ (b) $\mathcal{F}[g * h] = \mathcal{F}[g] + \mathcal{F}[h]$
 (c) $\mathcal{F}[gh] = \mathcal{F}[g] \mathcal{F}[h]$ (d) $\mathcal{F}[g * h] = \mathcal{F}[g] / \mathcal{F}[h]$

Answer: (a). Convolution $\leftrightarrow$ multiplication.

Q61. Direct DFT of N points costs $O(N^2)$; the FFT costs:

- (a) $O(N \log N)$ (b) $O(N^{1.5})$
 (c) $O(\log N)$ (d) $O(N)$

Answer: (a). Divide and conquer: $T(N) = 2T(N/2) + O(N)$.

Q62. A Gaussian filter is a:

- (a) High-pass filter (b) Band-stop filter
 (c) All-pass filter (d) Low-pass filter

Answer: (d). Its FT is a Gaussian centred at zero frequency.

Q63. Why does the box filter produce ringing artefacts that the Gaussian avoids?

- (a) Its Fourier transform is a sinc with negative side lobes (b) It is non-linear
 (c) It is not separable (d) It is larger

Answer: (a). The sinc passes some high frequencies with alternating sign.

Q64. A hybrid image combines:

- (a) Low frequencies of one image with high frequencies of another (b) Two resolutions of the same image
 (c) Two RGB channels (d) Two images' phases

Answer: (a). Near: see the high-frequency image; far: see the low-frequency image.

Q65. An image of a single blurry white blob has a Fourier magnitude that looks like:

- (a) A small central blob (b) A cross
 (c) Uniform noise (d) Two horizontal lobes

Answer: (a). Smooth image $\Rightarrow$ only low frequencies (matching exercise 1 $\rightarrow$ D).

B.7 Module 7: Sampling and Interpolation

Q66. Cost of convolving an $N \times N$ image with a $k \times k$ kernel in the Fourier domain is approximately:

- (a) $O(N^2 k^2)$
- (b) $O(N^2 \log N^2 + k^2 \log k^2)$
- (c) $O(k^2)$
- (d) $O(N^4)$

Answer: (b). As on the slide.

Q67. Shannon's sampling theorem requires:

- (a) $f_s = f_{\max}/2$
- (b) $f_s \geq 2f_{\max}$
- (c) $f_s \leq 2f_{\max}$
- (d) $f_s \geq f_{\max}$

Answer: (b). Otherwise aliasing occurs.

Q68. A 70 Hz sinusoid sampled at 100 Hz appears as:

- (a) 30 Hz
- (b) 170 Hz
- (c) 70 Hz
- (d) 100 Hz

Answer: (a). $|70 - 100| = 30$.

Q69. The wagon-wheel effect is an example of:

- (a) Quantisation
- (b) Temporal aliasing
- (c) Motion blur
- (d) Spatial aliasing

Answer: (b). Frame rate too low for the rotation frequency.

Q70. The standard anti-aliasing step before sub-sampling is:

- (a) Sharpening
- (b) Median filtering
- (c) Gaussian (low-pass) pre-filtering
- (d) Histogram equalisation

Answer: (c). Remove frequencies above the new Nyquist limit first.

Q71. Nearest-neighbour interpolation corresponds to convolving with a:

- (a) Sinc kernel
- (b) Tent kernel
- (c) Box kernel
- (d) Gaussian kernel

Answer: (c). Box $\rightarrow$ piecewise constant; tent $\rightarrow$ linear; sinc $\rightarrow$ ideal.

Q72. The ideal reconstruction kernel for a band-limited signal is:

- (a) Box
- (b) Tent
- (c) Sinc
- (d) Cubic

Answer: (c). Sinc = ideal low-pass filter in frequency; infinitely wide in space.

Q73. Bicubic interpolation in 2D uses how many neighbouring pixels?

- (a) 1
- (b) 16
- (c) 9
- (d) 4

Answer: (b). 4×4 ; bilinear uses 2×2 , nearest uses 1.

Q74. In $g(i, j) = \sum_{k,l} f(k, l)h(i - rk, j - rl)$, r is:

- (a) The number of channels
- (b) The kernel radius
- (c) The noise level
- (d) The upsampling rate

Answer: (d). Input sample (k, l) is placed at output position (rk, rl) .

Q75. Bilinear interpolation at the exact centre of four pixels 10, 20, 30, 40 gives:

- (a) 30
- (b) 25
- (c) 40
- (d) 20

Answer: (b). All weights $\frac{1}{4}$: $(10 + 20 + 30 + 40)/4 = 25$.

Q76. Throwing away every other row/column of a finely striped image without pre-filtering most likely produces:

- (a) Moiré / false patterns
- (b) A colour shift only
- (c) A brighter image
- (d) A perfect half-size image

Answer: (a). Aliasing: high frequencies fold into false low frequencies.

Formula Sheet

Image formation

- Lambert: $I = \rho E_0 \max(0, \hat{\mathbf{n}} \cdot \hat{\mathbf{s}})$; absorbed $1 - \rho$.
- BRDF: $f_r = dL_r / dE_i$; Lambertian $f_r = \rho / \pi$.
- Sensor channel: $R_k = \int L(\lambda) \rho(\lambda) S_k(\lambda) d\lambda$.
- Bayer: 50% G, 25% R, 25% B.
- Fill factor = $\frac{a_x a_y}{p_x p_y}$; b bits $\Rightarrow 2^b$ levels, +6.02 dB/bit.
- Shannon: $f_s \geq 2f_{\max}$.
- YCbCr: $Y = 0.299R + 0.587G + 0.114B$.
- CMY = 1 - RGB.
- MSE = $\frac{1}{n} \sum (I - \hat{I})^2$; PSNR = $10 \log_{10} \frac{I_{\max}^2}{\text{MSE}}$.

Image representation / point ops

- Digital: $F[i, j] = \text{quantize}\{f(i\Delta, j\Delta)\}$.
- Costs/pixel: point $O(1)$, local $O(p^2)$, global $O(N^2)$.
- Negative: $\hat{I} = I_{\max} + I_{\min} - I$.
- Stretch: $\hat{I} = \frac{I - \min I}{\max I - \min I} (I_{\max} - I_{\min}) + I_{\min}$.
- Hist. eq.: $c_k = \frac{1}{MN} \sum_{l \leq k} h(l)$, $\hat{I} = I_{\max} c_I$.
- Averaging K frames: $\sigma \rightarrow \sigma / \sqrt{K}$.

Filtering

- Correlation: $G = \sum_{u,v} H(u,v) I(i+u, j+v)$, $G = H \otimes I$.
- Convolution: $G = \sum_{u,v} H(u,v) I(i-u, j-v)$, $G = H * I$.
- Impulse: $\delta \otimes H = H(-i, -j)$; $\delta * H = H$.
- Gaussian: $h = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/2\sigma^2}$; width $\approx 2\lceil 3\sigma \rceil + 1$.
- Binomial: $\frac{1}{16} [1, 2, 1]^T [1, 2, 1]$; Sobel $\frac{1}{4} [1, 2, 1]^T \cdot \frac{1}{2} [-1, 0, 1]$.
- Separable: $K = \mathbf{v}\mathbf{h}^T$ (rank 1); cost $2k$ vs k^2 .
- SVD: $K = \sum \sigma_i \mathbf{u}_i \mathbf{v}_i^T$; kernels $\sqrt{\sigma_1} \mathbf{u}_1, \sqrt{\sigma_1} \mathbf{v}_1$.
- Noise after filter: $\sigma^2 \sum H^2$.
- Output size: $\lfloor (N + 2p - k) / s \rfloor + 1$; same: $p = (k - 1) / 2$.
- Convolution: commutative, associative, distributive, scalars factor, identity δ .

Non-linear filters & matching

- NCC = $\frac{\sum (T - \bar{T})(P - \bar{P})}{\|T - \bar{T}\| \|P - \bar{P}\|} \in [-1, 1]$.
- Median: minimises $\sum |x_k - m|$.
- Bilateral: $g = \frac{\sum I(k,l)w}{\sum w}$,
 $w = \exp\left(-\frac{(i-k)^2 + (j-l)^2}{2\sigma_d^2} - \frac{\|I(i,j) - I(k,l)\|^2}{2\sigma_r^2}\right)$.

Fourier

- Building block $A \sin(\omega x + \phi)$.
- Square wave = $\frac{4}{\pi} \sum_{k \text{ odd}} \frac{1}{k} \sin(2\pi k t)$.
- FT: $H(\omega) = \int h(x) e^{-j\omega x} dx$.
- DFT: $H(k) = \sum_{x=0}^{N-1} h(x) e^{-j2\pi k x / N}$;
 IDFT: $h(x) = \frac{1}{N} \sum_k H(k) e^{j2\pi k x / N}$.
- $A = |\varphi| = \sqrt{\text{Re}^2 + \text{Im}^2}$, $\phi = \text{atan2}(\text{Im}, \text{Re})$.
- $\mathcal{F}[g * h] = \mathcal{F}[g] \mathcal{F}[h]$; $g * h = \mathcal{F}^{-1}[\mathcal{F}[g] \mathcal{F}[h]]$.
- Shift: $f(x - x_0) \leftrightarrow F e^{-j\omega x_0}$; derivative $\leftrightarrow j\omega F$;
 $f(ax) \leftrightarrow \frac{1}{|a|} F(\omega/a)$.
- Real f : $F(-\omega) = \overline{F(\omega)}$.
- Gaussian $\leftrightarrow$ Gaussian ($\sigma \leftrightarrow 1/\sigma$); box $\leftrightarrow$ sinc.
- DFT $O(N^2)$; FFT $O(N \log N)$.

Sampling & interpolation

- Spatial conv $O(N^2 k^2)$ vs FFT $O(N^2 \log N^2 + k^2 \log k^2)$.
- Alias: $f_{\text{alias}} = |f - f_s \text{round}(f/f_s)|$.
- Anti-alias: Gaussian blur, then $\downarrow r$.
- Reconstruct: $\hat{f} = h * f_F$.
- Interpolation: $g(i, j) = \sum f(k, l) h(i - rk, j - rl)$.
- Decimation: $g(i, j) = \sum f(k, l) h(i - \frac{k}{r}, j - \frac{l}{r})$ [slide]; std. $\sum f(k, l) h(ri - k, rj - l)$.
- Bilinear: $(1 - \alpha)(1 - \beta)f_{00} + \alpha(1 - \beta)f_{10} + (1 - \alpha)\beta f_{01} + \alpha\beta f_{11}$.
- Kernels: box = NN, tent = linear, cubic = bicubic, sinc = ideal.